

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HCM
KHOA ĐIỆN - ĐIỆN TỬ



HCMUTE

ĐỒ ÁN 1
THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG QUAN TRẮC THỜI TIẾT
CỤC BỘ ỨNG DỤNG TRUYỀN THÔNG LORA, MẬT MÃ HẠNG
NHẸ VÀ HỒI QUY TUYẾN TÍNH

SVTH: Nguyễn Thái Anh – 23139002
Lê Đức Trí – 23139049

GVHD: TS. Huỳnh Thế Thiện

Thành phố Hồ Chí Minh, tháng 06 năm 2026

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HCM
KHOA ĐIỆN - ĐIỆN TỬ



HCMUTE

ĐỒ ÁN 1
THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG QUAN TRẮC THỜI TIẾT
CỤC BỘ ỨNG DỤNG TRUYỀN THÔNG LORA, MẬT MÃ HẠNG
NHẸ VÀ HỒI QUY TUYẾN TÍNH

SVTH: Nguyễn Thái Anh – 23139002
Lê Đức Trí – 23139049

GVHD: TS. Huỳnh Thế Thiện

Thành phố Hồ Chí Minh, tháng 06 năm 2026

LỜI CAM ĐOAN

Chúng tôi xin cam đoan đồ án với đề tài "Thiết kế và triển khai hệ thống quan trắc thời tiết cục bộ ứng dụng truyền thông LoRa, mật mã hạng nhẹ và hồi quy tuyến tính" là công trình nghiên cứu, tìm hiểu và thực hiện của riêng nhóm, dưới sự hướng dẫn khoa học của TS. Huỳnh Thế Thiện.

Các số liệu, kết quả trình bày trong báo cáo là hoàn toàn trung thực, được thu thập từ quá trình thực nghiệm thực tế và chưa từng được công bố trong bất kỳ công trình nào khác. Mọi thông tin tham khảo từ các tài liệu, bài báo khoa học, hoặc mã nguồn mở đều được trích dẫn nguồn gốc rõ ràng và tuân thủ đúng quy định về sở hữu trí tuệ.

Nếu có bất kỳ sự gian lận hay sao chép trái phép nào, chúng tôi xin chịu hoàn toàn trách nhiệm trước Hội đồng đánh giá và nhà trường.

Thành phố Hồ Chí Minh, ngày ... tháng 06 năm 2026

Nhóm sinh viên thực hiện

(Ký và ghi rõ họ tên)

Nguyễn Thái Anh

Lê Đức Trí

LỜI CẢM ƠN

Lời đầu tiên, chúng tôi xin gửi lời cảm ơn chân thành đến Ban Giám hiệu, cùng toàn thể quý thầy cô Khoa Điện - Điện tử, Trường Đại học Công nghệ Kỹ thuật TP. HCM đã tạo môi trường học tập tốt nhất, truyền đạt những kiến thức quý báu và nền tảng chuyên môn vững chắc trong suốt quá trình chúng tôi rèn luyện tại trường.

Đặc biệt, chúng tôi xin bày tỏ lòng biết ơn sâu sắc đến TS. Huỳnh Thế Thiện – người đã trực tiếp hướng dẫn, tận tình chỉ bảo, định hướng nghiên cứu và giải đáp những vướng mắc của nhóm trong suốt thời gian thực hiện đề án. Những lời khuyên và kinh nghiệm thực tiễn từ thầy là nguồn động lực to lớn giúp chúng tôi vượt qua khó khăn, hoàn thiện đề tài đúng tiến độ và đạt được kết quả như mong muốn.

Mặc dù đã cố gắng hết sức để hoàn thiện đề án, nhưng do giới hạn về mặt thời gian và kinh nghiệm thực tiễn, thiếu sót là điều không thể tránh khỏi. Chúng tôi rất mong nhận được sự góp ý, chỉ bảo thêm từ quý thầy cô trong Hội đồng đánh giá đề tài được hoàn thiện hơn và làm hành trang quý giá cho công việc thực tế sau này.

Chúng tôi xin chân thành cảm ơn!

MỤC LỤC

MỤC LỤC	5
DANH MỤC TỪ VIẾT TẮT	8
DANH MỤC HÌNH ẢNH	11
DANH MỤC BẢNG.....	13
MỞ ĐẦU	14
1. Lý do chọn đề tài	14
2. Mục tiêu đề tài	15
3. Đối tượng và phạm vi nghiên cứu	15
4. Phương pháp thực hiện.....	16
5. Giới hạn nghiên cứu	17
6. Bố cục báo cáo	18
CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI.....	19
1.1. Tổng quan về quan trắc thời tiết cục bộ	19
1.2. Vai trò của IoT trong quan trắc môi trường	19
1.3. Bài toán thu thập dữ liệu vi khí hậu	20
1.4. Vấn đề truyền dữ liệu không dây trong hệ thống IoT	21
1.5. Vấn đề bảo mật dữ liệu cảm biến	21
1.6. Vai trò của học máy trong dự báo xu hướng nhiệt độ cục bộ	22
1.7. Giải pháp đề xuất của đề tài	23
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT	25
2.1. Tổng quan về kiến trúc IoT	25
2.2. Vi điều khiển STM32F103C8T6.....	26
2.3. ESP32-S3 và FreeRTOS	28
2.4. Chuẩn giao tiếp I2C.....	29
2.5. Chuẩn giao tiếp UART.....	31
2.6. Cảm biến nhiệt độ và độ ẩm SHT30	33
2.7. Cảm biến áp suất khí quyển BMP388.....	34
2.8. Cảm biến giám sát nguồn INA219	35
2.9. Module truyền thông LoRa E32.....	37

2.10. Công nghệ quản lý nguồn và lưu trữ năng lượng.....	38
2.10.1. Module sạc pin năng lượng mặt trời CN3791	39
2.10.2. Pin Lithium EVE và mạch bảo vệ 1S	40
2.11. Kết cấu cơ khí và bảo vệ thiết bị ngoài trời	41
2.11.1. Vỏ hộp chống nước IP67 và van thở cân bằng áp suất.....	41
2.11.2. Lòng Stevenson trong quan trắc khí tượng.....	42
2.12. Thuật toán ASCON-128a AEAD	43
2.13. Giao thức HTTPS trong truyền dữ liệu IoT	45
2.14. Supabase trong lưu trữ và xử lý dữ liệu IoT	46
2.15. BLE provisioning và OTA firmware update.....	47
2.16. Multiple Linear Regression trong dự báo nhiệt độ cục bộ ngắn hạn	49
2.16.1. Cơ sở của mô hình Multiple Linear Regression trong dự báo nhiệt độ.....	50
2.16.2. Mô hình MLR cơ bản với các biến môi trường.....	50
2.16.3. Vai trò của yếu tố thời gian trong dự báo nhiệt độ cục bộ	51
2.16.4. Mã hóa đặc trưng thời gian chu kỳ	52
2.16.5. Dạng mô hình MLR mở rộng áp dụng trong đề tài	53
CHƯƠNG 3. PHÂN TÍCH YÊU CẦU VÀ THIẾT KẾ HỆ THỐNG	55
3.1. Định hướng thiết kế hệ thống.....	55
3.2. Yêu cầu hệ thống.....	56
3.3. Kiến trúc tổng thể và luồng hoạt động của hệ thống.....	58
3.4. Thiết kế STM32 Sensor Node	63
3.5. Thiết kế cấu trúc payload và frame dữ liệu LoRa	66
3.6. Thiết kế cơ chế bảo mật hai tuyến bằng ASCON-128a	71
3.7. Thiết kế ESP32-S3 Gateway	74
CHƯƠNG 4. TRIỂN KHAI HỆ THỐNG	77
4.1. Môi trường và phạm vi triển khai	77
4.2. Triển khai trạm đo STM32 Sensor Node	78
4.2.1. Phần cứng trạm đo	78
4.2.2. Firmware trạm đo	83
4.3. Triển khai ESP32-S3 Gateway.....	85
4.3.1. Phần cứng Gateway	85

4.3.2. Firmware Gateway	89
4.3.3. Triển khai mô hình Multiple Linear Regression trên Gateway	93
4.4. Triển khai Supabase, Web Dashboard và OTA	96
4.4.1. Supabase Cloud	96
4.4.2. Web Dashboard và OTA firmware update	97
4.5. Tích hợp và vận hành hệ thống sau triển khai.....	100
CHƯƠNG 5. THỰC NGHIỆM, KẾT QUẢ VÀ ĐÁNH GIÁ	102
5.1. Kết quả vận hành STM32 Sensor Node	102
5.2. Kết quả truyền dữ liệu LoRa và xác thực ASCON	104
5.3. Kết quả xử lý dữ liệu tại ESP32-S3 Gateway	106
5.4. Kết quả lưu trữ và hiển thị trên Supabase/Web Dashboard/App	107
5.5. Đánh giá độ trễ truyền dữ liệu HTTPS	112
5.6. Đánh giá mô hình dự báo Multiple Linear Regression	114
5.7. Kết quả cập nhật firmware OTA	118
5.8. Đánh giá chung và hạn chế	120
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	122
6.1. Kết luận	122
6.2. Hướng phát triển	123
TÀI LIỆU THAM KHẢO	125

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Tên tiếng Anh đầy đủ	Giải nghĩa tiếng Việt
ADC	Analog-to-Digital Converter	Bộ chuyển đổi tín hiệu tương tự sang số
AEAD	Authenticated Encryption with Associated Data	Mã hóa xác thực có dữ liệu liên kết
API	Application Programming Interface	Giao diện lập trình ứng dụng
BLE	Bluetooth Low Energy	Bluetooth năng lượng thấp
CRC	Cyclic Redundancy Check	Kiểm tra độ dư vòng (Kiểm tra lỗi)
GPIO	General-Purpose Input/Output	Chân tín hiệu vào/ra đa dụng
HTTP	Hypertext Transfer Protocol	Giao thức truyền tải siêu văn bản
HTTPS	Hypertext Transfer Protocol Secure	Giao thức truyền tải siêu văn bản bảo mật
I2C	Inter-Integrated Circuit	Chuẩn giao tiếp dữ liệu nối tiếp đồng bộ

IoT	Internet of Things	Internet vạn vật
LoRa	Long Range	Công nghệ truyền thông vô tuyến tầm xa
MLR	Multiple Linear Regression	Hồi quy tuyến tính đa biến
NIST	National Institute of Standards and Technology	Viện Tiêu chuẩn và Công nghệ Quốc gia Hoa Kỳ
NVS	Non-Volatile Storage	Bộ nhớ lưu trữ không bay hơi
OTA	Over-The-Air	Cập nhật phần mềm (firmware) qua mạng không dây
REST	Representational State Transfer	Kiểu kiến trúc phần mềm cho các hệ thống phân tán
RSSI	Received Signal Strength Indicator	Chỉ số cường độ tín hiệu nhận
RTC	Real-Time Clock	Đồng hồ thời gian thực
RTOS	Real-Time Operating System	Hệ điều hành thời gian thực
SHA	Secure Hash Algorithm	Thuật toán băm an toàn

TLS	Transport Layer Security	Giao thức bảo mật tầng truyền tải
UART	Universal Asynchronous Receiver-Transmitter	Khối truyền nhận dữ liệu nối tiếp bất đồng bộ

DANH MỤC HÌNH ẢNH

Hình 2.1. Kiến trúc tổng thể hệ thống quan trắc thời tiết cục bộ IoT	26
Hình 2.2. Vi điều khiển STM32F103C8T6	27
Hình 2.3. Vi điều khiển ESP32-S3 N16R8.....	28
Hình 2.4. Cấu trúc bus I2C giữa Master và các Slave	30
Hình 2.5. Trình tự truyền nhận dữ liệu trên bus I2C	31
Hình 2.6. Sơ đồ kết nối giao tiếp UART	32
Hình 2.7. Cảm biến nhiệt độ - độ ẩm SHT30	33
Hình 2.8. Cảm biến áp suất - nhiệt độ BMP388	34
Hình 2.9. Cảm biến đo dòng - áp INA219	36
Hình 2.10. Module Lora E32-433Mhz 1W.....	37
Hình 2.11. Module CN3791 6V.....	39
Hình 2.12. Pin EVE 18650 25P 2500mAh 3.7V 12C Lithium-ion	40
Hình 2.13. Mạch Sạc Và Bảo Vệ Pin 1S 3.7V 4A.....	40
Hình 2.14. Van thở Nylon M12x1.5, IP68.....	42
Hình 3.1. Sơ đồ khối chức năng của hệ thống quan trắc thời tiết cục bộ	58
Hình 3.2. Lưu đồ hoạt động của STM32 Sensor Node.....	60
Hình 3.3. Lưu đồ hoạt động của ESP32-S3 Gateway và khối lưu trữ.....	62
Hình 3.4. Sơ đồ khối phần cứng STM32 Sensor Node.....	63
Hình 3.5. Biểu đồ time-slice trong một chu kỳ đo và truyền dữ liệu của STM32.....	66
Hình 3.6. Cấu trúc payload cảm biến 10 byte.....	69
Hình 3.7. Cấu trúc frame LoRa 18 byte.....	70
Hình 4.1. Schematic tổng thể trạm đo STM32 Sensor Node.....	79
Hình 4.2. PCB layout và mô hình 3D của trạm đo STM32 Sensor Node	81
Hình 4.3. Trạm đo STM32 Sensor Node sau khi lắp ráp trong vỏ bảo vệ.....	83
Hình 4.4. Schematic tổng thể ESP32-S3 Gateway	86
Hình 4.5. PCB layout và mô hình 3D của ESP32-S3 Gateway	88
Hình 4.6. ESP32-S3 Gateway sau khi lắp ráp	89
Hình 4.7. Luồng triển khai Supabase Cloud	97
Hình 4.8. Luồng cập nhật firmware OTA cho ESP32-S3 Gateway	100

Hình 4.9. Luồng tích hợp vận hành hệ thống sau triển khai	101
Hình 5.1. Log STM32 Sensor Node trong một chu kỳ đo và truyền dữ liệu.....	103
Hình 5.2. Payload 10 byte được tạo tại STM32 Sensor Node	103
Hình 5.3. Frame LoRa sau khi mã hóa tại STM32 Sensor Node.....	103
Hình 5.4. Trạng thái chuyển đổi mode PS/SYNC	104
Hình 5.5. Log ESP32-S3 Gateway nhận đủ frame LoRa 18 byte.....	105
Hình 5.6. Log xác thực và giải mã frame LoRa bằng ASCON key1.....	105
Hình 5.7. Log khôi phục dữ liệu cảm biến tại ESP32-S3 Gateway	106
Hình 5.8. Log tạo giá trị predicted_temp_2h tại ESP32-S3 Gateway	107
Hình 5.9. Log tạo payload_key2 trước khi gửi Supabase.....	107
Hình 5.10. Bản ghi dữ liệu cảm biến trên bảng weather_logs	108
Hình 5.11. Bản ghi trạng thái ESP32-S3 Gateway trên Supabase.....	109
Hình 5.12. Web Dashboard hiển thị dữ liệu quan trắc mới nhất và biểu đồ lịch sử.....	110
Hình 5.13. App giám sát hiển thị dữ liệu quan trắc từ Supabase.....	111
Hình 5.14. Trạng thái Gateway và tuyến LoRa trên Web Dashboard	111
Hình 5.15. Log đo độ trễ HTTPS từ ESP32-S3 Gateway đến Supabase.....	112
Hình 5.16. Gói HTTPS đến Supabase được quan sát bằng Wireshark.....	113
Hình 5.17. Thời gian phản hồi request Supabase trên Chrome DevTools.....	113
Hình 5.18. So sánh nhiệt độ thực tế và nhiệt độ dự báo của mô hình MLR đang triển khai.....	116
Hình 5.19. Tương quan giữa nhiệt độ thực tế và nhiệt độ dự báo	117
Hình 5.20. Sai số dự báo của mô hình MLR theo thời gian	117
Hình 5.21. Web Dashboard upload firmware OTA lên Supabase Storage.....	119
Hình 5.22. Log ESP32-S3 Gateway cập nhật OTA thành công	120
Hình 5.23. Log Gateway bỏ qua OTA khi firmware đã được áp dụng trước đó	120

DANH MỤC BẢNG

Bảng 4.1. Công cụ và nền tảng triển khai hệ thống	78
Bảng 5.1. Kết quả đo độ trễ truyền dữ liệu HTTPS.....	114
Bảng 5.2. Kết quả đánh giá sai số của mô hình MLR đang triển khai trên ESP32-S3 Gateway	115
Bảng 5.3. Một số mẫu đại diện đối chiếu giữa nhiệt độ dự báo và nhiệt độ thực tế sau 2 giờ ...	116

MỞ ĐẦU

1. Lý do chọn đề tài

Trong những năm gần đây, các hệ thống quan trắc môi trường và thời tiết ngày càng được ứng dụng rộng rãi trong nhiều lĩnh vực như nông nghiệp thông minh, nhà kính, nhà thông minh, giám sát đô thị và nghiên cứu môi trường. Các thông số như nhiệt độ, độ ẩm, áp suất khí quyển và trạng thái nguồn cấp có vai trò quan trọng trong việc theo dõi điều kiện môi trường, đánh giá xu hướng thay đổi và hỗ trợ quá trình ra quyết định trong các hệ thống tự động.

Tuy nhiên, phần lớn dữ liệu thời tiết hiện nay thường được cung cấp bởi các trạm khí tượng ở phạm vi rộng, chẳng hạn như cấp tỉnh, thành phố hoặc khu vực địa lý lớn. Dữ liệu này có giá trị tham khảo tổng quát nhưng chưa phản ánh đầy đủ sự thay đổi môi trường tại từng vị trí cụ thể. Trên thực tế, các yếu tố như mật độ cây xanh, bề mặt bê tông, độ cao, hướng gió, điều kiện che chắn và vị trí lắp đặt có thể tạo ra sự khác biệt đáng kể về nhiệt độ, độ ẩm và áp suất khí quyển giữa các khu vực gần nhau. Hiện tượng này thường được gọi là vi khí hậu.

Do đó, việc xây dựng một hệ thống quan trắc thời tiết cục bộ có khả năng thu thập dữ liệu ngay tại điểm đặt thiết bị là cần thiết. Hệ thống này có thể hỗ trợ giám sát điều kiện môi trường theo thời gian thực, phục vụ các ứng dụng như nhà kính, vườn thông minh, nông nghiệp chính xác hoặc các hệ thống giám sát môi trường quy mô nhỏ.

Bên cạnh yêu cầu thu thập dữ liệu, một hệ thống IoT hiện đại cần đảm bảo khả năng truyền dữ liệu ổn định, có cơ chế bảo vệ dữ liệu và có khả năng mở rộng lên nền tảng lưu trữ hoặc giám sát từ xa. Trong đề tài này, hệ thống được thiết kế theo kiến trúc gồm node cảm biến sử dụng vi điều khiển STM32 và gateway sử dụng ESP32. STM32 đảm nhiệm chức năng thu thập dữ liệu từ cảm biến, xử lý dữ liệu, đóng gói và truyền dữ liệu qua module LoRa E32. ESP32 đóng vai trò gateway, nhận dữ liệu từ LoRa, kiểm tra tính hợp lệ của frame, giải mã dữ liệu, thực hiện dự báo xu hướng nhiệt độ cục bộ và gửi dữ liệu lên nền tảng Cloud thông qua giao thức HTTPS.

Ngoài ra, đề tài còn hướng đến việc tích hợp cơ chế bảo mật dữ liệu bằng thuật toán ASCON-128a theo cơ chế AEAD, viết đầy đủ là Authenticated Encryption with Associated Data. Hệ thống sử dụng hai lớp bảo vệ dữ liệu: tuyến thiết bị đo ngoài trời đến bộ trung tâm Gateway sử dụng ASCON-128a với authentication tag 4 byte để tối ưu kích thước frame LoRa, trong khi tuyến bộ trung tâm Gateway đến Supabase Edge Function sử dụng khóa thứ hai với authentication tag 16 byte để tăng mức xác thực khi truyền dữ liệu lên

Cloud. Bên cạnh đó, dữ liệu thu thập được còn được khai thác bằng mô hình Multiple Linear Regression nhằm khảo sát khả năng dự báo xu hướng nhiệt độ cục bộ trong ngắn hạn.

Việc kết hợp giữa hệ thống nhúng, truyền thông không dây, bảo mật dữ liệu, nền tảng Cloud và học máy giúp đề tài có tính ứng dụng thực tế, đồng thời tạo nền tảng cho các hướng phát triển mở rộng trong tương lai.

2. Mục tiêu đề tài

Mục tiêu của đề tài là thiết kế và triển khai một hệ thống quan trắc thời tiết cục bộ sử dụng kiến trúc IoT gồm thiết bị đo ngoài trời và bộ trung tâm Gateway. Hệ thống có khả năng thu thập dữ liệu môi trường, truyền dữ liệu không dây qua LoRa, bảo vệ dữ liệu bằng cơ chế mã hóa xác thực, gửi dữ liệu lên Cloud và khai thác dữ liệu bằng mô hình học máy cơ bản.

Các mục tiêu cụ thể của đề tài bao gồm:

- Thiết kế STM32 Sensor Node để thu thập dữ liệu môi trường và giám sát trạng thái nguồn tại vị trí quan trắc.
- Thiết kế định dạng payload nhỏ gọn và truyền dữ liệu từ Sensor Node đến ESP32-S3 Gateway thông qua LoRa E32.
- Ứng dụng ASCON-128a để mã hóa và xác thực dữ liệu trên các tuyến truyền quan trọng của hệ thống.
- Thiết kế ESP32-S3 Gateway để tiếp nhận, kiểm tra, giải mã, xử lý dữ liệu và gửi dữ liệu lên Supabase Cloud.
- Xây dựng hệ thống lưu trữ, giám sát và hiển thị dữ liệu trên Supabase và Web Dashboard.
- Khảo sát khả năng dự báo xu hướng nhiệt độ cục bộ bằng mô hình Multiple Linear Regression.
- Tích hợp BLE provisioning và OTA firmware update nhằm hỗ trợ cấu hình và vận hành Gateway trong thực tế.

3. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu của đề tài là hệ thống quan trắc thời tiết cục bộ với các thành phần phần cứng và phần mềm cốt lõi bao gồm: vi điều khiển STM32F103C8T6, bộ trung tâm Gateway, hệ thống cảm biến môi trường SHT30, BMP388 và INA219, module truyền

thông vô tuyến LoRa E32, thuật toán mật mã hạng nhẹ ASCON-128a, nền tảng Supabase và mô hình học máy Multiple Linear Regression.

Về phạm vi thực hiện, đề tài tập trung giải quyết luồng dữ liệu từ thiết bị đầu cuối đến nền tảng Cloud. Cụ thể, đề tài tiến hành thiết kế kiến trúc tổng thể và phát triển firmware cho cả thiết bị đo ngoài trời lẫn bộ trung tâm Gateway. STM32 thực hiện chu trình thu thập dữ liệu vi khí hậu, kiểm tra lỗi cảm biến, chuyển đổi dữ liệu sang định dạng biểu diễn số nguyên theo hệ số tỉ lệ, mã hóa payload bằng ASCON-128a và truyền frame qua LoRa E32. bộ trung tâm Gateway nhận frame LoRa, kiểm tra authentication tag, giải mã dữ liệu, chuyển đổi dữ liệu về giá trị vật lý, thực hiện dự báo xu hướng nhiệt độ bằng mô hình Multiple Linear Regression và gửi dữ liệu lên Supabase thông qua HTTPS.

Ở tầng Cloud, hệ thống sử dụng Supabase Edge Function để tiếp nhận dữ liệu từ bộ trung tâm Gateway. Payload được mã hóa lại bằng khóa thứ hai trước khi gửi lên Cloud, sau đó Edge Function thực hiện kiểm tra, giải mã và ghi dữ liệu vào PostgreSQL Database. Dữ liệu lưu trữ có thể được sử dụng cho mục đích hiển thị, theo dõi lịch sử, phân tích xu hướng và đánh giá kết quả dự báo.

Trong phạm vi đề tài, hệ thống được triển khai với một node cảm biến STM32 và một gateway ESP32. Mô hình này có thể mở rộng trong tương lai để hỗ trợ nhiều node cảm biến cùng gửi dữ liệu về một gateway hoặc một nền tảng Cloud chung.

4. Phương pháp thực hiện

Đề tài được thực hiện theo phương pháp kết hợp giữa nghiên cứu lý thuyết, thiết kế hệ thống, triển khai thực nghiệm và kiểm thử đánh giá. Trước tiên, nhóm tiến hành khảo sát các tài liệu kỹ thuật liên quan đến vi điều khiển STM32, ESP32, cảm biến môi trường, module truyền thông LoRa, thuật toán ASCON-128a, giao thức HTTPS, nền tảng Cloud và mô hình hồi quy tuyến tính đa biến. Từ đó, kiến trúc tổng thể của hệ thống được xây dựng theo mô hình Sensor Node – Gateway – Cloud.

Sau khi xác định kiến trúc hệ thống, các module phần cứng và phần mềm được triển khai theo từng khối chức năng. STM32 được sử dụng để đọc dữ liệu cảm biến, xử lý lỗi, đóng gói payload, mã hóa dữ liệu và truyền frame qua LoRa. ESP32 được sử dụng để nhận dữ liệu từ LoRa, kiểm tra tính hợp lệ của frame, giải mã payload, chuyển đổi dữ liệu về giá trị vật lý, thực hiện mô hình dự báo và gửi dữ liệu lên Supabase Edge Function thông qua HTTPS.

Đối với phần bảo mật, dữ liệu cảm biến sau khi đóng gói được xử lý bằng thuật toán ASCON-128a theo cơ chế AEAD. Ở tuyến STM32 đến ESP32, hệ thống sử dụng authentication tag 4 byte nhằm giảm kích thước frame truyền qua LoRa. Ở tuyến ESP32 đến Cloud, payload được mã hóa lại bằng khóa thứ hai và sử dụng authentication tag 16 byte nhằm tăng mức xác thực khi dữ liệu được gửi lên Supabase Edge Function.

Đối với phần học máy, mô hình Multiple Linear Regression được sử dụng như một mô hình cơ bản để khảo sát khả năng dự báo xu hướng nhiệt độ cục bộ trong ngắn hạn. Mô hình sử dụng các thông số môi trường và thông tin thời gian làm đầu vào để tạo ra giá trị dự báo nhiệt độ trong khoảng thời gian tiếp theo.

Ngoài luồng dữ liệu chính, hệ thống còn triển khai các chức năng hỗ trợ vận hành như BLE provisioning và OTA firmware update. BLE provisioning giúp bộ trung tâm Gateway có thể được cấu hình WiFi trong quá trình triển khai thực tế. OTA cho phép cập nhật firmware từ xa thông qua thông tin ota_url, ota_seq và ota_sha256 được lưu trên Cloud.

Quá trình kiểm thử được định hướng theo mô hình chữ V, trong đó mỗi giai đoạn thiết kế có một giai đoạn kiểm thử tương ứng. Các kiểm thử bao gồm kiểm thử đơn vị, kiểm thử tích hợp, kiểm thử hệ thống, kiểm thử bảo mật, kiểm thử truyền thông, kiểm thử OTA và kiểm thử mô hình học máy.

5. Giới hạn nghiên cứu

Trong phạm vi hiện tại, đề tài tập trung vào việc hiện thực hóa luồng thu thập, truyền tải, bảo mật, lưu trữ và khai thác dữ liệu vi khí hậu từ thiết bị đo ngoài trời đến bộ trung tâm Gateway và Supabase Cloud. Hệ thống tận dụng nền tảng Backend-as-a-Service Supabase với PostgreSQL Database, Realtime và Edge Functions để giảm yêu cầu triển khai một máy chủ backend riêng, đồng thời vẫn đảm bảo khả năng lưu trữ và xử lý dữ liệu từ gateway.

Bên cạnh những kết quả đạt được, đề tài vẫn còn một số giới hạn nhất định. Hệ thống chưa đi sâu vào thiết kế cơ khí chống chịu thời tiết ngoài trời, chưa đánh giá độ bền trong môi trường khắc nghiệt và chưa tối ưu hóa hệ thống năng lượng mặt trời cho vận hành độc lập dài hạn. Mô hình dự báo được sử dụng trong đề tài là Multiple Linear Regression, có mục tiêu khảo sát xu hướng nhiệt độ cục bộ ngắn hạn chứ chưa thay thế các mô hình dự báo khí tượng chuyên nghiệp.

Về bảo mật, đề tài tập trung vào mã hóa và xác thực payload cảm biến bằng ASCON-128a. Các vấn đề nâng cao như quản lý khóa động, trao đổi khóa an toàn, chứng thực thiết bị ở

quy mô lớn hoặc cơ chế phân quyền phức tạp chưa phải trọng tâm chính của đề tài. Trong cơ chế AEAD, phần Associated Data hiện chưa được sử dụng và được để trống; các thông tin như device ID, loại gói tin hoặc phiên truyền có thể được bổ sung vào Associated Data trong các phiên bản phát triển tiếp theo.

Về khả năng mở rộng, phiên bản hiện tại triển khai theo mô hình một thiết bị đo ngoài trời và một bộ trung tâm Gateway. Việc mở rộng thành mạng nhiều node, quản lý nhiều thiết bị, đồng bộ khóa cho nhiều node và tối ưu giao diện dashboard hoàn chỉnh cho người dùng cuối được xem là những hướng phát triển tiếp theo.

6. Bố cục báo cáo

Báo cáo được trình bày theo các nội dung chính sau:

Chương 1: Tổng quan đề tài

Chương 2: Cơ sở lý thuyết

Chương 3: Phân tích yêu cầu và thiết kế hệ thống

Chương 4: Triển khai hệ thống

Chương 5: Thực nghiệm, kết quả và đánh giá

Chương 6: Kết luận và hướng phát triển

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

1.1. Tổng quan về quan trắc thời tiết cục bộ

Quan trắc thời tiết là quá trình thu thập, theo dõi và phân tích các thông số môi trường như nhiệt độ, độ ẩm, áp suất khí quyển, tốc độ gió, lượng mưa và cường độ ánh sáng. Các dữ liệu này có vai trò quan trọng trong nhiều lĩnh vực như nông nghiệp, giao thông, đô thị thông minh, nhà thông minh và nghiên cứu môi trường.

Các trạm khí tượng truyền thống thường được đặt tại những vị trí cố định và cung cấp dữ liệu cho một khu vực rộng. Tuy nhiên, điều kiện thời tiết tại từng vị trí nhỏ có thể thay đổi đáng kể so với dữ liệu trung bình của toàn khu vực. Ví dụ, nhiệt độ trong khu vực có nhiều bề mặt bê tông có thể cao hơn khu vực có nhiều cây xanh; độ ẩm trong nhà kính có thể khác với môi trường bên ngoài; hoặc điều kiện tại những khu vực có độ che chắn khác nhau có thể tạo ra sự sai lệch đáng kể về thông số môi trường.

Quan trắc thời tiết cục bộ tập trung vào việc đo đạc dữ liệu môi trường tại một vị trí cụ thể. Dữ liệu này phản ánh sát hơn điều kiện thực tế tại nơi lắp đặt thiết bị. Nhờ đó, hệ thống có thể hỗ trợ các ứng dụng yêu cầu dữ liệu theo vị trí, chẳng hạn như điều khiển tưới tiêu trong nông nghiệp, giám sát môi trường nhà kính, cảnh báo điều kiện bất lợi hoặc xây dựng bộ dữ liệu vi khí hậu phục vụ phân tích dài hạn.

Trong đề tài này, hệ thống quan trắc thời tiết cục bộ được xây dựng nhằm thu thập các thông số cơ bản gồm nhiệt độ, độ ẩm, áp suất khí quyển và trạng thái nguồn cấp. Các dữ liệu này được xử lý tại thiết bị đo ngoài trời, truyền về bộ trung tâm Gateway thông qua LoRa, sau đó được gửi lên Cloud để lưu trữ, phân tích và phục vụ khảo sát dự báo xu hướng nhiệt độ cục bộ.

1.2. Vai trò của IoT trong quan trắc môi trường

Internet of Things là mô hình kết nối các thiết bị vật lý với mạng Internet nhằm thu thập, trao đổi và xử lý dữ liệu. Trong lĩnh vực quan trắc môi trường, IoT cho phép các cảm biến được triển khai tại nhiều vị trí khác nhau, thu thập dữ liệu theo thời gian thực và gửi về hệ thống trung tâm để lưu trữ, giám sát hoặc phân tích.

So với phương pháp đo thủ công, hệ thống IoT có nhiều ưu điểm. Dữ liệu có thể được thu thập liên tục, giảm phụ thuộc vào con người và cho phép người dùng theo dõi từ xa. Ngoài ra, khi dữ liệu được lưu trữ trong thời gian dài, hệ thống có thể xây dựng các biểu đồ xu hướng, phát hiện bất thường hoặc làm đầu vào cho các mô hình học máy.

Trong hệ thống của đề tài, IoT được áp dụng theo mô hình gồm ba lớp: Sensor Node, Gateway và Cloud. Sensor Node sử dụng STM32 để thu thập dữ liệu từ các cảm biến môi trường. Dữ liệu sau đó được mã hóa, đóng gói và truyền qua LoRa E32 đến bộ trung tâm Gateway. bộ trung tâm Gateway nhận dữ liệu, giải mã, xử lý, thực hiện dự báo xu hướng nhiệt độ và gửi dữ liệu lên Supabase thông qua HTTPS. Tại tầng Cloud, Supabase Edge Function tiếp nhận dữ liệu, giải mã payload được bảo vệ bằng khóa thứ hai và ghi dữ liệu vào PostgreSQL Database.

Việc sử dụng IoT giúp hệ thống có khả năng mở rộng. Trong tương lai, nhiều node cảm biến có thể được triển khai tại nhiều vị trí khác nhau và gửi dữ liệu về cùng một gateway hoặc cùng một nền tảng Cloud. Đây là cơ sở để xây dựng mạng quan trắc vi khí hậu phân tán.

1.3. Bài toán thu thập dữ liệu vi khí hậu

Vi khí hậu là tập hợp các điều kiện khí hậu trong một phạm vi nhỏ, có thể khác biệt so với khu vực xung quanh. Các yếu tố tạo nên vi khí hậu bao gồm địa hình, vật liệu bề mặt, cây xanh, nguồn nhiệt, độ thông gió, độ cao và điều kiện che chắn. Vì vậy, dữ liệu khí tượng tổng quát từ các trạm lớn có thể chưa phản ánh đúng điều kiện thực tế tại từng điểm cụ thể.

Bài toán đặt ra là cần xây dựng một hệ thống có khả năng thu thập dữ liệu môi trường ngay tại vị trí cần giám sát. Hệ thống cần đo được các thông số cơ bản như nhiệt độ, độ ẩm và áp suất khí quyển. Ngoài ra, đối với hệ thống hoạt động độc lập hoặc triển khai ngoài trời, việc theo dõi trạng thái nguồn cũng cần thiết để đánh giá tình trạng hoạt động của thiết bị.

Dữ liệu vi khí hậu có giá trị không chỉ ở thời điểm đo hiện tại mà còn ở chuỗi dữ liệu theo thời gian. Khi được lưu trữ liên tục, dữ liệu có thể phản ánh xu hướng biến thiên của môi trường. Ví dụ, sự thay đổi nhiệt độ theo thời điểm trong ngày, độ ẩm môi trường và áp suất khí quyển có thể cung cấp thông tin để khảo sát xu hướng ngắn hạn. Dù hệ thống trong phạm vi đề tài chưa thay thế các mô hình dự báo khí tượng chuyên nghiệp, dữ liệu thu thập được vẫn có thể dùng để khảo sát xu hướng và xây dựng mô hình dự báo đơn giản.

Trong đề tài này, dữ liệu vi khí hậu được thu thập thông qua các cảm biến SHT30, BMP388 và INA219. Dữ liệu ban đầu được đọc dưới dạng số thực, sau đó được chuyển sang định dạng biểu diễn số nguyên theo hệ số tỉ lệ để giảm kích thước payload truyền. Cách tiếp cận này phù hợp với hệ thống nhúng có tài nguyên hạn chế và truyền thông không dây có băng thông thấp.

1.4. Vấn đề truyền dữ liệu không dây trong hệ thống IoT

Trong các hệ thống IoT quan trắc môi trường, dữ liệu thường cần được truyền từ node cảm biến đến gateway hoặc server trung tâm. Tùy vào phạm vi triển khai, môi trường lắp đặt và yêu cầu năng lượng, hệ thống có thể sử dụng nhiều công nghệ truyền thông khác nhau như WiFi, Bluetooth, Zigbee, LoRa hoặc mạng di động.

Đối với các node cảm biến đặt ngoài trời hoặc ở vị trí xa gateway, LoRa là một lựa chọn phù hợp do có khả năng truyền xa, tiêu thụ năng lượng thấp và phù hợp với các gói dữ liệu nhỏ. Trong đề tài này, module LoRa E32 được sử dụng để truyền dữ liệu giữa thiết bị đo ngoài trời và bộ trung tâm Gateway. Module này giao tiếp với vi điều khiển thông qua UART, giúp việc tích hợp vào hệ thống nhúng trở nên đơn giản hơn.

Tuy nhiên, truyền dữ liệu không dây cũng đặt ra nhiều vấn đề kỹ thuật. Dữ liệu có thể bị mất gói, nhiễu, sai lệch hoặc nhận không đúng kích thước frame. Do đó, dữ liệu truyền cần có cấu trúc rõ ràng để gateway có thể phân tích và xử lý. Ngoài ra, việc sử dụng frame counter giúp hệ thống theo dõi thứ tự gói tin, hỗ trợ phát hiện mất gói và đóng vai trò đầu vào cho quá trình sinh nonce trong cơ chế mã hóa.

Trong hệ thống của đề tài, payload cảm biến sau khi được đóng gói sẽ được mã hóa bằng ASCON-128a. Frame truyền qua LoRa gồm frame counter, ciphertext và authentication tag 4 byte. Thiết kế này giúp giảm kích thước gói tin, phù hợp với đặc điểm truyền thông LoRa, đồng thời vẫn hỗ trợ kiểm tra tính hợp lệ của dữ liệu khi bộ trung tâm Gateway nhận frame.

Trong quá trình thử nghiệm, frame counter được vận hành theo từng phiên hệ thống. Khi cần đưa hệ thống về trạng thái ban đầu, toàn bộ phiên thử nghiệm sẽ được reset đồng bộ, bao gồm thiết bị đo ngoài trời, bộ trung tâm Gateway và dữ liệu thử nghiệm trên Cloud. Cách làm này giúp đảm bảo frame counter bắt đầu lại từ giá trị ban đầu một cách nhất quán trong môi trường demo và kiểm thử.

1.5. Vấn đề bảo mật dữ liệu cảm biến

Dữ liệu trong hệ thống IoT thường được truyền qua môi trường không dây nên có nguy cơ bị nghe lén, sửa đổi hoặc giả mạo. Đối với các hệ thống quan trắc môi trường, nhiều người có thể cho rằng dữ liệu cảm biến không quá nhạy cảm. Tuy nhiên, khi hệ thống được mở rộng cho các ứng dụng nông nghiệp, cảnh báo môi trường hoặc điều khiển tự động, dữ liệu sai lệch có thể dẫn đến các quyết định điều khiển không chính xác.

Bảo mật dữ liệu cảm biến trong hệ thống IoT không chỉ bao gồm mã hóa nội dung dữ liệu mà còn cần đảm bảo tính toàn vẹn và xác thực. Nếu một frame bị thay đổi trong quá trình truyền, gateway cần phát hiện và loại bỏ frame đó thay vì sử dụng dữ liệu sai. Vì vậy, cơ chế mã hóa xác thực là phù hợp cho hệ thống truyền dữ liệu cảm biến qua môi trường không dây.

Trong đề tài này, thuật toán ASCON-128a được sử dụng theo cơ chế AEAD, viết đầy đủ là Authenticated Encryption with Associated Data. Cơ chế này cho phép hệ thống vừa mã hóa payload cảm biến, vừa tạo authentication tag để kiểm tra tính toàn vẹn và tính hợp lệ của dữ liệu. Khi bộ trung tâm Gateway nhận frame, hệ thống kiểm tra authentication tag trước khi chấp nhận và giải mã dữ liệu. Nếu tag không hợp lệ, frame sẽ bị loại bỏ.

Hệ thống sử dụng hai tuyến bảo vệ dữ liệu. Ở tuyến thiết bị đo ngoài trời đến bộ trung tâm Gateway, payload cảm biến được mã hóa bằng khóa thứ nhất và sử dụng authentication tag 4 byte nhằm tối ưu kích thước frame LoRa. Ở tuyến bộ trung tâm Gateway đến Supabase Edge Function, payload được mã hóa lại bằng khóa thứ hai và sử dụng authentication tag 16 byte nhằm tăng mức xác thực khi truyền dữ liệu lên Cloud.

Trong phạm vi đề tài, phần Associated Data của cơ chế AEAD chưa được sử dụng và được để trống. Điều này có nghĩa là hệ thống tập trung mã hóa và xác thực trực tiếp phần payload cảm biến. Các hướng mở rộng như bổ sung device ID, loại gói tin, phiên truyền hoặc thông tin định tuyến vào Associated Data có thể được nghiên cứu trong các phiên bản phát triển tiếp theo.

Các vấn đề nâng cao như quản lý khóa động, trao đổi khóa an toàn, chứng thực thiết bị ở quy mô lớn hoặc đánh giá an toàn ở mức sản phẩm thương mại chưa phải trọng tâm chính của đề tài.

1.6. Vai trò của học máy trong dự báo xu hướng nhiệt độ cục bộ

Học máy là một hướng tiếp cận cho phép hệ thống khai thác dữ liệu lịch sử để tìm ra mối quan hệ giữa các biến đầu vào và giá trị cần dự báo. Trong bài toán quan trắc thời tiết cục bộ, dữ liệu như nhiệt độ, độ ẩm và áp suất khí quyển có thể được lưu trữ theo thời gian để phục vụ phân tích xu hướng.

Các mô hình dự báo thời tiết chuyên nghiệp thường rất phức tạp và sử dụng nhiều nguồn dữ liệu khác nhau. Tuy nhiên, trong phạm vi một hệ thống IoT cục bộ, mục tiêu không phải là thay thế mô hình khí tượng lớn mà là khai thác dữ liệu đo tại điểm đặt để khảo sát xu

hướng ngắn hạn hoặc hỗ trợ cảnh báo đơn giản. Đây là hướng tiếp cận phù hợp với các ứng dụng như nhà kính, vườn thông minh hoặc hệ thống giám sát môi trường tại chỗ.

Trong đề tài này, mô hình Multiple Linear Regression được sử dụng như một mô hình học máy cơ bản nhằm khảo sát khả năng dự báo xu hướng nhiệt độ cục bộ trong ngắn hạn. Mô hình có ưu điểm là đơn giản, dễ triển khai, dễ giải thích và phù hợp để đánh giá bước đầu mối quan hệ giữa các biến môi trường.

Dữ liệu đầu vào của mô hình gồm nhiệt độ, độ ẩm, áp suất khí quyển và các đặc trưng thời gian như giờ trong ngày hoặc ngày trong năm. Giá trị đầu ra là nhiệt độ dự báo trong khoảng thời gian tiếp theo, cụ thể là dự báo nhiệt độ sau 2 giờ. Kết quả của mô hình Multiple Linear Regression giúp đánh giá tiềm năng khai thác bộ dữ liệu vi khí hậu thu thập được từ hệ thống IoT.

1.7. Giải pháp đề xuất của đề tài

Từ các vấn đề đã phân tích, đề tài đề xuất xây dựng một hệ thống quan trắc thời tiết cục bộ theo kiến trúc gồm ba lớp chính: lớp node cảm biến, lớp gateway và lớp xử lý/lưu trữ dữ liệu trên Cloud.

Ở lớp node cảm biến, STM32F103C8T6 được sử dụng để giao tiếp với các cảm biến môi trường. Cảm biến SHT30 đo nhiệt độ và độ ẩm, cảm biến BMP388 đo áp suất khí quyển, cảm biến INA219 theo dõi trạng thái nguồn. Dữ liệu sau khi đọc từ cảm biến được xử lý, kiểm tra lỗi và đóng gói thành payload nhỏ gọn theo định dạng biểu diễn số nguyên theo hệ số tỉ lệ.

Ở lớp truyền thông, module LoRa E32 được sử dụng để truyền dữ liệu không dây từ thiết bị đo ngoài trời đến bộ trung tâm Gateway. Frame truyền được thiết kế gồm frame counter, ciphertext và authentication tag 4 byte. Việc sử dụng frame counter giúp theo dõi thứ tự gói tin và hỗ trợ quá trình sinh nonce, trong khi ciphertext và tag giúp bảo vệ dữ liệu bằng ASCON-128a.

Ở lớp gateway, ESP32 nhận dữ liệu từ module LoRa E32, kiểm tra tính hợp lệ của frame, giải mã payload và chuyển đổi các giá trị f về giá trị vật lý ban đầu. Sau đó, ESP32 thực hiện mô hình Multiple Linear Regression để dự báo xu hướng nhiệt độ cục bộ trong ngắn hạn. Trước khi gửi dữ liệu lên Cloud, ESP32 mã hóa lại payload bằng khóa thứ hai và sử dụng authentication tag 16 byte. Dữ liệu sau đó được gửi đến Supabase Edge Function thông qua HTTPS.

Ở lớp Cloud, Supabase Edge Function tiếp nhận dữ liệu từ bộ trung tâm Gateway, kiểm tra token thiết bị, giải mã payload và ghi dữ liệu vào PostgreSQL Database. Supabase Realtime có thể được sử dụng để hỗ trợ cập nhật dữ liệu cho giao diện giám sát. Ngoài dữ liệu cảm biến, hệ thống còn lưu trạng thái gateway như thời điểm nhận gói LoRa gần nhất, phiên bản firmware, trạng thái kết nối và thông tin phục vụ OTA.

Ngoài luồng dữ liệu chính, bộ trung tâm Gateway còn được thiết kế thêm các cơ chế hỗ trợ vận hành. BLE provisioning được sử dụng để cấu hình WiFi theo hướng production: khi chưa có WiFi trong NVS hoặc credential cũ không còn hợp lệ, thiết bị sẽ mở BLE provisioning và chờ người dùng nhập WiFi mới; hệ thống không sử dụng fallback hardcoded WiFi trong vận hành chính thức. Cơ chế OTA firmware update cho phép cập nhật firmware từ xa thông qua thông tin ota_url, ota_seq và ota_sha256. Trong đó, ota_seq giúp kiểm tra thứ tự bản cập nhật, còn SHA-256 giúp tránh cập nhật lặp lại cùng một firmware.

Giải pháp đề xuất có ưu điểm là phân tách rõ ràng giữa nhiệm vụ thu thập dữ liệu và nhiệm vụ giao tiếp mạng. STM32 tập trung vào xử lý cảm biến, đóng gói và truyền dữ liệu nhúng, trong khi ESP32 đảm nhiệm vai trò gateway, bảo mật tuyến Cloud, dự báo xu hướng nhiệt độ, cấu hình WiFi, OTA và kết nối với hệ thống bên ngoài. Kiến trúc này giúp hệ thống dễ mở rộng, dễ kiểm thử và phù hợp với định hướng phát triển các hệ thống IoT quan trắc môi trường.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Tổng quan về kiến trúc IoT

Internet of Things, viết tắt là IoT, là mô hình trong đó các thiết bị vật lý được gắn cảm biến, bộ xử lý, phần mềm và khả năng kết nối mạng để có thể thu thập, trao đổi và xử lý dữ liệu. Trong các hệ thống IoT, thiết bị không chỉ hoạt động độc lập mà còn có khả năng kết nối với các thiết bị khác hoặc với hệ thống trung tâm thông qua mạng truyền thông. Nhờ đó, dữ liệu từ môi trường thực có thể được số hóa, truyền đi, lưu trữ và khai thác cho các mục đích giám sát, phân tích hoặc điều khiển.

Một hệ thống IoT thường được xây dựng theo kiến trúc nhiều tầng. Cách tổ chức theo tầng giúp chia nhỏ hệ thống thành các khối chức năng rõ ràng, giảm độ phức tạp và thuận tiện cho việc mở rộng. Mặc dù các tài liệu có thể mô tả kiến trúc IoT theo nhiều mô hình khác nhau, một cách phân chia phổ biến là gồm tầng thiết bị cảm nhận, tầng truyền thông hoặc gateway, và tầng ứng dụng hoặc cloud. Tầng thiết bị cảm nhận có nhiệm vụ thu thập dữ liệu từ môi trường thông qua cảm biến. Tầng truyền thông đảm nhiệm việc chuyển dữ liệu từ thiết bị lên hệ thống trung tâm. Tầng ứng dụng hoặc cloud đảm nhiệm lưu trữ, xử lý, hiển thị và khai thác dữ liệu [1].

Ở tầng thiết bị, hệ thống thường bao gồm các sensor node. Sensor node là thiết bị nhúng có nhiệm vụ đọc dữ liệu từ cảm biến, xử lý sơ bộ dữ liệu và truyền dữ liệu ra ngoài. Tùy theo yêu cầu ứng dụng, sensor node có thể sử dụng vi điều khiển công suất thấp, cảm biến số, cảm biến tương tự, module truyền thông và nguồn pin. Trong các hệ thống quan trắc môi trường, sensor node thường đo các đại lượng như nhiệt độ, độ ẩm, áp suất khí quyển, ánh sáng, nồng độ khí hoặc trạng thái nguồn cấp.

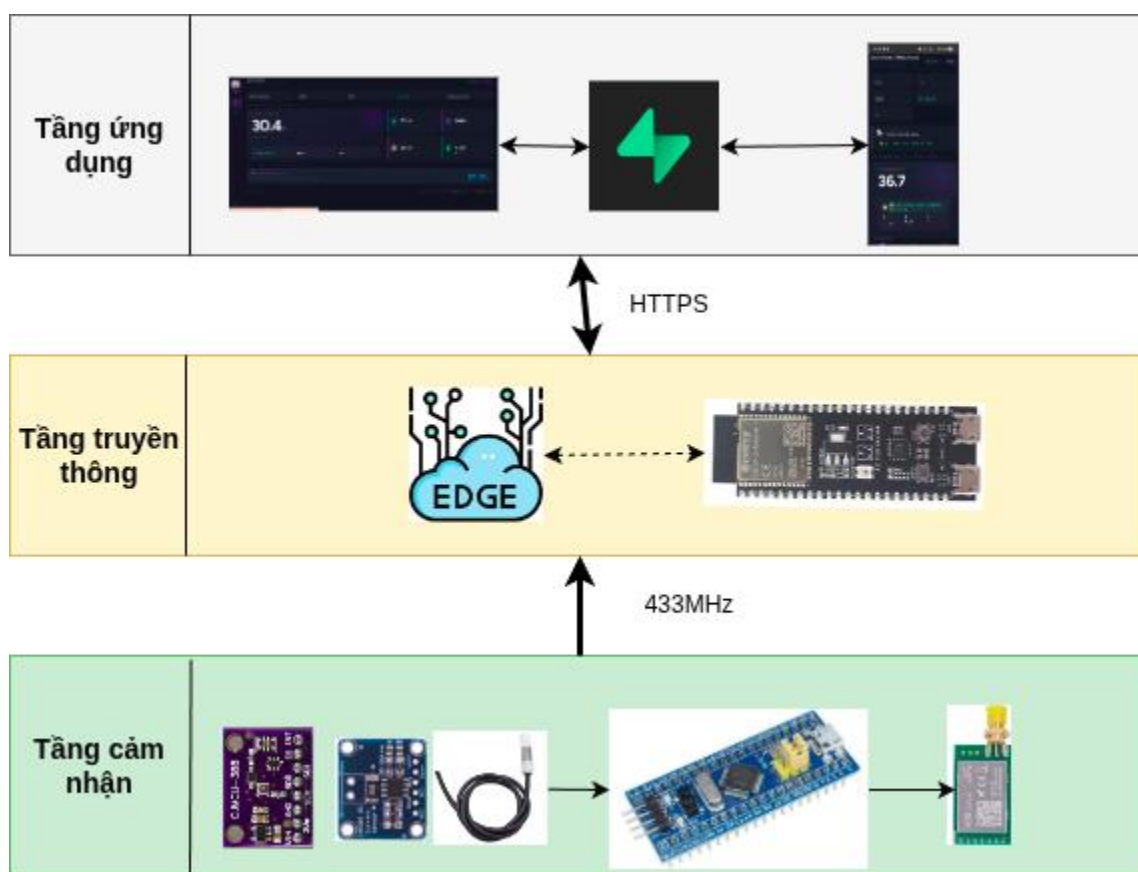
Tầng truyền thông đóng vai trò đưa dữ liệu từ sensor node đến nơi xử lý trung tâm. Tùy vào phạm vi truyền, băng thông, năng lượng và môi trường triển khai, tầng truyền thông có thể sử dụng nhiều công nghệ khác nhau như WiFi, Bluetooth Low Energy, Zigbee, LoRa, Ethernet hoặc mạng di động. Trong nhiều hệ thống IoT, gateway được dùng như một thiết bị trung gian giữa sensor node và Internet. Gateway có thể nhận dữ liệu từ các node thông qua giao tiếp cục bộ hoặc truyền thông vô tuyến tầm xa, sau đó chuyển tiếp dữ liệu lên server hoặc cloud bằng WiFi, Ethernet hoặc mạng di động.

Tầng cloud hoặc server chịu trách nhiệm tiếp nhận, lưu trữ và xử lý dữ liệu. Dữ liệu IoT thường có tính thời gian, vì mỗi bản ghi gắn với thời điểm đo hoặc thời điểm nhận. Do đó, tầng cloud cần hỗ trợ lưu trữ dữ liệu lịch sử, truy vấn theo thời gian, giám sát trạng thái thiết bị và cung cấp dữ liệu cho giao diện người dùng. Ngoài ra, tầng cloud còn có thể thực

hiện các chức năng nâng cao như phân tích xu hướng, phát hiện bất thường, cảnh báo hoặc cập nhật cấu hình thiết bị từ xa.

Ưu điểm của kiến trúc nhiều tầng là mỗi tầng có thể tập trung vào một nhóm nhiệm vụ riêng. Tầng cảm biến tập trung vào việc đo đạc và tiết kiệm năng lượng. Tầng gateway tập trung vào truyền thông, xử lý trung gian và chuyển đổi giao thức. Tầng cloud tập trung vào lưu trữ, phân tích và giao diện người dùng. Nhờ sự phân tách này, hệ thống có thể dễ dàng mở rộng từ một node lên nhiều node, thay đổi công nghệ truyền thông hoặc thay đổi nền tảng lưu trữ mà không nhất thiết phải thiết kế lại toàn bộ hệ thống.

Trong đề tài này, kiến trúc IoT nhiều tầng được dùng làm nền tảng để xây dựng hệ thống quan trắc thời tiết cục bộ. Phần phân tích chi tiết cách chia nhiệm vụ giữa các thành phần cụ thể sẽ được trình bày ở Chương 3.

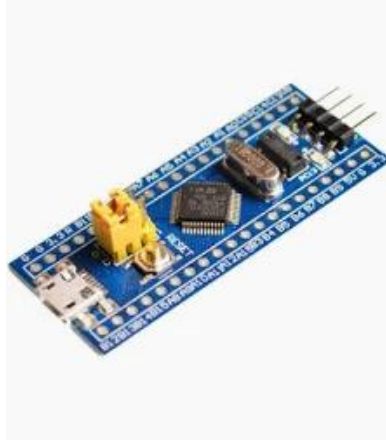


Hình 2.1. Kiến trúc tổng thể hệ thống quan trắc thời tiết cục bộ IoT

2.2. Vi điều khiển STM32F103C8T6

Vi điều khiển là một mạch tích hợp bao gồm lõi xử lý, bộ nhớ và các ngoại vi vào/ra trên cùng một chip. Khác với vi xử lý đa dụng cần nhiều thành phần bên ngoài để tạo thành một

hệ thống hoàn chỉnh, vi điều khiển được thiết kế để điều khiển trực tiếp các thiết bị ngoại vi trong hệ thống nhúng. Vì vậy, vi điều khiển thường được sử dụng trong các ứng dụng cần kích thước nhỏ, chi phí thấp, tiêu thụ điện năng thấp và khả năng phản hồi thời gian thực.



Hình 2.2. Vi điều khiển STM32F103C8T6

STM32F103C8T6 là vi điều khiển thuộc dòng STM32F1 của STMicroelectronics. Dòng STM32F103 thuộc nhóm mainstream performance line, sử dụng lõi Arm Cortex-M3 32 bit. Theo tài liệu của STMicroelectronics, dòng STM32F103 medium-density performance line sử dụng lõi Arm Cortex-M3 hoạt động ở tần số tối đa 72 MHz, tích hợp bộ nhớ Flash, SRAM và nhiều ngoại vi như ADC, timer, I2C, SPI, USART, USB và CAN [2]. Với mã STM32F103C8, chip thường được biết đến với 64 KB Flash và 20 KB SRAM, phù hợp cho nhiều ứng dụng điều khiển nhúng vừa và nhỏ.

Lõi Arm Cortex-M3 được thiết kế cho các hệ thống nhúng cần hiệu năng tốt nhưng vẫn giữ mức tiêu thụ năng lượng hợp lý. Đây là lõi xử lý 32 bit theo kiến trúc RISC, hỗ trợ tập lệnh Thumb-2, cơ chế ngắt lồng nhau NVIC và các tính năng phục vụ xử lý thời gian thực. So với các vi điều khiển 8 bit truyền thống, vi điều khiển 32 bit như STM32F103 cho phép xử lý dữ liệu nhanh hơn, quản lý ngoại vi phức tạp hơn và hỗ trợ tốt hơn cho các giao thức truyền thông hiện đại.

Một đặc điểm quan trọng của STM32F103C8T6 là hệ thống ngoại vi đa dạng. Các ngoại vi như I2C, SPI và USART cho phép vi điều khiển giao tiếp với nhiều cảm biến hoặc module truyền thông. ADC tích hợp cho phép đọc tín hiệu tương tự. Timer hỗ trợ tạo xung, đo thời gian, điều khiển PWM hoặc xử lý các tác vụ định kỳ. DMA giúp truyền dữ liệu giữa bộ nhớ và ngoại vi mà không cần CPU can thiệp liên tục, từ đó giảm tải cho lõi xử lý trong một số ứng dụng.

Trong hệ thống cảm biến, STM32F103C8T6 phù hợp với vai trò bộ điều khiển trung tâm của sensor node. Vi điều khiển có thể đọc dữ liệu từ cảm biến, kiểm tra lỗi truyền thông, thực hiện xử lý sơ bộ và gửi dữ liệu đến module truyền thông. Nhờ có nhiều ngoại vi giao tiếp, STM32 có thể kết nối đồng thời với các cảm biến sử dụng I2C và module truyền thông sử dụng UART. Đây là lý do STM32F103C8T6 thường được sử dụng trong các hệ thống đo lường, giám sát và điều khiển nhúng.

2.3. ESP32-S3 và FreeRTOS

ESP32-S3 là vi điều khiển tích hợp kết nối không dây của Espressif Systems, thuộc họ ESP32-S. Khác với các vi điều khiển chỉ tập trung vào đọc cảm biến hoặc điều khiển ngoại vi, ESP32-S3 được thiết kế cho các ứng dụng IoT cần kết nối mạng, xử lý dữ liệu và giao tiếp với các dịch vụ cloud. Theo Espressif, ESP32-S3 sử dụng vi xử lý dual-core Xtensa LX7, có thể hoạt động ở tần số 240 MHz, tích hợp 512 KB SRAM, WiFi 2.4 GHz chuẩn 802.11 b/g/n và Bluetooth 5 Low Energy [3].



Hình 2.3. Vi điều khiển ESP32-S3 N16R8

Việc tích hợp WiFi và Bluetooth Low Energy giúp ESP32-S3 phù hợp với vai trò thiết bị kết nối mạng hoặc gateway IoT. WiFi cho phép thiết bị truyền dữ liệu lên Internet thông qua router hoặc access point. Bluetooth Low Energy phù hợp cho các chức năng cấu hình cục bộ, trao đổi dữ liệu khoảng cách ngắn hoặc provisioning. Ngoài ra, ESP32-S3 còn có nhiều ngoại vi như SPI, I2C, UART, I2S, PWM, ADC, RMT và các GPIO lập trình được, giúp chip có thể giao tiếp với nhiều loại cảm biến và module ngoại vi [3].

ESP32-S3 được phát triển trong hệ sinh thái ESP-IDF. ESP-IDF là framework chính thức của Espressif dùng để phát triển firmware cho các chip ESP32. Framework này cung cấp driver ngoại vi, TCP/IP stack, WiFi stack, Bluetooth stack, HTTP client, TLS, OTA, NVS và nhiều thành phần phần mềm khác. Nhờ đó, lập trình viên có thể xây dựng ứng dụng IoT phức tạp mà không phải tự triển khai toàn bộ các tầng phần mềm thấp.

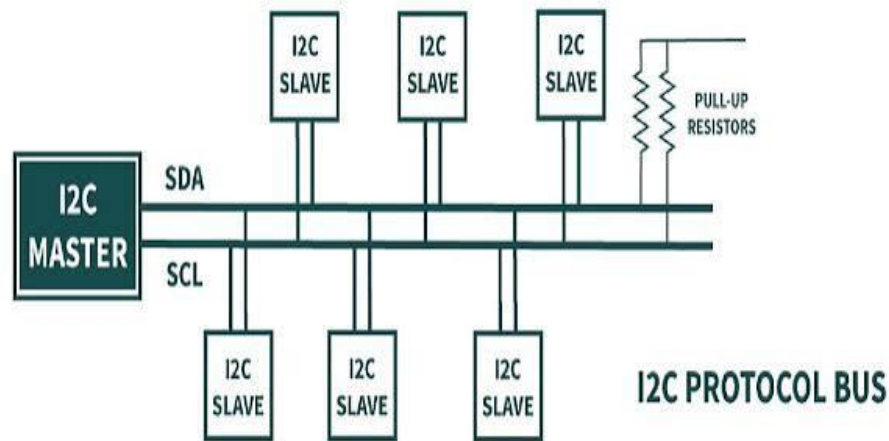
Một thành phần quan trọng trong ESP-IDF là FreeRTOS. FreeRTOS là hệ điều hành thời gian thực mã nguồn mở, được tích hợp vào ESP-IDF như một thành phần lõi. Theo tài liệu Espressif, các ứng dụng ESP-IDF và nhiều thành phần trong ESP-IDF được xây dựng dựa trên FreeRTOS; ESP-IDF cũng cung cấp các triển khai FreeRTOS để hỗ trợ SMP trên các chip ESP đa lõi [4]. FreeRTOS cho phép chương trình được chia thành nhiều task, mỗi task đảm nhiệm một chức năng riêng và được hệ điều hành lập lịch thực thi.

Trong một hệ thống gateway IoT, FreeRTOS giúp tổ chức phần mềm rõ ràng hơn. Các tác vụ như nhận dữ liệu, xử lý dữ liệu, gửi dữ liệu lên mạng, cập nhật trạng thái, kiểm tra cấu hình hoặc thực hiện OTA có thể được tách thành các task riêng. Ngoài task, FreeRTOS còn cung cấp queue, semaphore, mutex, timer và event group. Các cơ chế này giúp các task trao đổi dữ liệu, đồng bộ tài nguyên và tránh xung đột khi nhiều tác vụ cùng truy cập một thành phần phần cứng hoặc phần mềm.

So với mô hình lập trình vòng lặp đơn giản, mô hình dùng FreeRTOS phù hợp hơn với các gateway có nhiều chức năng chạy đồng thời. Tuy nhiên, việc sử dụng RTOS cũng yêu cầu thiết kế cẩn thận về bộ nhớ stack, độ ưu tiên task, hàng đợi dữ liệu và tài nguyên dùng chung. Nếu phân chia task không hợp lý, hệ thống có thể gặp lỗi tràn stack, nghẽn hàng đợi, deadlock hoặc mất dữ liệu khi một task xử lý quá lâu. Vì vậy, trong Chương 3 và Chương 4, việc tổ chức phần mềm trên ESP32-S3 cần được trình bày như một phần của thiết kế và triển khai hệ thống.

2.4. Chuẩn giao tiếp I2C

I2C, viết đầy đủ là Inter-Integrated Circuit, là chuẩn giao tiếp nối tiếp đồng bộ được phát triển ban đầu bởi Philips Semiconductors, nay thuộc NXP Semiconductors. I2C được thiết kế để các vi mạch trong cùng một hệ thống có thể trao đổi dữ liệu với nhau thông qua một bus đơn giản. Theo đặc tả UM10204 của NXP, I2C sử dụng hai đường tín hiệu chính là SDA và SCL. SDA là đường dữ liệu nối tiếp, còn SCL là đường xung clock dùng để đồng bộ quá trình truyền nhận dữ liệu [5].

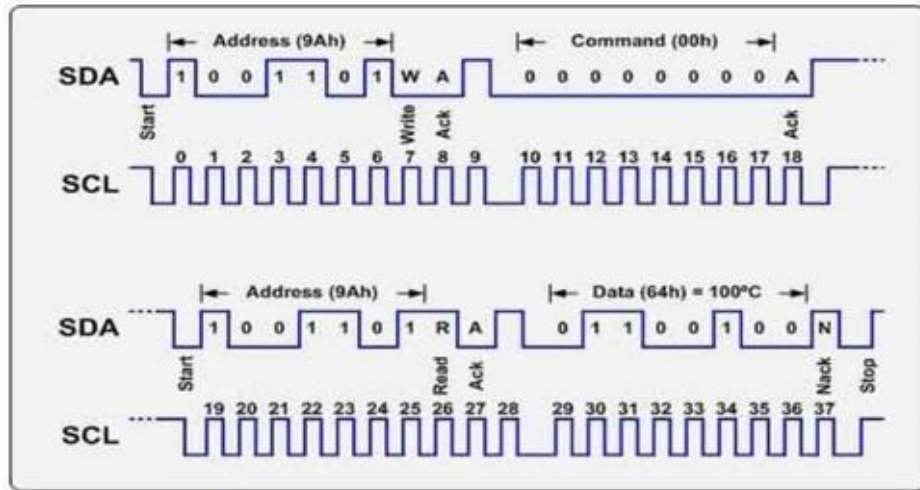


Hình 2.4. Cấu trúc bus I2C giữa Master và các Slave

Một đặc điểm quan trọng của I2C là nhiều thiết bị có thể cùng kết nối trên một bus. Thiết bị điều khiển quá trình truyền được gọi là controller, còn thiết bị được truy cập gọi là target. Trong các tài liệu cũ, hai vai trò này thường được gọi là master và slave. Controller tạo xung clock, phát điều kiện START, gửi địa chỉ target, xác định hướng đọc hoặc ghi, và phát điều kiện STOP để kết thúc giao dịch. Mỗi target trên bus có một địa chỉ riêng, thường là địa chỉ 7 bit, giúp controller chọn đúng thiết bị cần giao tiếp.

Về mặt điện, hai đường SDA và SCL của I2C thường sử dụng cấu trúc open-drain hoặc open-collector. Điều này có nghĩa là thiết bị chỉ chủ động kéo đường tín hiệu xuống mức thấp, còn mức cao được tạo bởi điện trở kéo lên. Theo tài liệu NXP, cả SDA và SCL là các đường hai chiều, được nối với nguồn dương thông qua điện trở kéo lên [5]. Nhờ đặc tính này, nhiều thiết bị có thể dùng chung bus mà không gây xung đột mức cao trực tiếp, đồng thời hỗ trợ các cơ chế như arbitration và clock stretching.

Một giao dịch I2C bắt đầu bằng điều kiện START. Điều kiện START được định nghĩa là sự chuyển mức từ cao xuống thấp trên SDA trong khi SCL đang ở mức cao. Sau START, controller gửi byte địa chỉ gồm 7 bit địa chỉ và 1 bit R/W để xác định thao tác đọc hoặc ghi. Nếu target nhận đúng địa chỉ, nó phản hồi bằng ACK. Sau đó, dữ liệu được truyền theo từng byte 8 bit. Theo đặc tả I2C, mỗi byte truyền trên SDA có độ dài 8 bit và phải được theo sau bởi một bit ACK hoặc NACK. Giao dịch kết thúc bằng điều kiện STOP, là sự chuyển mức từ thấp lên cao trên SDA trong khi SCL đang ở mức cao [5].



Hình 2.5. Trình tự truyền nhận dữ liệu trên bus I2C

I2C hỗ trợ nhiều tốc độ truyền khác nhau như Standard-mode, Fast-mode, Fast-mode Plus và High-speed mode. Tốc độ thực tế phụ thuộc vào thiết bị, điện dung bus, giá trị điện trở kéo lên, chiều dài dây và môi trường nhiễu. Vì I2C là bus dùng đường tín hiệu kéo lên thụ động, cạnh lên của tín hiệu phụ thuộc nhiều vào hằng số thời gian RC của bus. Nếu dây quá dài, điện dung lớn hoặc điện trở kéo lên không phù hợp, tín hiệu có thể bị méo, làm giảm độ tin cậy truyền thông.

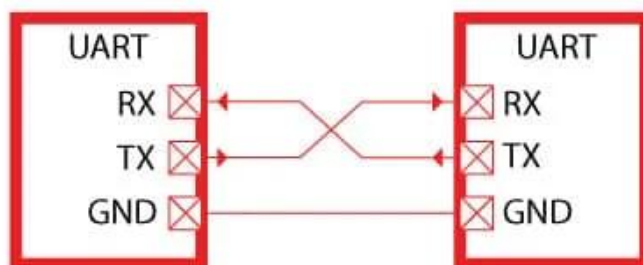
Trong các hệ thống cảm biến, I2C được sử dụng phổ biến vì chỉ cần hai dây tín hiệu để kết nối nhiều cảm biến số. Các cảm biến như cảm biến nhiệt độ, độ ẩm, áp suất, ánh sáng hoặc giám sát nguồn thường hỗ trợ I2C. Tuy nhiên, I2C chủ yếu phù hợp cho kết nối trong phạm vi ngắn trên cùng bo mạch hoặc giữa các module gần nhau. Khi triển khai ngoài thực tế, cần chú ý địa chỉ thiết bị, điện áp logic, điện trở kéo lên, tốc độ bus và khả năng chống nhiễu của đường truyền.

2.5. Chuẩn giao tiếp UART

UART, viết đầy đủ là Universal Asynchronous Receiver-Transmitter, là khối giao tiếp nối tiếp bất đồng bộ được sử dụng rộng rãi trong máy tính, hệ thống nhúng và thiết bị ngoại vi. Khác với I2C hoặc SPI, UART không sử dụng đường clock riêng để đồng bộ giữa bên truyền và bên nhận. Thay vào đó, hai thiết bị phải được cấu hình cùng các tham số truyền thông như baud rate, số bit dữ liệu, parity và số bit stop. Việc đồng bộ từng ký tự được thực hiện thông qua start bit và stop bit.

Một kết nối UART cơ bản thường gồm hai đường tín hiệu là TX và RX. Chân TX của thiết bị này được nối với chân RX của thiết bị kia, và ngược lại. Khi dùng hai đường riêng biệt

cho truyền và nhận, UART có thể hỗ trợ truyền song công, tức là hai thiết bị có thể truyền và nhận đồng thời. Trong hệ thống nhúng, UART thường sử dụng mức logic TTL hoặc CMOS, chẳng hạn 3,3 V hoặc 5 V, tùy theo vi điều khiển và module ngoại vi. Vì vậy, khi kết nối UART giữa hai thiết bị, cần đảm bảo mức điện áp logic tương thích.



Hình 2.6. Sơ đồ kết nối giao tiếp UART

Dữ liệu UART được truyền theo từng frame. Khi đường truyền ở trạng thái rỗi, tín hiệu thường ở mức logic cao. Mỗi frame bắt đầu bằng start bit ở mức logic thấp để báo cho bộ nhận biết rằng một ký tự mới sắp được truyền. Sau start bit là các bit dữ liệu, thường là 8 bit. Các bit dữ liệu thường được truyền theo thứ tự từ bit ít quan trọng nhất đến bit quan trọng nhất. Sau phần dữ liệu có thể có parity bit để kiểm tra lỗi đơn giản. Cuối frame là một hoặc nhiều stop bit ở mức logic cao để báo kết thúc ký tự truyền [6].

Cấu hình UART phổ biến trong hệ thống nhúng là 8N1, nghĩa là 8 bit dữ liệu, không parity và 1 stop bit. Với cấu hình này, để truyền một byte dữ liệu 8 bit, đường truyền thực tế cần 10 bit gồm 1 start bit, 8 data bit và 1 stop bit. Do đó, tốc độ dữ liệu hữu ích luôn nhỏ hơn baud rate danh nghĩa. Ví dụ, ở baud rate 9600 bps với cấu hình 8N1, tốc độ truyền dữ liệu hữu ích xấp xỉ 960 byte mỗi giây trong điều kiện lý tưởng.

Do UART là giao tiếp bất đồng bộ, sai lệch baud rate giữa hai thiết bị có thể gây lỗi nhận dữ liệu. Nếu bộ nhận lấy mẫu sai thời điểm, dữ liệu có thể bị sai bit hoặc xuất hiện lỗi framing. Vì vậy, hai thiết bị cần được cấu hình thống nhất về baud rate, số bit dữ liệu, parity và stop bit. Ngoài ra, khi dữ liệu truyền liên tục, firmware cần xử lý bộ đệm nhận kịp thời để tránh tràn buffer.

UART thường được dùng để kết nối vi điều khiển với module truyền thông, module GPS, module Bluetooth, module LoRa, mạch chuyển USB-UART hoặc thiết bị debug. Ưu điểm

của UART là đơn giản, dễ triển khai và có sẵn trên hầu hết vi điều khiển. Nhược điểm là mỗi kết nối UART cơ bản chỉ phục vụ giao tiếp điểm-điểm giữa hai thiết bị và không có cơ chế định địa chỉ nhiều thiết bị trên cùng bus như I2C.

2.6. Cảm biến nhiệt độ và độ ẩm SHT30

SHT30 là cảm biến nhiệt độ và độ ẩm số thuộc họ SHT3x-DIS của Sensirion. Đây là dòng cảm biến tích hợp phần tử đo độ ẩm, cảm biến nhiệt độ, mạch xử lý tín hiệu và giao tiếp số trong cùng một chip. Theo datasheet của Sensirion, SHT3x-DIS cung cấp đầu ra số đã được hiệu chuẩn, tuyến tính hóa và bù nhiệt độ; cảm biến hỗ trợ giao tiếp I2C với tốc độ lên đến 1 MHz và có hai địa chỉ I2C có thể lựa chọn [7].



Hình 2.7. Cảm biến nhiệt độ - độ ẩm SHT30

Cảm biến độ ẩm trong SHT30 hoạt động dựa trên sự thay đổi đặc tính điện của phần tử cảm biến khi độ ẩm môi trường thay đổi. Cảm biến nhiệt độ được tích hợp để đo nhiệt độ môi trường, đồng thời hỗ trợ quá trình bù nhiệt độ cho phép đo độ ẩm. Dữ liệu sau khi được cảm biến xử lý nội bộ sẽ được trả về dưới dạng số, giúp vi điều khiển không cần tự thiết kế mạch đo tương tự phức tạp.

SHT30 giao tiếp với vi điều khiển thông qua I2C. Trong quá trình đo, vi điều khiển gửi lệnh đo đến cảm biến. Sau khi nhận lệnh, cảm biến thực hiện quá trình lấy mẫu và xử lý, sau đó trả về dữ liệu nhiệt độ và độ ẩm. Datasheet cho biết ở chế độ single shot, mỗi lệnh đo tạo ra một cặp dữ liệu gồm một giá trị nhiệt độ 16 bit và một giá trị độ ẩm 16 bit [7]. Mỗi nhóm dữ liệu đo được đi kèm CRC 8 bit, giúp tăng độ tin cậy trong truyền thông I2C [7].

CRC là điểm quan trọng của SHT30 trong các hệ thống cần độ tin cậy dữ liệu. Khi vi điều khiển đọc dữ liệu từ cảm biến, firmware có thể tính lại CRC và so sánh với CRC do cảm biến gửi kèm. Nếu CRC không khớp, dữ liệu có thể bị lỗi trong quá trình truyền hoặc đọc.

và cần được loại bỏ. Điều này hữu ích trong hệ thống quan trắc môi trường, vì dữ liệu sai có thể làm ảnh hưởng đến quá trình hiển thị, lưu trữ hoặc dự báo.

SHT30 hỗ trợ nhiều chế độ đo với các mức repeatability khác nhau. Repeatability cao thường cho độ nhiễu thấp hơn nhưng thời gian đo và năng lượng tiêu thụ có thể tăng. Repeatability thấp có thể phù hợp với các ứng dụng cần tiết kiệm năng lượng hoặc tốc độ đo cao hơn. Việc lựa chọn chế độ đo phụ thuộc vào yêu cầu của hệ thống, chẳng hạn độ chính xác mong muốn, chu kỳ lấy mẫu và khả năng cấp nguồn.

Trong hệ thống quan trắc thời tiết cục bộ, cảm biến nhiệt độ và độ ẩm là thành phần quan trọng vì hai đại lượng này phản ánh trực tiếp trạng thái môi trường. Dữ liệu nhiệt độ và độ ẩm cũng có thể được sử dụng làm đầu vào cho các mô hình phân tích hoặc dự báo xu hướng. Phản ứng dụng cụ thể của SHT30 trong thiết kế Sensor Node sẽ được trình bày ở Chương 3.

2.7. Cảm biến áp suất khí quyển BMP388

BMP388 là cảm biến áp suất khí quyển số của Bosch Sensortec. Cảm biến này có khả năng đo áp suất và nhiệt độ, hỗ trợ giao tiếp I2C hoặc SPI. Theo datasheet của Bosch Sensortec, BMP388 là cảm biến áp suất số dựa trên nguyên lý cảm biến đã được kiểm chứng, có kích thước rất nhỏ, tiêu thụ dòng thấp và phù hợp với các thiết bị dùng pin. Datasheet cũng nêu các ứng dụng điển hình của BMP388 gồm IoT, định vị trong nhà, thiết bị đeo, trạm thời tiết gia đình và dự báo thời tiết [8].



Hình 2.8. Cảm biến áp suất - nhiệt độ BMP388

Cảm biến áp suất khí quyển dùng để đo áp suất của không khí tại vị trí đặt thiết bị. Áp suất khí quyển thay đổi theo độ cao, điều kiện thời tiết và các biến động khí quyển. Vì vậy, dữ liệu áp suất thường được dùng trong các ứng dụng như đo độ cao, hỗ trợ định vị, theo dõi

thời tiết và quan trắc môi trường. Trong một hệ thống thời tiết cục bộ, áp suất là một biến bổ sung quan trọng bên cạnh nhiệt độ và độ ẩm.

BMP388 có dải đo áp suất từ 300 hPa đến 1250 hPa và dải nhiệt độ hoạt động từ -40 °C đến +85 °C theo tài liệu của Bosch Sensortec [8]. Cảm biến hỗ trợ các chế độ nguồn như sleep mode, forced mode và normal mode. Ở forced mode, vi điều khiển có thể kích hoạt từng lần đo khi cần, phù hợp với hệ thống cần kiểm soát chu kỳ lấy mẫu và tiết kiệm năng lượng. Ở normal mode, cảm biến có thể đo liên tục theo cấu hình oversampling và output data rate.

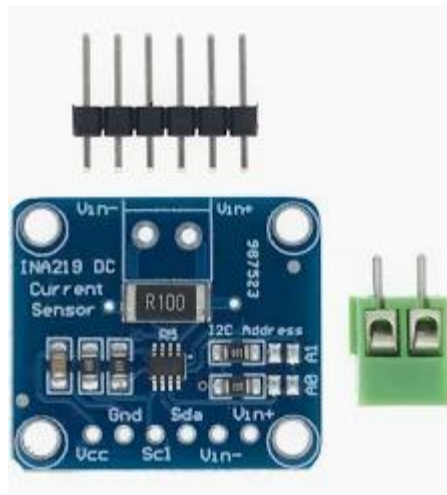
Một điểm quan trọng của BMP388 là quá trình bù dữ liệu đo. Cảm biến áp suất thường không chỉ trả về giá trị áp suất đã sẵn sàng dùng ngay ở dạng vật lý, mà cần sử dụng dữ liệu thô kết hợp với các hệ số hiệu chỉnh nội bộ được lưu trong cảm biến. Các hệ số này được nhà sản xuất xác định trong quá trình hiệu chuẩn. Firmware hoặc thư viện điều khiển cần đọc các hệ số này và áp dụng công thức bù để tính ra áp suất và nhiệt độ có ý nghĩa vật lý.

BMP388 cũng hỗ trợ các chức năng giúp cải thiện chất lượng đo như oversampling, bộ lọc IIR và FIFO. Oversampling giúp giảm nhiễu bằng cách lấy nhiều mẫu và xử lý trung bình. Bộ lọc IIR giúp làm mượt dữ liệu, đặc biệt hữu ích khi áp suất thay đổi chậm nhưng dữ liệu đo có nhiễu. FIFO cho phép lưu tạm dữ liệu trong cảm biến, giảm nguy cơ mất mẫu khi vi điều khiển không đọc kịp trong một số tình huống.

Trong hệ thống quan trắc thời tiết cục bộ, BMP388 phù hợp để bổ sung thông tin áp suất khí quyển. Dữ liệu áp suất không chỉ phục vụ hiển thị mà còn có thể dùng làm biến đầu vào cho mô hình phân tích xu hướng. Phần sử dụng BMP388 cụ thể trong Sensor Node sẽ được trình bày trong chương thiết kế hệ thống.

2.8. Cảm biến giám sát nguồn INA219

INA219 là cảm biến giám sát dòng điện, điện áp và công suất của Texas Instruments. Theo datasheet, INA219 là current shunt and power monitor có giao tiếp I2C hoặc SMBus, có khả năng đo điện áp rơi trên điện trở shunt và điện áp bus [9]. Cảm biến này thường được sử dụng trong các hệ thống cần theo dõi dòng tiêu thụ, điện áp nguồn hoặc công suất của tải.



Hình 2.9. Cảm biến đo dòng - áp INA219

Nguyên lý cơ bản của INA219 dựa trên phép đo điện áp rơi trên điện trở shunt. Khi dòng điện đi qua điện trở shunt, trên điện trở xuất hiện một điện áp nhỏ tỉ lệ với dòng điện theo định luật Ohm. INA219 đo điện áp vi sai giữa hai chân IN+ và IN- để xác định shunt voltage. Đồng thời, cảm biến cũng đo bus voltage, tức là điện áp nguồn hoặc điện áp cấp cho tải so với mass. Từ shunt voltage, bus voltage và giá trị điện trở shunt, hệ thống có thể tính toán dòng điện và công suất tiêu thụ.

INA219 tích hợp ADC để chuyển đổi các tín hiệu đo thành giá trị số. Datasheet cho biết INA219 có độ phân giải ADC 12 bit, shunt voltage LSB 10 μ V và bus voltage LSB 4 mV [9]. Thiết bị hỗ trợ các thanh ghi cấu hình, thanh ghi shunt voltage, bus voltage, current và power. Để đọc được trực tiếp dòng điện và công suất, hệ thống cần cấu hình calibration register. Nếu chưa cấu hình calibration register, người dùng vẫn có thể đọc trực tiếp shunt voltage và bus voltage để tự xử lý bên ngoài.

Giao tiếp I2C giúp INA219 dễ tích hợp vào các hệ thống nhúng. Cảm biến có nhiều địa chỉ lập trình được, cho phép đặt nhiều INA219 trên cùng một bus trong các ứng dụng cần giám sát nhiều nhánh nguồn. Thiết bị cũng hỗ trợ các chế độ hoạt động khác nhau như đo liên tục hoặc đo theo yêu cầu. Việc lựa chọn chế độ phụ thuộc vào yêu cầu cập nhật dữ liệu và mức tiêu thụ năng lượng của hệ thống.

Trong hệ thống IoT dùng pin hoặc nguồn cấp ngoài, giám sát nguồn là chức năng quan trọng. Dữ liệu điện áp và dòng điện giúp đánh giá trạng thái pin, phát hiện sụt áp, kiểm tra tải tiêu thụ và hỗ trợ bảo trì thiết bị. Đối với sensor node đặt xa hoặc hoạt động độc lập, việc biết trạng thái nguồn giúp hệ thống đánh giá độ ổn định vận hành và phát hiện sớm các trường hợp nguồn yếu hoặc lỗi cấp nguồn.

2.9. Module truyền thông LoRa E32

LoRa là công nghệ truyền thông vô tuyến tầm xa, công suất thấp, phù hợp với các hệ thống IoT cần truyền lượng dữ liệu nhỏ qua khoảng cách xa. LoRa sử dụng kỹ thuật trải phổ dạng chirp, còn gọi là Chirp Spread Spectrum. Trong kỹ thuật này, tín hiệu được biểu diễn bằng các chirp có tần số thay đổi theo thời gian. Cách điều chế này giúp tín hiệu có khả năng chống nhiễu tốt, hỗ trợ truyền xa và phù hợp với các ứng dụng tốc độ dữ liệu thấp.

Cần phân biệt giữa LoRa và LoRaWAN. LoRa là công nghệ ở tầng vật lý, liên quan đến kỹ thuật điều chế tín hiệu vô tuyến. LoRaWAN là giao thức mạng lớp trên, định nghĩa cách các thiết bị đầu cuối, gateway, network server và application server tương tác với nhau. Một hệ thống có thể sử dụng chip hoặc module LoRa để truyền điểm-điểm mà không dùng LoRaWAN. Cách này thường phù hợp với các hệ thống đơn giản, số lượng node ít hoặc phạm vi nghiên cứu hẹp.

E32-433T30D là module truyền thông không dây của EBYTE. Theo user manual của EBYTE, E32-433T30D là module wireless serial port dựa trên chip RF SX1278 của Semtech, hoạt động trong dải 410 MHz đến 441 MHz, mặc định 433 MHz, sử dụng công nghệ LoRa spread spectrum [10]. Tài liệu cũng nêu module hỗ trợ công suất phát tối đa khoảng 1 W, tốc độ truyền không khí từ 0,3 kbps đến 19,2 kbps, giao tiếp TTL UART 3,3 V và có thể đạt khoảng cách tham khảo 8 km trong điều kiện thử nghiệm thông thoáng



Hình 2.10. Module Lora E32-433Mhz 1W

Về phần cứng, module E32 có các chân chính gồm VCC, GND, RXD, TXD, M0, M1 và AUX. Chân RXD và TXD dùng để giao tiếp UART với vi điều khiển. Chân M0 và M1 dùng để chọn chế độ hoạt động. Chân AUX dùng để báo trạng thái làm việc của module,

ví dụ module đang bận truyền, còn dữ liệu trong buffer hoặc đang tự kiểm tra sau khi khởi động. Do đó, khi thiết kế firmware, vi điều khiển thường cần kiểm tra AUX trước khi gửi dữ liệu để tránh gửi khi module chưa sẵn sàng.

Module E32 hỗ trợ nhiều chế độ hoạt động. Ở chế độ Normal, UART và kênh vô tuyến đều mở, module truyền nhận dữ liệu theo cơ chế transparent transmission. Ở chế độ Wake-up, module thêm mã đánh thức trước dữ liệu để hỗ trợ đánh thức thiết bị đầu xa. Ở chế độ Power-saving, module tiết kiệm năng lượng và lắng nghe theo chu kỳ. Ở chế độ Sleep, module chủ yếu dùng để cấu hình tham số qua UART. Các chế độ này được chọn thông qua tổ hợp chân M0 và M1 [11].

Trong chế độ transparent transmission, vi điều khiển gửi byte dữ liệu đến module E32 qua UART. Module E32 sau đó xử lý việc truyền dữ liệu qua sóng LoRa. Ở phía nhận, module E32 nhận dữ liệu vô tuyến và xuất lại dữ liệu qua UART cho vi điều khiển. Cách tổ chức này giúp việc tích hợp đơn giản hơn so với việc điều khiển trực tiếp chip RF qua SPI, vì firmware chỉ cần xử lý truyền nhận UART ở tầng vi điều khiển.

Trong hệ thống quan trắc, LoRa E32 phù hợp với đường truyền giữa sensor node và gateway khi lượng dữ liệu nhỏ và không yêu cầu băng thông cao. Tuy nhiên, khi dùng LoRa cần chú ý đến tốc độ truyền không khí, công suất phát, antenna, môi trường truyền, khoảng cách, nhiễu và quy định tần số tại khu vực sử dụng. Phần lựa chọn cấu hình truyền thông cụ thể sẽ được trình bày trong chương thiết kế và triển khai.

2.10. Công nghệ quản lý nguồn và lưu trữ năng lượng

Trong hệ thống quan trắc thời tiết cục bộ, khối nguồn có vai trò quan trọng vì thiết bị đo ngoài trời được định hướng hoạt động độc lập tại vị trí đo. Thiết bị không chỉ cần nguồn cấp ổn định cho vi điều khiển, cảm biến và module truyền thông LoRa, mà còn cần khả năng duy trì hoạt động trong thời gian dài khi đặt ngoài môi trường. Vì vậy, hệ thống sử dụng giải pháp kết hợp giữa năng lượng mặt trời, pin lưu trữ Lithium, mạch sạc, mạch bảo vệ pin và bộ chuyển đổi nguồn.

Khối quản lý nguồn trong hệ thống gồm tấm pin năng lượng mặt trời, module sạc CN3791, pin Lithium EVE 1S, mạch bảo vệ pin 1S và bộ chuyển đổi nguồn TPS63020. Trong đó, CN3791 đảm nhiệm vai trò sạc pin từ nguồn năng lượng mặt trời, pin Lithium EVE đóng vai trò lưu trữ năng lượng, mạch bảo vệ 1S giúp hạn chế các trạng thái bất lợi trong quá trình sạc và xả, còn TPS63020 tạo ra đường nguồn ổn định cấp cho các khối điện tử của node.

2.10.1. Module sạc pin năng lượng mặt trời CN3791

CN3791 là module sạc pin Lithium thường được sử dụng trong các hệ thống dùng nguồn đầu vào từ tấm pin năng lượng mặt trời. Trong hệ thống này, CN3791 có nhiệm vụ nhận năng lượng từ tấm pin mặt trời và điều khiển quá trình sạc cho pin Lithium EVE 1S. Việc sử dụng module sạc riêng giúp quá trình nạp pin được kiểm soát tốt hơn so với việc cấp trực tiếp điện áp từ tấm pin vào pin lưu trữ.



Hình 2.11. Module CN3791 6V

Về nguyên lý, pin Lithium cần được sạc theo quy trình phù hợp để hạn chế tình trạng sạc quá mức, suy giảm tuổi thọ hoặc mất an toàn trong quá trình sử dụng. Module CN3791 hỗ trợ điều tiết dòng sạc và điện áp sạc, nhờ đó pin có thể được nạp từ nguồn năng lượng mặt trời trong điều kiện điện áp đầu vào thay đổi theo cường độ ánh sáng. Đây là đặc điểm phù hợp với thiết bị quan trắc ngoài môi trường, vì nguồn năng lượng mặt trời không ổn định tuyệt đối trong suốt thời gian vận hành [11]

Trong hệ thống quan trắc thời tiết cục bộ, CN3791 đóng vai trò trung gian giữa tấm pin năng lượng mặt trời và pin lưu trữ. Khi có ánh sáng, tấm pin tạo ra điện năng và đưa đến module sạc. Module sạc sau đó điều khiển quá trình nạp cho pin Lithium. Khi nguồn năng lượng mặt trời không đủ hoặc không có, hệ thống sử dụng năng lượng đã được lưu trữ trong pin để tiếp tục duy trì hoạt động của thiết bị đo ngoài trời.

Việc sử dụng CN3791 trong hệ thống giúp node có khả năng hoạt động độc lập hơn so với thiết kế chỉ dùng nguồn cấp ngoài. Tuy nhiên, hiệu quả của khối sạc phụ thuộc vào công suất tấm pin, điều kiện chiếu sáng, dung lượng pin, dòng tiêu thụ của thiết bị và chu kỳ hoạt động của node. Do đó, trong quá trình thiết kế hệ thống, khối nguồn cần được xem xét cùng với chiến lược tiết kiệm năng lượng của firmware.

2.10.2. Pin Lithium EVE và mạch bảo vệ 1S

Pin Lithium EVE được sử dụng làm nguồn lưu trữ năng lượng cho thiết bị đo ngoài trời. Pin có nhiệm vụ cung cấp điện cho hệ thống trong các khoảng thời gian tẩm pin năng lượng mặt trời không tạo đủ năng lượng, chẳng hạn khi trời tối, ánh sáng yếu hoặc thiết bị đặt ở vị trí bị che khuất. Nhờ có pin lưu trữ, node có thể duy trì hoạt động theo chu kỳ mà không phụ thuộc hoàn toàn vào nguồn sáng tức thời.



Hình 2.12. Pin EVE 18650 25P 2500mAh 3.7V 12C Lithium-ion

Trong hệ thống này, pin được sử dụng theo cấu hình một cell Lithium, thường gọi là 1S. Cấu hình 1S có điện áp thay đổi theo trạng thái sạc của pin, vì vậy không thể cấp trực tiếp cho toàn bộ hệ thống nếu các khối điện tử yêu cầu điện áp ổn định. Đối với nhiều cell Lithium-ion thông dụng, điện áp danh định thường nằm quanh mức 3,6 V đến 3,7 V và điện áp sạc đầy thường quanh mức 4,2 V, tùy theo hóa học và datasheet cụ thể của cell. Do đó, điện áp từ pin được đưa qua bộ chuyển đổi nguồn để tạo đường cấp phù hợp cho vi điều khiển, cảm biến và module LoRa.



Hình 2.13. Mạch Sạc Và Bảo Vệ Pin 1S 3.7V 4A

Bên cạnh pin lưu trữ, mạch bảo vệ 1S được sử dụng để tăng độ an toàn trong quá trình vận hành. Mạch bảo vệ có nhiệm vụ hạn chế các trạng thái bất lợi như sạc quá mức, xả quá sâu hoặc dòng bất thường. Các trạng thái này nếu xảy ra trong thời gian dài có thể ảnh hưởng đến tuổi thọ pin, độ ổn định của hệ thống và độ an toàn của thiết bị. Đối với pin Lithium-ion, các điều kiện điện áp vượt ngoài vùng làm việc an toàn hoặc trạng thái quá nhiệt có

thể gây suy giảm tuổi thọ và rủi ro an toàn, vì vậy việc sử dụng mạch bảo vệ là cần thiết trong các hệ thống dùng pin độc lập.

Pin Lithium và mạch bảo vệ 1S tạo thành khối lưu trữ năng lượng chính của node. Khối này kết hợp với module sạc CN3791 và bộ chuyển đổi nguồn TPS63020 để hình thành chuỗi cấp nguồn hoàn chỉnh: tấm pin năng lượng mặt trời tạo năng lượng, CN3791 sạc pin, pin lưu trữ năng lượng, mạch bảo vệ giám sát trạng thái pin và TPS63020 ổn định điện áp đầu ra cho hệ thống. Cách tổ chức này phù hợp với thiết bị cảm biến cần hoạt động theo chu kỳ và ưu tiên tiết kiệm năng lượng.

2.11. Kết cấu cơ khí và bảo vệ thiết bị ngoài trời

Đối với hệ thống quan trắc thời tiết cục bộ, phần cơ khí bảo vệ có ảnh hưởng trực tiếp đến khả năng vận hành và độ tin cậy của thiết bị. Thiết bị đo ngoài trời được đặt tại vị trí quan trắc nên phải chịu tác động của môi trường như ánh nắng, nước mưa, bụi, hơi ẩm và thay đổi nhiệt độ. Nếu chỉ đặt bo mạch và cảm biến trực tiếp ngoài môi trường, thiết bị có thể bị sai lệch dữ liệu đo hoặc hư hỏng phần cứng.

Vì vậy, hệ thống sử dụng kết cấu bảo vệ gồm vỏ hộp chống nước, van thở cân bằng áp suất và lồng Stevenson cho khu vực cảm biến. Vỏ hộp có nhiệm vụ bảo vệ bo mạch điện tử và các khối nguồn. Van thở hỗ trợ giảm hiện tượng tích tụ hơi ẩm và cân bằng áp suất giữa bên trong và bên ngoài hộp. Lồng Stevenson hỗ trợ tạo điều kiện đo phù hợp hơn cho cảm biến nhiệt độ và độ ẩm bằng cách hạn chế tác động trực tiếp của bức xạ mặt trời và nước mưa, đồng thời vẫn cho phép không khí lưu thông quanh cảm biến.

2.11.1. Vỏ hộp chống nước IP67 và van thở cân bằng áp suất

Vỏ hộp chống nước IP67 được sử dụng để bảo vệ bo mạch và các linh kiện điện tử của thiết bị đo ngoài trời khi thiết bị đặt trong điều kiện môi trường. Theo hệ thống phân cấp bảo vệ IP trong tiêu chuẩn IEC 60529, chữ số đầu tiên thể hiện mức bảo vệ chống bụi và vật rắn, còn chữ số thứ hai thể hiện mức bảo vệ chống nước. Với IP67, thiết bị đạt mức bảo vệ chống bụi ở cấp 6 và bảo vệ chống ngâm nước tạm thời ở cấp 7 trong điều kiện thử nghiệm tiêu chuẩn (International Electrotechnical Commission, 2013).

Trong hệ thống này, vỏ hộp chứa các thành phần như bo mạch xử lý, khối nguồn, pin lưu trữ, mạch sạc, mạch bảo vệ và các đầu nối cần thiết. Việc sử dụng vỏ hộp chống nước giúp tăng khả năng bảo vệ phần cứng, tuy nhiên hộp kín cũng có thể tạo ra một số vấn đề trong quá trình vận hành. Khi nhiệt độ môi trường thay đổi, không khí bên trong hộp có thể giãn

nở hoặc co lại, tạo chênh lệch áp suất giữa bên trong và bên ngoài. Ngoài ra, hơi ẩm có thể tích tụ bên trong hộp nếu quá trình thoát ẩm không được hỗ trợ.

Để hạn chế vấn đề trên, van thở cân bằng áp suất được bố trí trên vỏ bảo vệ. Theo thông số của nhà sản xuất, van thở dạng nylon sử dụng màng ePTFE cho phép không khí đi qua nhưng hạn chế hơi ẩm và bụi xâm nhập, giúp cân bằng áp suất trong vỏ thiết bị và giảm hiện tượng ngưng tụ. Một số thông số kỹ thuật của loại van này gồm cấp bảo vệ IP67–IP68 khi sử dụng với vòng đệm O-ring, dải nhiệt độ làm việc khoảng -40°C đến 100°C và lưu lượng khí điển hình được công bố theo điều kiện chênh áp nhất định [12].



Hình 2.14. Van thở Nylon M12x1.5, IP68

Trong hệ thống quan trắc thời tiết cục bộ, vỏ hộp chống nước và van thở không trực tiếp tham gia vào quá trình đo dữ liệu, nhưng có vai trò quan trọng đối với độ ổn định của phần cứng. Các thành phần này giúp bảo vệ bo mạch và khối nguồn, đồng thời hỗ trợ thiết bị hoạt động bền hơn trong điều kiện thực nghiệm ngoài môi trường.

2.11.2. Lồng Stevenson trong quan trắc khí tượng

Lồng Stevenson là kết cấu được sử dụng trong quan trắc khí tượng nhằm tạo môi trường đo phù hợp hơn cho các cảm biến nhiệt độ và độ ẩm. Mục đích chính của lồng Stevenson là hạn chế ảnh hưởng trực tiếp của bức xạ mặt trời và nước mưa lên cảm biến, đồng thời vẫn cho phép không khí lưu thông qua khu vực đo. Nhờ đó, cảm biến có thể phản ánh trạng thái không khí xung quanh tốt hơn so với trường hợp đặt trực tiếp dưới ánh nắng hoặc trong hộp kín.

Trong hệ thống này, lồng Stevenson được sử dụng cho khu vực cảm biến của thiết bị đo ngoài trời. Cảm biến SHT30 đo nhiệt độ và độ ẩm tương đối, nên vị trí lắp đặt có ảnh hưởng lớn đến kết quả đo. Nếu cảm biến bị chiếu nắng trực tiếp, nhiệt độ đo được có thể cao hơn

hiệt độ không khí thực tế. Nếu cảm biến bị đặt trong không gian quá kín, độ ẩm và nhiệt độ bên trong có thể thay đổi chậm hơn so với môi trường bên ngoài. Vì vậy, lồng Stevenson được dùng để vừa che chắn cảm biến, vừa duy trì khả năng trao đổi không khí.

Lồng Stevenson thường có cấu trúc gồm nhiều lớp che hoặc khe thông gió, giúp giảm tác động trực tiếp của nắng và mưa nhưng không ngăn hoàn toàn sự lưu thông không khí. Trong quan trắc khí tượng, cảm biến nhiệt độ không khí thường cần được đặt trong môi trường thông gió và được che chắn khỏi bức xạ trực tiếp để giảm sai lệch đo [13]. Đối với hệ thống quan trắc cục bộ, việc sử dụng lồng Stevenson giúp tăng tính hợp lý của dữ liệu đo trong điều kiện ngoài trời.

Tuy nhiên, kết quả đo vẫn có thể chịu ảnh hưởng bởi vị trí lắp đặt, độ cao đặt cảm biến, hướng nắng, bề mặt xung quanh và mức độ thông gió thực tế. Vì vậy, lồng Stevenson không làm thay đổi nguyên lý đo của cảm biến, mà đóng vai trò cải thiện điều kiện lắp đặt để dữ liệu đo ít bị ảnh hưởng bởi các yếu tố môi trường trực tiếp.

Trong phạm vi đề tài, lồng Stevenson được xem là một thành phần hỗ trợ quan trọng cho khối cảm biến môi trường. Khi kết hợp với vỏ hộp chống nước và van thở, lồng Stevenson góp phần hoàn thiện kết cấu bảo vệ ngoài trời cho thiết bị đo ngoài trời, giúp thiết bị phù hợp hơn với mục tiêu quan trắc thời tiết cục bộ.

2.12. Thuật toán ASCON-128a AEAD

Trong các hệ thống IoT, dữ liệu thường được truyền qua các môi trường không dây hoặc mạng Internet. Các môi trường truyền này có thể bị nghe lén, sửa đổi hoặc giả mạo dữ liệu. Vì vậy, bảo mật dữ liệu không chỉ là việc che giấu nội dung bản tin mà còn phải đảm bảo rằng bản tin không bị thay đổi và thật sự được tạo bởi bên có khóa hợp lệ. Đây là lý do các cơ chế mã hóa xác thực ngày càng được dùng trong hệ thống nhúng và IoT.

ASCON là họ thuật toán mật mã hạng nhẹ được thiết kế cho các thiết bị giới hạn tài nguyên. Theo NIST, chuẩn Ascon-based Lightweight Cryptography được xây dựng cho các thiết bị constrained devices và bao gồm các chức năng mã hóa xác thực, hàm băm và hàm mở rộng đầu ra [14]. Ascon ban đầu được đề xuất trong các cuộc thi và chương trình chuẩn hóa mật mã hạng nhẹ, sau đó được NIST lựa chọn làm nền tảng cho chuẩn mật mã hạng nhẹ. Mục tiêu của Ascon là cung cấp mức an toàn phù hợp trong khi vẫn triển khai hiệu quả trên phần cứng và phần mềm nhúng.

ASCON-128a thuộc nhóm thuật toán AEAD. AEAD là viết tắt của Authenticated Encryption with Associated Data, nghĩa là mã hóa xác thực có dữ liệu liên kết. Cơ chế

AEAD kết hợp hai mục tiêu: bảo mật nội dung bản tin và xác thực tính toàn vẹn của bản tin. Kết quả của quá trình mã hóa không chỉ có ciphertext mà còn có authentication tag. Nếu dữ liệu bị sửa đổi, tag sẽ không hợp lệ và quá trình giải mã bị từ chối.

Một phép mã hóa AEAD thường có bốn đầu vào: khóa bí mật, nonce, plaintext và associated data. Khóa bí mật được chia sẻ giữa bên gửi và bên nhận. Nonce là giá trị dùng một lần cho mỗi phép mã hóa với cùng một khóa. Plaintext là dữ liệu cần được bảo vệ. Associated data là dữ liệu không bị mã hóa nhưng vẫn được đưa vào quá trình xác thực, chẳng hạn header, định danh thiết bị hoặc loại gói tin. Kết quả đầu ra gồm ciphertext và authentication tag.

Ascon sử dụng trạng thái nội bộ 320 bit, được chia thành 5 word 64 bit. Theo tài liệu đặc tả Ascon, tất cả thành viên trong họ Ascon dùng cùng một phép hoán vị nhẹ; mỗi vòng biến đổi gồm ba bước chính: cộng hằng số vòng, lớp thay thế phi tuyến dùng S-box 5 bit và lớp khuếch tán tuyến tính sử dụng các phép xoay bit và XOR [14]. Cấu trúc này giúp Ascon đạt được sự khuếch tán dữ liệu trong trạng thái nội bộ, đồng thời giữ chi phí triển khai tương đối thấp.

Quá trình mã hóa xác thực của Ascon có thể mô tả theo bốn pha. Pha thứ nhất là initialization, trong đó trạng thái được khởi tạo từ khóa và nonce. Pha thứ hai là associated data processing, trong đó associated data được đưa vào trạng thái nếu có sử dụng. Pha thứ ba là plaintext processing, trong đó plaintext được hấp thụ vào trạng thái và ciphertext được sinh ra. Pha cuối cùng là finalization, trong đó khóa được đưa vào trạng thái một lần nữa và authentication tag được sinh ra [14].

Trong quá trình giải mã, bên nhận sử dụng cùng khóa, nonce và associated data để xử lý ciphertext. Nếu tag tính lại khớp với tag nhận được, bản tin được xem là hợp lệ và plaintext được sử dụng. Nếu tag sai, bản tin phải bị loại bỏ. Đây là điểm quan trọng trong truyền thông cảm biến, vì hệ thống có thể phát hiện frame sai hoặc bị sửa đổi trước khi dùng dữ liệu cho lưu trữ, hiển thị hoặc điều khiển.

Trong phạm vi cơ sở lý thuyết, ASCON-128a được xem như một thuật toán phù hợp cho IoT nhờ định hướng lightweight cryptography. Phần thiết kế cụ thể như chọn khóa, cách tạo nonce, kích thước tag và cách đưa ASCON vào frame dữ liệu sẽ được trình bày ở Chương 3.

2.13. Giao thức HTTPS trong truyền dữ liệu IoT

HTTP là giao thức tầng ứng dụng được dùng phổ biến để truyền dữ liệu giữa client và server. Tuy nhiên, HTTP thông thường truyền dữ liệu ở dạng không được bảo vệ, nên không phù hợp khi dữ liệu đi qua Internet và cần bảo mật. HTTPS giải quyết vấn đề này bằng cách đặt HTTP bên trong kênh TLS. Nói cách khác, HTTPS là HTTP chạy trên TLS, trong đó TLS cung cấp mã hóa, xác thực và kiểm tra toàn vẹn dữ liệu.

TLS, viết đầy đủ là Transport Layer Security, là giao thức bảo mật dùng để bảo vệ truyền thông client/server. RFC 8446 mô tả TLS 1.3 là giao thức được thiết kế để ngăn nghe lén, sửa đổi và giả mạo thông điệp trong quá trình truyền thông [15]. Khi thiết lập kết nối HTTPS, client và server thực hiện TLS handshake. Trong quá trình này, hai bên thống nhất phiên bản giao thức, bộ thuật toán mật mã, trao đổi khóa và xác thực server thông qua chứng chỉ số. Sau khi handshake hoàn tất, dữ liệu HTTP được truyền bên trong kênh đã mã hóa.

TLS cung cấp ba nhóm chức năng chính. Thứ nhất là tính bí mật, vì dữ liệu được mã hóa trong quá trình truyền. Thứ hai là tính toàn vẹn, vì dữ liệu bị sửa đổi trên đường truyền có thể bị phát hiện. Thứ ba là xác thực server, vì client có thể kiểm tra chứng chỉ số để xác định đang kết nối đến đúng server. Trong một số hệ thống, TLS cũng có thể hỗ trợ xác thực client, nhưng trong nhiều ứng dụng IoT, xác thực thiết bị thường được thực hiện thêm ở tầng ứng dụng bằng token, khóa API hoặc cơ chế chữ ký.

Đối với thiết bị IoT, HTTPS có ưu điểm là tương thích với hầu hết nền tảng cloud, REST API và serverless function. Thiết bị có thể gửi dữ liệu bằng phương thức POST, nhận câu lệnh bằng GET hoặc tương tác với API theo các chuẩn web phổ biến. Tuy nhiên, HTTPS cũng yêu cầu tài nguyên phần cứng và phần mềm lớn hơn so với các giao thức nhẹ hơn. Thiết bị cần đủ RAM cho TLS handshake, chứng chỉ, bộ đệm truyền nhận và các phép toán mật mã.

Trong kiến trúc IoT, HTTPS thường phù hợp hơn với gateway hoặc thiết bị có kết nối WiFi/Ethernet, trong khi các sensor node nhỏ có thể dùng giao thức cục bộ nhẹ hơn để gửi dữ liệu về gateway. Gateway sau đó chuyển dữ liệu lên cloud qua HTTPS. Cách tổ chức này giúp sensor node giảm tải xử lý mạng, trong khi gateway đảm nhiệm phần truyền thông Internet và bảo mật kênh truyền.

Trong đề tài, HTTPS được dùng làm nền tảng truyền dữ liệu từ gateway lên cloud. Phần cấu trúc request, dữ liệu gửi kèm, token xác thực và cách xử lý phía cloud sẽ được trình bày trong chương thiết kế hệ thống.

2.14. Supabase trong lưu trữ và xử lý dữ liệu IoT

Sau khi dữ liệu IoT được gửi lên Internet, hệ thống cần một tầng backend để tiếp nhận, xử lý, lưu trữ và cung cấp dữ liệu cho giao diện người dùng. Nếu tự xây dựng backend hoàn toàn, nhóm phát triển phải quản lý server, cơ sở dữ liệu, API, xác thực, phân quyền, lưu file và triển khai hạ tầng. Với các hệ thống thử nghiệm, đồ án hoặc prototype, mô hình Backend-as-a-Service giúp giảm đáng kể khối lượng triển khai backend.

Supabase là một nền tảng Backend-as-a-Service mã nguồn mở, cung cấp nhiều thành phần backend như PostgreSQL Database, Edge Functions, Realtime, Authentication và Storage. Theo tài liệu Supabase, mỗi project Supabase có một cơ sở dữ liệu Postgres đầy đủ; database là nền tảng cho Auth, Storage, Realtime và Edge Functions [16]. Điều này giúp Supabase phù hợp với các ứng dụng cần lưu dữ liệu có cấu trúc và phát triển nhanh tầng backend.

PostgreSQL là hệ quản trị cơ sở dữ liệu quan hệ mạnh, hỗ trợ SQL, bảng, ràng buộc, chỉ mục, view, function và nhiều kiểu dữ liệu mở rộng. Trong hệ thống IoT, dữ liệu cảm biến thường là dữ liệu chuỗi thời gian, mỗi bản ghi gắn với thời điểm đo và định danh thiết bị. Lưu dữ liệu trong PostgreSQL giúp hệ thống dễ truy vấn lịch sử, lọc theo thời gian, tính thống kê, tạo dashboard và xuất dữ liệu phục vụ phân tích.

Supabase Edge Functions là các hàm TypeScript phía server, chạy tại edge và có thể được dùng làm HTTP endpoint [16]. Edge Function phù hợp cho các tác vụ như kiểm tra request, xác thực token, xử lý dữ liệu đầu vào, gọi API ngoài hoặc ghi dữ liệu vào database. Trong hệ thống IoT, Edge Function có thể đóng vai trò lớp trung gian giữa thiết bị và database, giúp tránh cho thiết bị ghi trực tiếp vào cơ sở dữ liệu mà không qua kiểm tra.

Supabase Realtime cung cấp khả năng nhận cập nhật khi dữ liệu trong database thay đổi. Chức năng này hữu ích cho dashboard giám sát, vì giao diện người dùng có thể nhận dữ liệu mới mà không cần liên tục truy vấn lại toàn bộ bảng. Với hệ thống quan trắc môi trường, realtime giúp hiển thị dữ liệu gần thời gian thực và cập nhật trạng thái thiết bị khi có bản ghi mới.

Supabase Storage là thành phần dùng để lưu trữ file. Trong các ứng dụng web, Storage thường dùng cho hình ảnh, tài liệu hoặc file nhị phân. Trong các hệ thống IoT, Storage có thể được dùng để lưu firmware, file cấu hình hoặc tài nguyên phục vụ cập nhật thiết bị. Khi kết hợp với database và Edge Function, Storage có thể trở thành một phần của cơ chế quản lý firmware từ xa.

Một khía cạnh quan trọng của Supabase là phân quyền dữ liệu. Row Level Security cho phép kiểm soát quyền truy cập ở mức từng dòng trong bảng. Trong hệ thống IoT, không phải client nào cũng nên có quyền ghi trực tiếp vào bảng dữ liệu cảm biến. Việc kết hợp RLS, token, Edge Function và policy giúp hệ thống kiểm soát tốt hơn dữ liệu đầu vào, quyền đọc của dashboard và quyền ghi của thiết bị.

2.15. BLE provisioning và OTA firmware update

Trong các thiết bị IoT dùng WiFi, việc cấu hình SSID và mật khẩu là bước quan trọng trước khi thiết bị có thể kết nối Internet. Nếu thông tin WiFi được nạp cứng trong firmware, thiết bị sẽ khó triển khai trong môi trường thực tế vì mỗi địa điểm có thể dùng mạng khác nhau. Khi đổi router hoặc đổi mật khẩu, người dùng phải nạp lại firmware. Vì vậy, nhiều thiết bị IoT sử dụng cơ chế provisioning để cấu hình WiFi sau khi thiết bị đã được sản xuất hoặc lắp đặt.

WiFi provisioning là quá trình đưa thông tin mạng WiFi vào thiết bị. ESP-IDF hỗ trợ WiFi provisioning thông qua nhiều phương thức, trong đó có Bluetooth Low Energy. Theo tài liệu Espressif, WiFi provisioning cho phép thiết bị nhận WiFi credentials thông qua SoftAP hoặc BLE, sau đó lưu lại để sử dụng cho các lần khởi động sau [17]. BLE provisioning phù hợp với thiết bị không có màn hình hoặc bàn phím, vì người dùng có thể cấu hình thông qua điện thoại.

Bluetooth Low Energy là công nghệ truyền thông không dây tầm ngắn, tiêu thụ năng lượng thấp. Trong provisioning, thiết bị ESP32 có thể phát dịch vụ BLE. Ứng dụng trên điện thoại kết nối đến thiết bị, gửi SSID và mật khẩu WiFi qua kênh provisioning. Sau khi nhận credential, thiết bị thử kết nối đến mạng WiFi. Nếu kết nối thành công, thông tin này được lưu vào vùng nhớ không mất dữ liệu, thường là NVS.

Trong các thiết bị IoT, ngoài chương trình firmware chính, thiết bị còn cần lưu trữ một số thông tin cấu hình nhỏ để phục vụ quá trình vận hành. Các thông tin này thường không nên mất đi khi thiết bị reset hoặc mất nguồn. Trên nền tảng ESP32-S3, một cơ chế thường được sử dụng cho mục đích này là NVS.

NVS, viết tắt của Non-Volatile Storage, là cơ chế lưu trữ dữ liệu không mất khi mất nguồn. NVS thường được dùng để lưu các cặp khóa – giá trị có kích thước nhỏ, chẳng hạn như tên mạng WiFi, mật khẩu WiFi, trạng thái cấu hình thiết bị, phiên bản firmware, trạng thái OTA đã áp dụng hoặc một số tham số vận hành. Khi thiết bị khởi động lại, firmware có thể đọc dữ liệu từ NVS để quyết định bước xử lý tiếp theo. Ví dụ, nếu trong NVS đã có WiFi credential hợp lệ, thiết bị có thể tự kết nối WiFi mà không cần người dùng cấu hình lại.

Ngược lại, nếu chưa có thông tin WiFi hoặc thông tin cũ không còn hợp lệ, thiết bị có thể chuyển sang chế độ cấu hình lại mạng.

BLE provisioning là cơ chế cấu hình WiFi cho thiết bị thông qua Bluetooth Low Energy. Thay vì nạp cứng SSID và mật khẩu WiFi vào firmware, thiết bị có thể mở một dịch vụ provisioning để người dùng nhập thông tin mạng trong quá trình triển khai thực tế. Cách làm này phù hợp với thiết bị IoT vì môi trường lắp đặt có thể thay đổi, tên mạng WiFi hoặc mật khẩu WiFi có thể khác nhau giữa các vị trí. Khi quá trình provisioning thành công, thông tin WiFi được lưu vào NVS để thiết bị sử dụng lại ở các lần khởi động sau.

Nhờ có BLE provisioning, thiết bị gateway có thể được triển khai linh hoạt hơn. Người dùng không cần nạp lại firmware chỉ để thay đổi WiFi. Thiết bị có thể tự kiểm tra trạng thái cấu hình khi khởi động: nếu đã có thông tin WiFi trong NVS thì tiến hành kết nối; nếu chưa có hoặc kết nối thất bại nhiều lần thì mở lại provisioning để người dùng nhập thông tin mới. Cơ chế này giúp hệ thống phù hợp hơn với điều kiện vận hành thực tế so với cách cấu hình WiFi cố định trong mã nguồn.

Bên cạnh cấu hình WiFi, thiết bị IoT cũng cần có khả năng cập nhật firmware sau khi đã triển khai. OTA firmware update, viết tắt của Over-The-Air firmware update, là cơ chế cho phép thiết bị tải và cài đặt firmware mới thông qua mạng mà không cần kết nối cáp nạp trực tiếp. Đây là chức năng quan trọng đối với các thiết bị IoT vì sau khi lắp đặt, thiết bị có thể cần sửa lỗi, bổ sung chức năng, thay đổi cấu hình bảo mật hoặc cải thiện độ ổn định của hệ thống.

Trên ESP32-S3, cơ chế OTA thường được xây dựng dựa trên bảng phân vùng flash có nhiều phân vùng ứng dụng. Theo tài liệu ESP-IDF, firmware mới không được ghi đè trực tiếp lên phân vùng ứng dụng đang chạy. Thay vào đó, firmware mới được tải xuống và ghi vào một phân vùng OTA khác. Sau khi quá trình ghi hoàn tất và image được kiểm tra hợp lệ, thông tin boot được cập nhật để thiết bị khởi động vào firmware mới ở lần reset tiếp theo [18]. Cách tổ chức này giúp giảm rủi ro trong quá trình cập nhật, vì firmware đang chạy không bị ghi đè trực tiếp trong lúc tải bản mới.

Một quy trình OTA cơ bản thường gồm ba giai đoạn. Giai đoạn thứ nhất là kiểm tra phiên bản hoặc cấu hình cập nhật để xác định có firmware mới hay không. Giai đoạn thứ hai là tải firmware mới qua mạng và ghi vào phân vùng OTA phù hợp. Giai đoạn thứ ba là xác minh firmware, cập nhật trạng thái boot và khởi động lại thiết bị để chạy firmware mới. Trong các hệ thống yêu cầu độ tin cậy cao, OTA thường kết hợp thêm version number, sequence number, checksum hoặc hash như SHA-256 để kiểm tra tính toàn vẹn của file và tránh cập nhật nhầm hoặc cập nhật lặp lại cùng một firmware.

NVS có vai trò quan trọng trong cả BLE provisioning và OTA. Đối với BLE provisioning, NVS lưu thông tin WiFi để thiết bị có thể tự kết nối lại sau khi reset. Đối với OTA, NVS có thể lưu trạng thái firmware đã áp dụng, mã định danh phiên bản, sequence number hoặc giá trị hash của firmware gần nhất. Nhờ đó, thiết bị có thể biết bản cập nhật nào đã được cài đặt và tránh tải lại cùng một firmware nhiều lần.

Trong phạm vi đề tài, BLE provisioning và OTA firmware update được xem là các cơ chế hỗ trợ vận hành cho ESP32-S3 Gateway. BLE provisioning giúp gateway cấu hình WiFi linh hoạt trong thực tế mà không cần nạp lại firmware. OTA giúp gateway có khả năng cập nhật phần mềm từ xa khi cần sửa lỗi hoặc bổ sung chức năng. Phần thiết kế chi tiết về điều kiện kích hoạt provisioning, cách lưu thông tin trong NVS, quy trình kiểm tra bản OTA mới và cách tránh cập nhật lặp sẽ được trình bày ở chương thiết kế hệ thống và chương triển khai firmware.

Trên ESP32-S3, OTA thường dựa trên bảng phân vùng có nhiều phân vùng ứng dụng. Theo tài liệu ESP-IDF, cơ chế OTA cho phép ghi firmware mới vào phân vùng không phải phân vùng đang chạy, sau đó cập nhật thông tin boot để thiết bị khởi động vào image mới [18]. Cách tổ chức này giúp giảm rủi ro so với việc ghi đè trực tiếp lên firmware đang chạy.

2.16. Multiple Linear Regression trong dự báo nhiệt độ cục bộ ngắn hạn

Trong hệ thống quan trắc thời tiết cục bộ, dữ liệu cảm biến không chỉ có ý nghĩa tại thời điểm đo hiện tại mà còn có thể được sử dụng để khảo sát xu hướng biến đổi trong tương lai gần. Các đại lượng như nhiệt độ, độ ẩm và áp suất khí quyển thay đổi theo thời gian và chịu ảnh hưởng bởi nhiều yếu tố như chu kỳ ngày đêm, điều kiện che chắn, thông gió, vị trí đặt cảm biến và trạng thái môi trường xung quanh. Khi các dữ liệu này được thu thập liên tục, chúng tạo thành một chuỗi quan sát theo thời gian và có thể được khai thác để xây dựng mô hình dự báo.

Trong phạm vi đề tài, bài toán dự báo được giới hạn ở nhiệt độ cục bộ sau 2 giờ tại vị trí đặt trạm đo. Hệ thống không hướng đến dự báo toàn bộ trạng thái thời tiết cho một khu vực rộng và cũng không thay thế các mô hình khí tượng chuyên nghiệp. Các hệ thống dự báo thời tiết chuyên nghiệp thường cần nhiều nguồn dữ liệu như radar, vệ tinh, trạm khí tượng diện rộng và mô hình khí tượng số. Ngược lại, hệ thống trong đề tài chỉ sử dụng dữ liệu đo tại một điểm cục bộ để tạo ra giá trị dự báo tham khảo trong ngắn hạn. Vì vậy, mô hình được lựa chọn cần có cấu trúc đơn giản, dễ giải thích và phù hợp với khả năng tính toán của gateway nhúng.

Multiple Linear Regression, hay hồi quy tuyến tính đa biến, được sử dụng trong đề tài như một mô hình học máy cơ bản để khai thác mối quan hệ giữa các thông số môi trường và nhiệt độ cần dự báo. Mô hình này phù hợp với giai đoạn khảo sát ban đầu vì công thức rõ ràng, quá trình suy luận nhẹ và có thể triển khai trực tiếp trên ESP32-S3 Gateway sau khi các hệ số đã được huấn luyện trước.

2.16.1. Cơ sở của mô hình Multiple Linear Regression trong dự báo nhiệt độ

Multiple Linear Regression là mô hình hồi quy dùng để biểu diễn mối quan hệ tuyến tính giữa nhiều biến đầu vào và một biến đầu ra liên tục. Trong bài toán dự báo nhiệt độ, biến đầu ra là nhiệt độ cần dự báo tại một thời điểm trong tương lai, còn các biến đầu vào là những thông số có liên quan đến trạng thái môi trường. Nếu hồi quy tuyến tính đơn chỉ xét một biến đầu vào, hồi quy tuyến tính đa biến cho phép mô hình kết hợp nhiều thông tin môi trường trong cùng một công thức.

Dạng tổng quát của mô hình Multiple Linear Regression được viết như sau:

$$\hat{y} = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n \quad (1)$$

Trong đó, $\hat{y}$ là giá trị dự báo của mô hình; $x_1, x_2, \dots, x_n$ là các biến đầu vào; b_0 là hệ số chặn và $b_1, b_2, \dots, b_n$ là các hệ số hồi quy. Các hệ số này được xác định trong quá trình huấn luyện dựa trên dữ liệu lịch sử. Sau khi quá trình huấn luyện hoàn tất, mô hình có thể tạo ra giá trị dự báo mới bằng cách thay các giá trị đầu vào vào công thức đã có hệ số. Trong nghiên cứu này, các tác giả sử dụng dữ liệu khí tượng trong quá khứ và mô hình Multiple Linear Regression để dự báo nhiệt độ trung bình của một ngày. Quy trình của bài báo gồm dữ liệu khí tượng ban đầu, bước lựa chọn đặc trưng, tạo tập dữ liệu tinh lọc, huấn luyện mô hình hồi quy tuyến tính đa biến và đánh giá kết quả dự báo. Bài báo cho thấy mô hình Multiple Linear Regression có thể dự báo nhiệt độ trung bình ngày với sai số trung bình khoảng 2,8 °C [19]. Kết quả của nghiên cứu trên có ý nghĩa đối với đề tài vì nó chứng minh rằng Multiple Linear Regression có thể được dùng như một mô hình cơ bản trong bài toán dự báo nhiệt độ.

2.16.2. Mô hình MLR cơ bản với các biến môi trường.

Khi áp dụng Multiple Linear Regression cho hệ thống quan trắc thời tiết cục bộ, dạng đơn giản nhất của mô hình có thể bắt đầu từ các biến môi trường hiện tại. Các biến này là những thông số được trạm đo đặc đo trực tiếp tại thời điểm lấy mẫu, chẳng hạn như nhiệt độ, độ ẩm và áp suất khí quyển. Nếu chỉ xét mô hình môi trường cơ bản, nhiệt độ sau 2 giờ có thể được biểu diễn theo dạng:

$$\hat{T}(t+2h) = b_0 + b_1T_t + b_2H_t + b_3P_t \quad (2)$$

Trong đó, $\hat{T}(t+2h)$ là nhiệt độ dự báo sau 2 giờ, T_t là nhiệt độ hiện tại, H_t là độ ẩm hiện tại và P_t là áp suất khí quyển hiện tại.

Dạng mô hình này có ý nghĩa vì nhiệt độ hiện tại thường có quan hệ trực tiếp với nhiệt độ trong tương lai gần. Nếu môi trường đang ở mức nhiệt cao, nhiệt độ sau 2 giờ có thể vẫn chịu ảnh hưởng bởi mức nền đó. Độ ẩm cung cấp thêm thông tin về trạng thái không khí, trong khi áp suất khí quyển phản ánh một phần điều kiện khí quyển tại thời điểm đo. Vì vậy, các biến môi trường hiện tại có thể tạo thành nền tảng ban đầu cho mô hình dự báo.

Tuy nhiên, mô hình chỉ dùng nhiệt độ, độ ẩm và áp suất vẫn có một giới hạn về mặt biểu diễn dữ liệu. Các biến này mô tả trạng thái môi trường tại thời điểm đo, nhưng chưa cho mô hình biết mẫu đo đó xảy ra vào thời điểm nào trong ngày. Trong bài toán dự báo nhiệt độ cục bộ sau 2 giờ, thông tin thời điểm đo có thể ảnh hưởng đến xu hướng nhiệt độ tiếp theo. Vì vậy, mô hình MLR cơ bản với các biến môi trường hiện tại là điểm xuất phát, nhưng chưa phải dạng biểu diễn đầy đủ cho bài toán của đề tài.

2.16.3. Vai trò của yếu tố thời gian trong dự báo nhiệt độ cục bộ

Nhiệt độ tại một vị trí cục bộ thường có chu kỳ ngày đêm rõ rệt. Trong ngày, nhiệt độ có xu hướng tăng khi bức xạ mặt trời tác động mạnh hơn và có xu hướng giảm vào buổi tối hoặc ban đêm khi môi trường mất nhiệt. Vì vậy, cùng một giá trị nhiệt độ hiện tại nhưng xuất hiện ở hai thời điểm khác nhau trong ngày có thể dẫn đến xu hướng nhiệt độ sau 2 giờ khác nhau.

Ví dụ, nếu hệ thống đo được một trạng thái môi trường có nhiệt độ, độ ẩm và áp suất gần giống nhau ở hai thời điểm khác nhau, mô hình chỉ dùng các biến môi trường có thể tạo ra cùng một giá trị dự báo. Tuy nhiên, nếu một mẫu xuất hiện vào buổi sáng, nhiệt độ sau 2 giờ có thể đang trong xu hướng tăng. Nếu mẫu tương tự xuất hiện vào buổi tối, nhiệt độ sau 2 giờ có thể đang trong xu hướng giảm. Điểm khác biệt nằm ở bối cảnh thời gian của mẫu đo.

Do đó, việc bổ sung yếu tố thời gian vào mô hình không phải vì MLR bắt buộc phải có biến giờ, mà vì bài toán dự báo nhiệt độ cục bộ sau 2 giờ cần biết mẫu đo đang nằm ở giai đoạn nào trong chu kỳ ngày. Nếu mô hình chỉ được thiết kế để khai thác nhiệt độ, độ ẩm và áp suất, nó vẫn có thể hoạt động như một mô hình cơ bản. Tuy nhiên, khi muốn mô hình phản ánh tốt hơn xu hướng ngày đêm của nhiệt độ, thông tin thời gian cần được đưa vào như một đặc trưng bổ sung.

Khoảng thời gian dự báo sau 2 giờ cũng có liên hệ với nhóm dự báo rất ngắn hạn trong khí tượng học. Một số tài liệu về nowcasting mô tả nowcasting là dạng dự báo thời tiết trong thời gian rất ngắn, thường dưới khoảng 2 giờ [20]. Tuy nhiên, đề tài này không triển khai nowcasting theo nghĩa đầy đủ của khí tượng học vì không sử dụng radar, vệ tinh hoặc mô hình dự báo thời tiết diện rộng. Vì vậy, trong báo cáo, cách gọi phù hợp hơn là dự báo nhiệt độ cục bộ ngắn hạn.

2.16.4. Mã hóa đặc trưng thời gian chu kỳ

Sau khi xác định yếu tố thời gian có ý nghĩa đối với dự báo nhiệt độ cục bộ, vấn đề tiếp theo là cách biểu diễn thời gian trong mô hình. Cách đơn giản nhất là đưa giờ trong ngày vào mô hình dưới dạng số nguyên từ 0 đến 23. Tuy nhiên, cách biểu diễn này không phù hợp với bản chất chu kỳ của thời gian khi sử dụng trong mô hình tuyến tính.

Về mặt thời gian thực, 23 giờ và 0 giờ là hai thời điểm liên kế nhau vì chúng chỉ cách nhau một giờ tại ranh giới giữa hai ngày. Tuy nhiên, nếu biểu diễn giờ bằng số nguyên, hai giá trị 23 và 0 lại cách nhau 23 đơn vị. Đối với mô hình tuyến tính, khoảng cách số học này có thể tạo ra một bước nhảy nhân tạo tại thời điểm chuyển ngày. Khi đó, mô hình có thể hiểu sai rằng cuối ngày và đầu ngày là hai vùng thời gian rất xa nhau, trong khi thực tế chúng nằm cạnh nhau trong chu kỳ 24 giờ.

Tài liệu scikit-learn về time-related feature engineering cũng chỉ ra vấn đề tương tự đối với đặc trưng thời gian dạng số thứ tự. Khi dùng trực tiếp các đặc trưng thời gian nguyên thủy cho mô hình tuyến tính, mô hình có thể không nắm bắt được tính tuần hoàn tự nhiên của thời gian. Tài liệu này nêu rằng đặc trưng thời gian dạng ordinal có thể gây ra bước nhảy trong dự báo khi giờ chuyển từ 23 về 0, đồng thời đề xuất mã hóa các đặc trưng tuần hoàn bằng biến đổi sine và cosine với chu kỳ tương ứng [21].

Vì vậy, thay vì đưa trực tiếp giờ (h) vào mô hình, giờ trong ngày được mã hóa thành hai đặc trưng:

$$\text{hour_sin} = \sin((2\pi h) / 24) \quad (3)$$

$$\text{hour_cos} = \cos((2\pi h) / 24) \quad (4)$$

Cách biểu diễn này có thể được hiểu như việc đặt thời điểm trong ngày lên một vòng tròn có chu kỳ 24 giờ. Mỗi giờ trong ngày tương ứng với một vị trí trên vòng tròn. Khi thời gian đi từ 23 giờ sang 0 giờ, vị trí trên vòng tròn thay đổi liên tục, không còn bước nhảy lớn như khi biểu diễn bằng số nguyên. Nhờ đó, mô hình có thể biểu diễn tính liên tục giữa cuối ngày và đầu ngày.

Với chu kỳ dài hơn, chẳng hạn ngày trong năm, cách mã hóa tương tự cũng có thể được sử dụng. Nếu (d) là ngày trong năm, đặc trưng thời gian theo năm có thể được biểu diễn như sau:

$$\text{day_sin} = \sin((2\pi d) / 366) \quad (5)$$

$$\text{day_cos} = \cos((2\pi d) / 366) \quad (6)$$

Việc dùng chu kỳ 366 ngày giúp bao quát cả trường hợp năm nhuận. Trong phạm vi đề tài, đặc trưng ngày trong năm được xem như thông tin bổ sung về bối cảnh thời gian dài hơn chu kỳ ngày đêm. Ý nghĩa chính của cách mã hóa sin/cos là làm cho biến thời gian phù hợp hơn với bản chất tuần hoàn của dữ liệu thời tiết.

Điều quan trọng là việc sử dụng sin và cos không làm mô hình trở thành một mô hình phi tuyến phức tạp. Các hàm sin/cos chỉ được dùng ở bước tạo đặc trưng đầu vào. Sau khi tính toán, (hour_sin), (hour_cos), (day_sin) và (day_cos) được xem như các biến đầu vào mới. Mô hình vẫn nhận từng biến đầu vào với hệ số hồi quy tương ứng rồi cộng lại. Vì vậy, bản chất của mô hình vẫn là Multiple Linear Regression theo các đặc trưng đã được biến đổi.

Cách dùng sin và cos để biểu diễn đặc trưng thời gian cũng tương đồng với các thành phần Fourier trong dự báo chuỗi thời gian. Giáo trình Forecasting: Principles and Practice trình bày rằng các Fourier terms, gồm các cặp sin và cos, có thể được đưa vào mô hình hồi quy để biểu diễn các chu kỳ hoặc tính mùa vụ của dữ liệu [22]. Điều này cho thấy việc bổ sung sin/cos vào mô hình hồi quy là một cách tạo đặc trưng có cơ sở lý thuyết, không phải một phép thêm tùy ý.

2.16.5. Dạng mô hình MLR mở rộng áp dụng trong đề tài

Từ các lập luận trên, mô hình trong đề tài có thể được hiểu là mô hình Multiple Linear Regression cơ bản được mở rộng bằng đặc trưng thời gian chu kỳ. Phần nền tảng của mô hình vẫn là hồi quy tuyến tính đa biến. Phần mở rộng nằm ở việc biến thời gian không được đưa trực tiếp vào công thức dưới dạng số nguyên, mà được mã hóa thành các thành phần sin/cos để phản ánh chu kỳ ngày đêm và chu kỳ theo ngày trong năm.

Dạng tổng quát của mô hình áp dụng trong đề tài có thể viết như sau:

$$\hat{T}(t+2h) = b_0 + b_1T_t + b_2H_t + b_3(P_t - 1000) + b_4\sin(2\pi h / 24) + b_5\cos(2\pi h / 24) + b_6\sin(2\pi d / 366) + b_7\cos(2\pi d / 366) + \beta_h \quad (7)$$

Trong công thức trên, $\hat{T}(t+2h)$ là nhiệt độ dự báo sau 2 giờ. Ký hiệu (T_t) là nhiệt độ hiện tại, (H_t) là độ ẩm tương đối hiện tại, (P_t) là áp suất khí quyển hiện tại, (h) là giờ trong ngày và (d) là ngày trong năm. Thành phần $((P_t - 1000))$ đưa áp suất về dạng lệch quanh mốc tham chiếu 1000 hPa, giúp giá trị áp suất trong công thức gọn hơn. Các hệ số ($b_0, b_1, \dots, b_7$) là các hệ số hồi quy được xác định trong quá trình huấn luyện.

Thành phần (β_h) biểu diễn hệ số hiệu chỉnh theo giờ. Trong phạm vi lý thuyết, thành phần này có thể được hiểu là một hiệu chỉnh bổ sung nhằm hỗ trợ mô hình giảm sai lệch có tính lặp lại theo từng thời điểm trong ngày. Tuy nhiên, giá trị cụ thể của (β_h) và các hệ số hồi quy không trình bày trong mục cơ sở lý thuyết, vì chúng phụ thuộc vào quá trình huấn luyện và triển khai thực tế.

Công thức trên chỉ mô tả dạng mô hình ở mức lý thuyết. Trong hệ thống thực tế, các hệ số của mô hình được huấn luyện trước từ dữ liệu lịch sử, sau đó được nhúng vào firmware của ESP32-S3 Gateway. Khi gateway nhận dữ liệu mới từ trạm đo, gateway chỉ thực hiện quá trình suy luận bằng cách tính các đặc trưng đầu vào, thay vào công thức hồi quy và tạo ra giá trị nhiệt độ dự báo sau 2 giờ.

Như vậy, mô hình trong đề tài không phải là mô hình học sâu, không phải mô hình khí tượng số và cũng không phải một hệ thống nowcasting đầy đủ. Đây là mô hình Multiple Linear Regression được mở rộng bằng đặc trưng thời gian chu kỳ. Cách tiếp cận này kế thừa nền tảng MLR đã được sử dụng trong nghiên cứu dự báo nhiệt độ, đồng thời bổ sung biểu diễn thời gian bằng sin/cos để phù hợp hơn với dữ liệu nhiệt độ cục bộ có tính tuần hoàn theo ngày đêm.

CHƯƠNG 3. PHÂN TÍCH YÊU CẦU VÀ THIẾT KẾ HỆ THỐNG

3.1. Định hướng thiết kế hệ thống

Hệ thống quan trắc thời tiết cục bộ được thiết kế nhằm thu thập dữ liệu môi trường tại vị trí quan trắc, truyền dữ liệu không dây về bộ trung tâm, xử lý dữ liệu, lưu trữ trên nền tảng trực tuyến và hiển thị thông tin cho người dùng thông qua giao diện giám sát. Do hệ thống cần hoạt động trong điều kiện ngoài trời và không phụ thuộc hoàn toàn vào nguồn điện cố định, kiến trúc thiết kế phải bảo đảm ba yêu cầu quan trọng: thiết bị đo có khả năng vận hành độc lập, dữ liệu truyền giữa các khối phải có cấu trúc rõ ràng và tầng giám sát phải có khả năng theo dõi trạng thái vận hành của hệ thống.

Theo định hướng trên, hệ thống được chia thành ba thành phần chính: STM32 Sensor Node, ESP32-S3 Gateway và Supabase Cloud/Web Dashboard. STM32 Sensor Node là thiết bị đặt tại vị trí quan trắc, đảm nhiệm nhiệm vụ gần phần cứng như đọc cảm biến, giám sát trạng thái nguồn, đóng gói dữ liệu, mã hóa payload và truyền dữ liệu qua LoRa. ESP32-S3 Gateway là bộ trung tâm tiếp nhận dữ liệu từ tuyến LoRa, kiểm tra tính hợp lệ của frame, giải mã dữ liệu, khôi phục giá trị cảm biến, thực hiện xử lý trung gian và gửi dữ liệu lên nền tảng Cloud. Supabase Cloud và Web Dashboard là tầng lưu trữ, quản lý và hiển thị dữ liệu cho người dùng.

Việc tách STM32 Sensor Node và ESP32-S3 Gateway thành hai khối xử lý riêng giúp hệ thống phù hợp hơn với đặc điểm của từng vi điều khiển. STM32F103C8T6 phù hợp với các tác vụ nhúng cơ bản như giao tiếp cảm biến, xử lý dữ liệu theo chu kỳ, điều khiển module LoRa và vận hành ở chế độ tiết kiệm năng lượng. Tuy nhiên, STM32 không phù hợp để trực tiếp đảm nhiệm các tác vụ như WiFi, HTTPS/TLS, BLE provisioning, OTA firmware update hoặc gửi dữ liệu trực tiếp lên Cloud. Ngược lại, ESP32-S3 có WiFi, BLE, bộ nhớ lớn hơn và được hỗ trợ bởi ESP-IDF/FreeRTOS, do đó phù hợp hơn với vai trò gateway xử lý, kết nối mạng và hỗ trợ các chức năng vận hành nâng cao.

Trong kiến trúc này, STM32 Sensor Node không cần xử lý các chức năng liên quan đến Internet, Supabase, dashboard hoặc OTA. Node chỉ tập trung vào việc tạo dữ liệu cảm biến đáng tin cậy và truyền dữ liệu về gateway theo chu kỳ. Cách tổ chức này giúp firmware STM32 đơn giản hơn, giảm số trạng thái cần quản lý và thuận lợi cho việc tối ưu năng lượng. Đồng thời, ESP32-S3 Gateway có thể tập trung vào các tác vụ yêu cầu tài nguyên cao hơn như quản lý WiFi, nhận frame LoRa, xác thực và giải mã dữ liệu, dự báo nhiệt độ, gửi dữ liệu lên Cloud, kiểm tra OTA và gửi heartbeat trạng thái thiết bị.

Về mặt luồng dữ liệu, hệ thống được thiết kế theo hướng dữ liệu đi từ lớp thiết bị đo đến lớp ứng dụng. Dữ liệu ban đầu được tạo tại STM32 Sensor Node từ các cảm biến môi trường và cảm biến giám sát nguồn. Sau đó, dữ liệu được đóng gói thành payload nhị phân, mã hóa và truyền qua LoRa đến ESP32-S3 Gateway. Gateway xử lý frame nhận được, khôi phục dữ liệu về giá trị vật lý, bổ sung thông tin xử lý cục bộ và gửi dữ liệu lên Supabase thông qua HTTPS. Sau khi dữ liệu được lưu trong cơ sở dữ liệu, Web Dashboard truy vấn và hiển thị thông tin cho người dùng.

Bên cạnh luồng dữ liệu cảm biến, hệ thống còn có các luồng vận hành phụ để hỗ trợ triển khai thực tế. Gateway gửi heartbeat định kỳ để cập nhật trạng thái hoạt động lên Cloud. Web Dashboard hỗ trợ theo dõi dữ liệu mới nhất, biểu đồ lịch sử, trạng thái gateway, trạng thái tuyến LoRa và thông tin firmware. BLE provisioning được sử dụng để cấu hình WiFi cho gateway khi triển khai ở môi trường mới. OTA firmware update được thiết kế để cập nhật firmware ESP32-S3 Gateway từ xa thông qua Supabase Storage và bảng cấu hình thiết bị.

3.2. Yêu cầu hệ thống

Yêu cầu hệ thống được xác định dựa trên mục tiêu xây dựng một mô hình quan trắc thời tiết cục bộ có khả năng hoạt động độc lập, thu thập dữ liệu môi trường theo chu kỳ, truyền dữ liệu không dây, bảo vệ dữ liệu bằng cơ chế mã hóa xác thực, lưu trữ dữ liệu trên nền tảng trực tuyến và hiển thị thông tin cho người dùng thông qua giao diện giám sát. Các yêu cầu này được chia thành các nhóm chính gồm yêu cầu thu thập dữ liệu, yêu cầu xử lý tại thiết bị đo, yêu cầu truyền thông, yêu cầu bảo mật, yêu cầu gateway, yêu cầu Cloud, yêu cầu giao diện và yêu cầu triển khai ngoài môi trường.

Việc thu thập dữ liệu hệ thống cần đo được các thông số môi trường cơ bản phục vụ quan trắc thời tiết cục bộ, bao gồm nhiệt độ, độ ẩm và áp suất khí quyển. Ngoài các thông số môi trường, thiết bị đo cần theo dõi trạng thái nguồn để đánh giá khả năng vận hành của node trong điều kiện dùng pin và năng lượng mặt trời. Dữ liệu không cần được truyền liên tục theo thời gian thực nghiêm ngặt, nhưng cần được thu thập theo chu kỳ ổn định để phục vụ theo dõi xu hướng biến đổi môi trường.

Khi xử lý dữ liệu STM32 Sensor Node cần đọc dữ liệu từ các cảm biến, kiểm tra trạng thái lỗi, chuẩn hóa dữ liệu và đóng gói thành payload có cấu trúc cố định. Dữ liệu cảm biến không nên truyền trực tiếp ở dạng số thực vì điều này làm tăng kích thước bản tin và gây khó khăn cho quá trình truyền qua LoRa. Thay vào đó, các giá trị đo cần được chuyển sang dạng số nguyên theo hệ số tỉ lệ trước khi đóng gói. Payload cũng cần có trường trạng thái để biểu diễn lỗi cảm biến hoặc lỗi đọc dữ liệu nếu có.

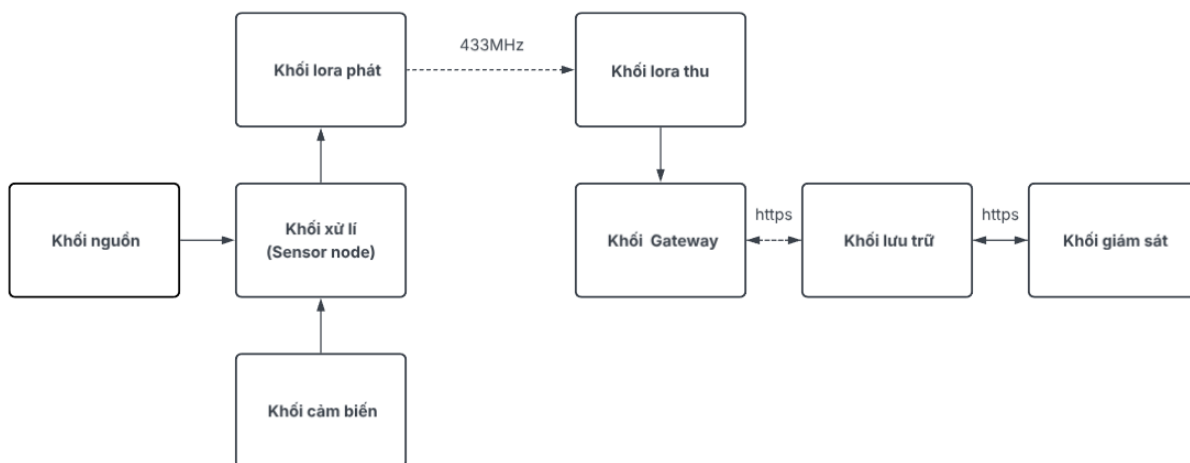
Vấn đề sử dụng hệ thống sử dụng tuyến LoRa để truyền dữ liệu từ thiết bị đo ngoài trời về bộ trung tâm Gateway. Tuyến truyền cần hỗ trợ việc gửi frame dữ liệu theo chu kỳ trong phạm vi quan trắc cục bộ. Do LoRa là tuyến truyền không dây và có thể chịu ảnh hưởng bởi khoảng cách, vật cản, vị trí anten và môi trường nhiễu, gateway cần kiểm tra kích thước frame trước khi xử lý. Frame không đúng kích thước hoặc không vượt qua bước xác thực phải bị loại bỏ để tránh dữ liệu sai đi vào các bước xử lý phía sau.

Để dữ liệu được bảo mật thì hệ thống cần có cơ chế bảo vệ payload cảm biến khi truyền qua tuyến LoRa và khi gửi từ gateway lên nền tảng Cloud. Cơ chế bảo vệ không chỉ nhằm che giấu nội dung payload mà còn phải giúp phát hiện dữ liệu bị sai lệch hoặc không được tạo bởi thành phần có khóa hợp lệ. Vì vậy, thuật toán mã hóa xác thực cần được sử dụng để kết hợp cả mã hóa và kiểm tra tính toàn vẹn dữ liệu. Ngoài ra, tuyến gửi dữ liệu lên Cloud cần có cơ chế kiểm tra token thiết bị để hạn chế request không hợp lệ.

Đối với Gateway, ESP32-S3 Gateway cần đóng vai trò trung gian giữa mạng LoRa cục bộ và nền tảng trực tuyến. Gateway cần nhận dữ liệu từ module LoRa, kiểm tra frame, giải mã payload, khôi phục dữ liệu cảm biến về đơn vị vật lý, thực hiện xử lý cục bộ nếu cần và gửi dữ liệu lên Cloud thông qua HTTPS. Ngoài luồng dữ liệu chính, gateway cần hỗ trợ heartbeat trạng thái, cấu hình WiFi bằng BLE provisioning và cập nhật firmware từ xa bằng OTA.

Đảm bảo nhu cầu lưu trữ và hiển thị, dữ liệu sau khi được gateway xử lý cần được gửi lên nền tảng Cloud để lưu trữ, truy vấn lịch sử và phục vụ giao diện giám sát. Cơ sở dữ liệu cần lưu các thông tin như mã thiết bị, thời gian ghi nhận, nhiệt độ, độ ẩm, áp suất, trạng thái nguồn, health_flag, frame_counter và giá trị dự báo nếu có. Giao diện giám sát cần hiển thị dữ liệu mới nhất, biểu đồ lịch sử, trạng thái gateway, trạng thái tuyến LoRa, trạng thái firmware và thông tin OTA.

Sử dụng nguồn và triển khai ngoài môi trường, STM32 Sensor Node cần có khả năng hoạt động độc lập trong điều kiện không cấp nguồn trực tiếp từ lưới điện. Vì vậy, node cần sử dụng khối nguồn gồm tấm pin năng lượng mặt trời, mạch sạc pin, pin lưu trữ, mạch bảo vệ pin và bộ chuyển đổi nguồn ổn định. Ngoài ra, thiết bị cần được đặt trong kết cấu bảo vệ phù hợp với điều kiện ngoài trời, bao gồm vỏ bảo vệ, khu vực che chắn cảm biến và giải pháp hạn chế hơi ẩm trong hộp.



Hình 3.1. Sơ đồ khối chức năng của hệ thống quan trắc thời tiết cục bộ

Từ sơ đồ khối chức năng ở Hình 3.1, có thể thấy các yêu cầu của hệ thống được phân bổ theo từng nhóm chức năng cụ thể. Khối nguồn cần bảo đảm khả năng cấp năng lượng ổn định cho thiết bị đo trong điều kiện vận hành ngoài môi trường. Khối cảm biến chịu trách nhiệm tạo dữ liệu đầu vào gồm nhiệt độ, độ ẩm, áp suất khí quyển và trạng thái nguồn. Khối xử lý tại Sensor Node cần đọc dữ liệu cảm biến, kiểm tra lỗi, chuẩn hóa dữ liệu, đóng gói payload và điều khiển quá trình truyền LoRa. Khối LoRa phát và LoRa thu đảm nhiệm tuyến truyền dữ liệu cục bộ giữa thiết bị đo và bộ trung tâm. Khối Gateway cần tiếp nhận frame LoRa, kiểm tra tính hợp lệ, giải mã dữ liệu, xử lý trung gian và gửi dữ liệu lên khối lưu trữ thông qua HTTPS. Khối lưu trữ có nhiệm vụ tiếp nhận, kiểm tra và lưu dữ liệu hợp lệ, trong khi khối giám sát cung cấp giao diện hiển thị dữ liệu, trạng thái vận hành và các chức năng hỗ trợ như cập nhật firmware.

Như vậy, các yêu cầu hệ thống không được xem xét riêng lẻ mà được gắn với từng khối chức năng trong sơ đồ tổng thể. Cách xác định này giúp quá trình thiết kế ở các mục tiếp theo có cơ sở rõ ràng hơn: thiết kế Sensor Node tập trung vào đo lường, nguồn và truyền LoRa; thiết kế Gateway tập trung vào tiếp nhận, xử lý và gửi dữ liệu; thiết kế Cloud và giao diện tập trung vào lưu trữ, hiển thị và hỗ trợ vận hành hệ thống.

3.3. Kiến trúc tổng thể và luồng hoạt động của hệ thống

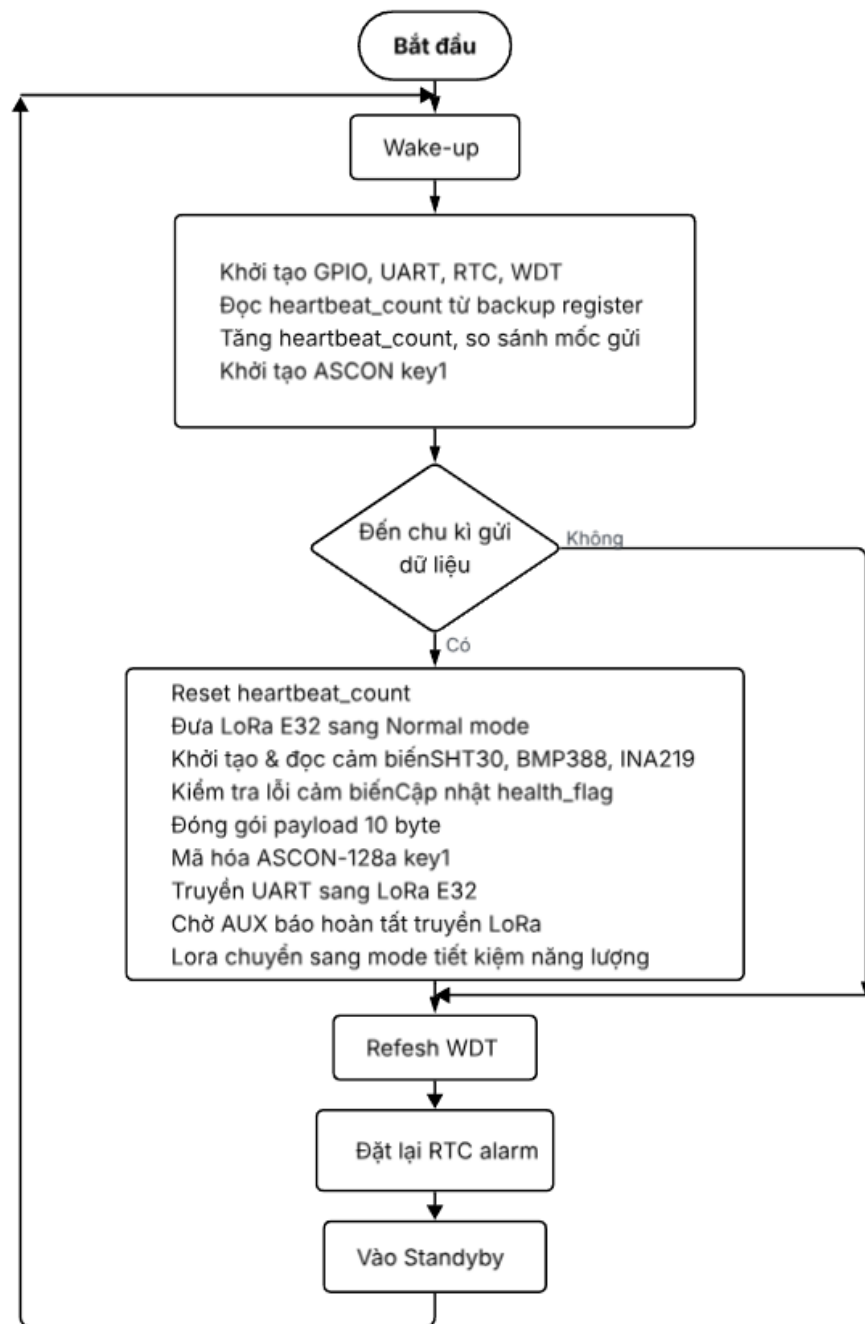
Ở mức kiến trúc tổng thể, hệ thống được tổ chức thành các khối triển khai chính gồm STM32 Sensor Node, tuyến truyền LoRa, ESP32-S3 Gateway, nền tảng Supabase Cloud và Web Dashboard. Các khối này được bố trí theo chuỗi xử lý dữ liệu liên tục, trong đó dữ liệu được tạo ra tại vị trí quan trắc, truyền về bộ trung tâm, xử lý tại Gateway, lưu trữ trên

Cloud và hiển thị trên giao diện giám sát. Cách tổ chức này giúp tách biệt rõ nhiệm vụ giữa phần đo lường tại hiện trường, phần truyền thông cục bộ, phần xử lý trung gian và phần ứng dụng phục vụ người dùng.

STM32 Sensor Node là khối đầu cuối của hệ thống, đảm nhiệm việc thu thập dữ liệu từ cảm biến, giám sát trạng thái nguồn, đóng gói dữ liệu và truyền frame qua LoRa. Tuyến LoRa đóng vai trò kết nối không dây giữa node đo và Gateway, phù hợp với yêu cầu truyền dữ liệu cục bộ trong phạm vi quan trắc. ESP32-S3 Gateway là khối trung gian giữa mạng LoRa và Internet, thực hiện tiếp nhận frame, kiểm tra tính hợp lệ, giải mã, xử lý dữ liệu và gửi kết quả lên Cloud. Supabase Cloud đảm nhiệm vai trò tiếp nhận, lưu trữ, quản lý dữ liệu và hỗ trợ lưu trữ firmware phục vụ cập nhật OTA. Web Dashboard là khối giao diện, dùng để hiển thị dữ liệu quan trắc, biểu đồ lịch sử, trạng thái Gateway và các thông tin vận hành của hệ thống.

Luồng hoạt động của hệ thống được chia thành hai phần để thuận tiện cho việc mô tả thiết kế. Phần thứ nhất là chu trình hoạt động của STM32 Sensor Node. Do node được định hướng hoạt động bằng pin kết hợp năng lượng mặt trời, firmware không vận hành liên tục ở trạng thái hoạt động đầy đủ mà làm việc theo chu kỳ. Sau khi được đánh thức, STM32 kiểm tra trạng thái chu kỳ gửi dữ liệu. Nếu chưa đến thời điểm truyền dữ liệu, node chỉ thực hiện các thao tác duy trì cần thiết như làm mới watchdog, đặt lại RTC alarm và quay lại chế độ Standby. Nếu đã đến chu kỳ gửi dữ liệu, node thực hiện quá trình đọc cảm biến, kiểm tra lỗi, cập nhật trạng thái, đóng gói payload, mã hóa dữ liệu và truyền frame qua module LoRa E32.

Hình 3.2 mô tả lưu đồ hoạt động của STM32 Sensor Node. Lưu đồ này tập trung vào các trạng thái chính của node gồm đánh thức, kiểm tra chu kỳ gửi, đọc cảm biến, xử lý dữ liệu, tạo frame LoRa và quay lại chế độ tiết kiệm năng lượng. Việc thể hiện nhánh “đã đến chu kỳ gửi dữ liệu” và “chưa đến chu kỳ gửi dữ liệu” giúp làm rõ cơ chế tiết kiệm năng lượng của node trong quá trình vận hành ngoài môi trường.



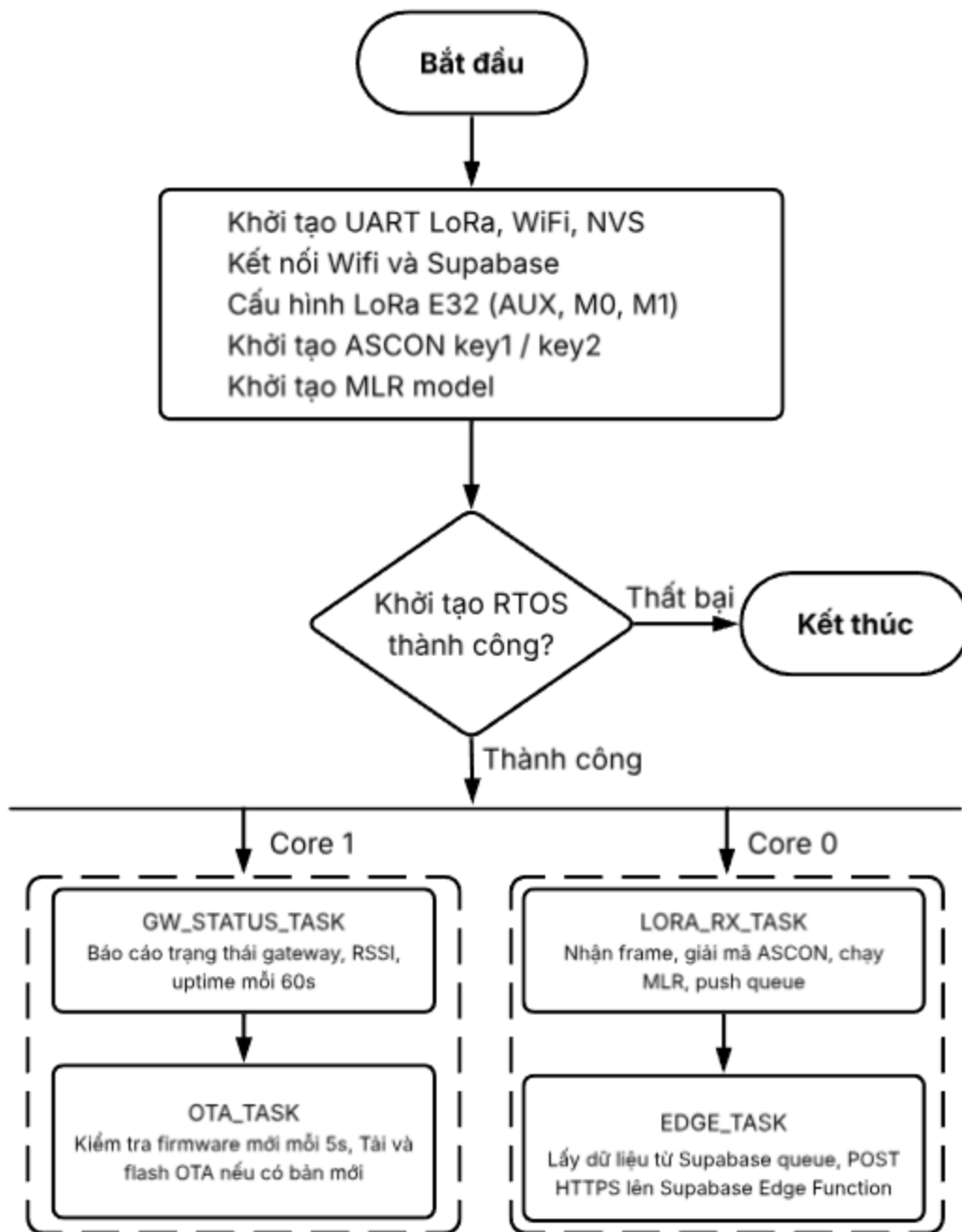
Hình 3.2. Lưu đồ hoạt động của STM32 Sensor Node

Phần thứ hai là luồng hoạt động của ESP32-S3 Gateway và khối lưu trữ. Gateway bắt đầu ở trạng thái chờ dữ liệu từ module LoRa thu. Khi nhận được frame, Gateway kiểm tra độ dài frame và cấu trúc dữ liệu trước khi đưa vào xử lý. Những frame không hợp lệ sẽ bị loại bỏ để tránh ghi dữ liệu sai lên Cloud. Với frame hợp lệ, Gateway tách các trường dữ liệu,

xác thực và giải mã payload, khôi phục giá trị vật lý của các thông số đo, sau đó bổ sung kết quả xử lý cục bộ trước khi gửi dữ liệu lên Supabase thông qua HTTPS.

Ở phía Cloud, Supabase Edge Function tiếp nhận request từ Gateway, kiểm tra thông tin xác thực và xử lý dữ liệu được gửi lên. Dữ liệu hợp lệ được ghi vào PostgreSQL Database để phục vụ lưu trữ và truy vấn. Web Dashboard sử dụng dữ liệu đã lưu trên Cloud để cập nhật thông tin quan trắc cho người dùng. Luồng xử lý này giúp tách phần giao diện khỏi phần nhúng, đồng thời bảo đảm dữ liệu hiển thị trên dashboard là dữ liệu đã qua bước xử lý và kiểm tra tại Gateway cũng như Cloud.

Hình 3.3 trình bày lưu đồ hoạt động của ESP32-S3 Gateway và khối lưu trữ. Lưu đồ này thể hiện chuỗi xử lý từ lúc Gateway nhận frame LoRa đến khi dữ liệu được lưu trên Cloud và hiển thị trên giao diện giám sát. Các nhánh kiểm tra trong lưu đồ được sử dụng để thể hiện quá trình loại bỏ frame lỗi, xử lý trường hợp xác thực không hợp lệ và chỉ ghi dữ liệu vào cơ sở dữ liệu khi request đáp ứng các điều kiện kiểm tra.



Hình 3.3. Lưu đồ hoạt động của ESP32-S3 Gateway và khối lưu trữ

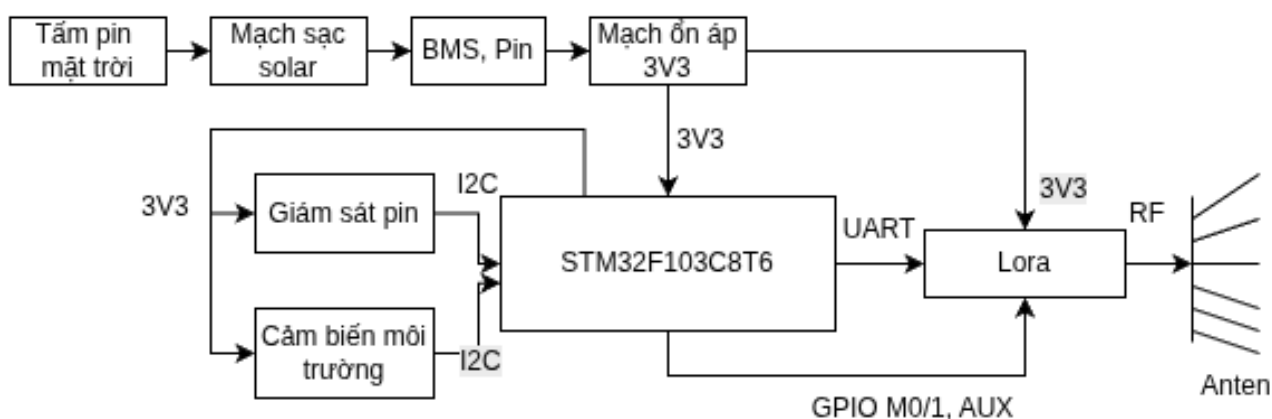
Với cách tổ chức trên, kiến trúc tổng thể của hệ thống được mô tả theo cả góc nhìn khối chức năng và luồng vận hành. Các mục tiếp theo sẽ đi vào thiết kế chi tiết của từng thành phần chính, bắt đầu từ STM32 Sensor Node, sau đó đến cấu trúc payload và frame truyền, cơ chế bảo mật, ESP32-S3 Gateway, Supabase Cloud và giao diện giám sát.

3.4. Thiết kế STM32 Sensor Node

STM32 Sensor Node là thiết bị đo được đặt tại vị trí quan trắc, có nhiệm vụ thu thập dữ liệu môi trường, giám sát trạng thái nguồn và truyền dữ liệu về bộ trung tâm thông qua tuyến LoRa. Trong hệ thống, node được thiết kế theo hướng hoạt động độc lập, sử dụng nguồn pin kết hợp năng lượng mặt trời và làm việc theo chu kỳ để giảm tiêu thụ năng lượng. Do đó, phần thiết kế STM32 Sensor Node không chỉ tập trung vào việc đọc cảm biến, mà còn cần xét đến khối nguồn, trạng thái vận hành của module LoRa, cơ chế ngủ/thức và khả năng duy trì hoạt động dài hạn ngoài môi trường.

Về cấu trúc phần cứng, STM32 Sensor Node được chia thành bốn nhóm chức năng chính: khối nguồn, khối cảm biến, khối xử lý trung tâm và khối truyền thông LoRa. Khối nguồn cung cấp năng lượng cho toàn bộ node. Khối cảm biến tạo dữ liệu đầu vào phục vụ quan trắc. Khối xử lý trung tâm điều phối quá trình đọc dữ liệu, kiểm tra lỗi, chuẩn hóa dữ liệu và tạo frame truyền. Khối truyền thông LoRa đảm nhiệm việc gửi dữ liệu từ node về ESP32-S3 Gateway.

Hình 3.4 trình bày sơ đồ khối phần cứng của STM32 Sensor Node. Hình này thể hiện quan hệ giữa khối nguồn, vi điều khiển STM32F103C8T6, các cảm biến và module LoRa ở mức thiết kế chức năng.



Hình 3.4. Sơ đồ khối phần cứng STM32 Sensor Node

Khối nguồn của STM32 Sensor Node được thiết kế để hỗ trợ thiết bị hoạt động độc lập ngoài môi trường. Nguồn năng lượng đầu vào được lấy từ tấm pin năng lượng mặt trời và đưa vào module sạc CN3791 để nạp cho pin Lithium EVE 1S. Pin lưu trữ được kết hợp với mạch bảo vệ pin 1S nhằm hạn chế các trạng thái bất lợi trong quá trình sạc và xả. Điện áp từ pin sau đó được đưa qua bộ chuyển đổi TPS63020 để tạo đường nguồn ổn định cấp

cho STM32F103C8T6, cụm cảm biến và module LoRa E32. Cách tổ chức này giúp node không phụ thuộc trực tiếp vào nguồn USB hoặc adapter trong quá trình vận hành.

Khối cảm biến của node gồm SHT30, BMP388 và INA219. SHT30 được sử dụng để đo nhiệt độ môi trường và độ ẩm tương đối. BMP388 được sử dụng để đo áp suất khí quyển, đồng thời cung cấp thêm giá trị nhiệt độ tại khu vực cảm biến. INA219 được sử dụng để giám sát trạng thái nguồn cấp của node, hỗ trợ đánh giá điều kiện vận hành của thiết bị đo. Các cảm biến này được kết nối với STM32 thông qua bus I2C, giúp giảm số lượng chân tín hiệu cần sử dụng và thuận lợi cho việc mở rộng hoặc kiểm thử từng cảm biến.

Khối xử lý trung tâm sử dụng vi điều khiển STM32F103C8T6. STM32 đảm nhiệm việc khởi tạo ngoại vi, giao tiếp I2C với các cảm biến, đọc dữ liệu đo, kiểm tra trạng thái lỗi, chuyển đổi dữ liệu sang dạng số nguyên theo hệ số tỉ lệ, đóng gói payload, mã hóa dữ liệu và điều khiển module LoRa E32. Việc đưa quá trình chuẩn hóa và đóng gói dữ liệu xuống STM32 giúp dữ liệu gửi đi có kích thước nhỏ, cấu trúc cố định và phù hợp với đặc điểm truyền dữ liệu dung lượng thấp của LoRa.

Khối truyền thông của node sử dụng module LoRa E32. STM32 giao tiếp với module LoRa thông qua UART để gửi frame dữ liệu. Ngoài đường UART, STM32 còn điều khiển các chân M0, M1 và theo dõi chân AUX của module LoRa nhằm xác định trạng thái hoạt động của module. Trước khi truyền dữ liệu, STM32 đưa module LoRa về chế độ hoạt động phù hợp. Sau khi truyền xong, module được đưa về trạng thái tiết kiệm năng lượng để giảm tiêu thụ trong khoảng thời gian chờ chu kỳ tiếp theo.

Trong thiết kế này, TPS63020 không chỉ đóng vai trò tạo nguồn ổn định mà còn được điều khiển theo trạng thái truyền dữ liệu. Chân PS/SYNC của TPS63020 được STM32 điều khiển để thay đổi chế độ vận hành của bộ chuyển đổi nguồn trong thời điểm module LoRa phát dữ liệu. Khi chuẩn bị truyền, firmware đưa PS/SYNC về trạng thái phù hợp để hỗ trợ tải tức thời của module LoRa. Sau khi truyền xong, tín hiệu này được đưa về trạng thái tiết kiệm năng lượng. Cách điều khiển này giúp khối nguồn phối hợp tốt hơn với chu kỳ truyền dữ liệu của node.

Chu kỳ hoạt động của STM32 Sensor Node được thiết kế theo hướng tiết kiệm năng lượng. Node không đọc cảm biến và truyền dữ liệu liên tục, mà chỉ thực hiện chu trình đo và truyền khi đến thời điểm lấy mẫu. Trong phần lớn thời gian, STM32 ở chế độ Standby. RTC alarm được sử dụng để đánh thức vi điều khiển theo chu kỳ. Sau mỗi lần đánh thức, firmware kiểm tra biến đếm heartbeat để xác định đã đến chu kỳ gửi dữ liệu hay chưa. Nếu chưa đến chu kỳ gửi, STM32 chỉ làm mới watchdog, đặt lại RTC alarm và quay lại Standby.

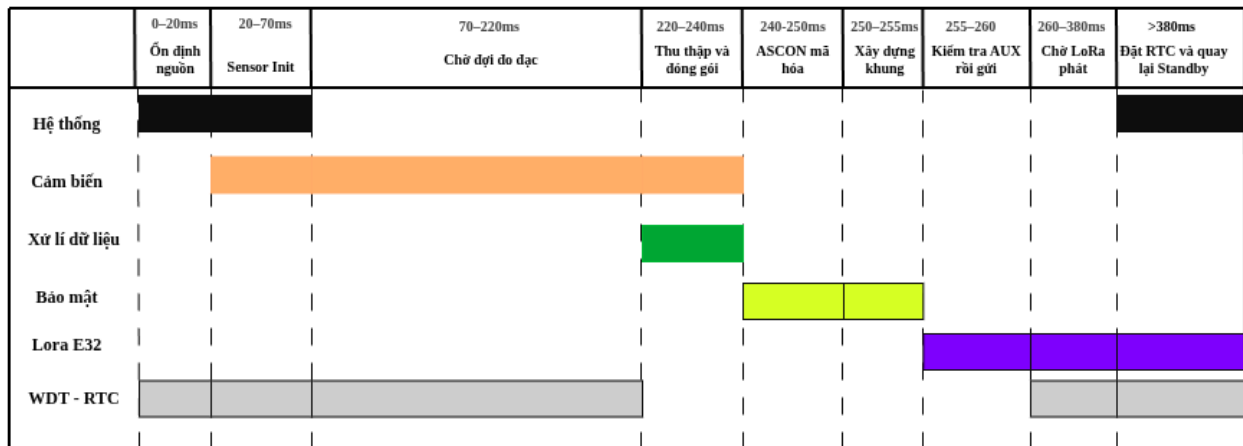
Nếu đã đến chu kỳ gửi, STM32 thực hiện đầy đủ quá trình đọc cảm biến, đóng gói, mã hóa và truyền LoRa.

Independent Watchdog được sử dụng để tăng độ tin cậy vận hành của node. Watchdog có nhiệm vụ reset hệ thống nếu firmware bị treo hoặc không còn làm mới watchdog đúng thời gian. Do node có nhiều trạng thái vận hành như Standby, wake-up, đọc cảm biến, truyền LoRa và quay lại ngủ, watchdog giúp hạn chế tình trạng hệ thống bị dừng ở một trạng thái không mong muốn trong quá trình vận hành dài hạn. Chu kỳ RTC heartbeat cần được cấu hình phù hợp để STM32 có thể thức dậy, làm mới watchdog và quay lại Standby trước khi watchdog timeout.

Khi đến chu kỳ gửi dữ liệu, trình tự xử lý của STM32 bắt đầu từ quá trình đánh thức khỏi Standby. Sau đó, node đưa module LoRa sang chế độ hoạt động, khởi tạo hoặc kiểm tra trạng thái cảm biến, chờ cảm biến ổn định và đọc dữ liệu từ SHT30, BMP388 và INA219. Các giá trị đọc được được kiểm tra lỗi, sau đó firmware cập nhật trường health_flag nếu phát hiện cảm biến hoặc giao tiếp có vấn đề. Dữ liệu hợp lệ được chuyển sang dạng số nguyên theo hệ số tỉ lệ để chuẩn bị đóng gói.

Sau bước xử lý dữ liệu cảm biến, STM32 tạo payload cảm biến 10 byte. Payload này được mã hóa bằng ASCON-128a với khóa thứ nhất. Sau khi mã hóa, STM32 ghép frame_counter, ciphertext và authentication tag để tạo thành frame LoRa 18 byte. Frame này được gửi qua UART đến module LoRa E32 để truyền về gateway. Sau khi quá trình truyền hoàn tất, STM32 kiểm tra trạng thái AUX, đưa LoRa về chế độ tiết kiệm năng lượng, làm mới watchdog, đặt RTC alarm cho lần đánh thức tiếp theo và quay lại Standby.

Hình 3.5 mô tả lát cắt thời gian của STM32 Sensor Node khi đến chu kỳ truyền dữ liệu. Trong phần lớn thời gian vận hành, node ở trạng thái Standby để giảm tiêu thụ năng lượng. Khi đến thời điểm truyền, vi điều khiển được đánh thức bởi RTC và lần lượt thực hiện các bước: ổn định nguồn, đọc dữ liệu cảm biến, tạo payload, mã hóa ASCON-128a, đóng gói frame LoRa và gửi dữ liệu qua module LoRa E32.



Hình 3.5. Biểu đồ time-slice trong một chu kỳ đo và truyền dữ liệu của STM32

Các mốc thời gian trong hình được xây dựng dựa trên các tham số cấu hình trong firmware. Trong đó, node chờ 20 ms để ổn định nguồn và khởi cảm biến, chờ 150 ms cho quá trình đo cảm biến, chờ 5 ms khi chuyển chế độ module LoRa E32 và chờ thêm 120 ms sau khi truyền trước khi đưa module về trạng thái tiết kiệm năng lượng. Riêng thời gian chờ AUX của LoRa E32 được cấu hình với ngưỡng tối đa 2000 ms, dùng để phát hiện trường hợp module không hoàn tất truyền trong thời gian cho phép.

Biểu đồ này không thể hiện toàn bộ chu kỳ chờ của node, mà chỉ phóng to giai đoạn node hoạt động tích cực khi có dữ liệu cần truyền. Qua đó có thể thấy firmware đã tổ chức quá trình vận hành theo từng giai đoạn rõ ràng: cảm biến chỉ hoạt động trong giai đoạn đo, ASCON chỉ được thực hiện sau khi payload đã tạo xong, còn LoRa E32 chỉ được đưa vào trạng thái truyền trong khoảng thời gian gửi frame. Cách tổ chức này phù hợp với yêu cầu tiết kiệm năng lượng của trạm đo ngoài trời.

3.5. Thiết kế cấu trúc payload và frame dữ liệu LoRa

Tuyến truyền giữa STM32 Sensor Node và ESP32-S3 Gateway sử dụng LoRa, do đó dữ liệu truyền cần có kích thước nhỏ, cấu trúc cố định và dễ kiểm tra ở phía nhận. Nếu truyền dữ liệu dưới dạng chuỗi ký tự hoặc JSON trực tiếp qua LoRa, kích thước gói tin sẽ tăng, thời gian phát sóng kéo dài và mức tiêu thụ năng lượng của node cũng tăng theo. Vì vậy, hệ thống sử dụng payload nhị phân có độ dài cố định để biểu diễn dữ liệu cảm biến trước khi đưa vào frame truyền.

Payload cảm biến được thiết kế với độ dài 10 byte. Các giá trị đo dạng số thực không được truyền trực tiếp theo kiểu floating-point, mà được chuyển sang dạng số nguyên theo hệ số tỉ lệ. Cách biểu diễn này giúp giảm kích thước dữ liệu, hạn chế sai khác khi xử lý giữa

STM32 và ESP32-S3, đồng thời đơn giản hóa quá trình đóng gói, mã hóa, truyền nhận và giải mã. Khi Gateway nhận được dữ liệu, các giá trị vật lý được khôi phục bằng công thức ngược tương ứng với từng trường dữ liệu.

Đối với nhiệt độ môi trường từ SHT30, giá trị đo được nhân với 100 và lưu bằng số nguyên có dấu 16 bit. Công thức mã hóa được xác định như sau:

$$T_{enc} = T_C \times 100 \quad (8)$$

Trong đó, T_C là nhiệt độ theo đơn vị $^{\circ}\text{C}$ và T_{enc} là giá trị nhiệt độ sau khi chuyển đổi để đưa vào payload. Khi ESP32-S3 Gateway nhận dữ liệu, nhiệt độ được khôi phục bằng công thức:

$$T_C = T_{enc} / 100 \quad (9)$$

Ví dụ, nếu nhiệt độ môi trường đo được là $30,96^{\circ}\text{C}$, giá trị được đưa vào payload là:

$$T_{enc} = 30,96 \times 100 = 3096 (^{\circ}\text{C}) \quad (10)$$

Cách biểu diễn này cho phép giữ độ phân giải $0,01^{\circ}\text{C}$ nhưng chỉ sử dụng 2 byte trong payload.

Đối với độ ẩm tương đối từ SHT30, giá trị đo được cũng được nhân với 100 và lưu bằng số nguyên không dấu 16 bit. Công thức mã hóa độ ẩm được xác định như sau:

$$H_{enc} = H_{RH} \times 100 \quad (11)$$

Trong đó, H_{RH} là độ ẩm tương đối theo đơn vị $\%$. Khi Gateway nhận dữ liệu, độ ẩm được khôi phục bằng công thức:

$$H_{RH} = H_{enc} / 100 \quad (12)$$

Ví dụ, nếu độ ẩm đo được là $77,82\%$, giá trị đưa vào payload là:

$$H_{enc} = 77,82 \times 100 = 7782 (\%) \quad (13)$$

Đối với áp suất khí quyển từ BMP388, hệ thống không lưu trực tiếp giá trị áp suất tuyệt đối sau khi nhân hệ số tỉ lệ, vì giá trị này có thể vượt quá phạm vi biểu diễn của số nguyên không dấu 16 bit. Thay vào đó, firmware lưu giá trị lệch so với một mốc áp suất quy ước. Trong thiết kế này, mốc áp suất được chọn là:

$$P_0 = 900 (\text{hPa}) \quad (14)$$

Mốc P_0 là tham số thiết kế dùng để nén dữ liệu áp suất trong payload, không phải giới hạn bắt buộc của cảm biến. Việc chọn mốc này dựa trên điều kiện triển khai dự kiến của hệ thống ở khu vực có độ cao thấp, nơi áp suất khí quyển thường nằm quanh vùng xấp xỉ 1000 hPa. Công thức mã hóa áp suất tại STM32 Sensor Node được xác định như sau:

$$P_{enc} = (P_{hPa} - P_0) \times 100 \quad (15)$$

Trong đó, P_{hPa} là áp suất đo được theo đơn vị hPa, P_0 là mốc áp suất quy ước và P_{enc} là giá trị áp suất sau khi chuyển đổi để đưa vào payload. Khi Gateway nhận dữ liệu, áp suất được khôi phục bằng công thức ngược:

$$P_{hPa} = (P_{enc} / 100) + P_0 \quad (16)$$

Ví dụ, nếu áp suất đo được là 1006,44 hPa, giá trị được đưa vào payload là:

$$P_{enc} = (1006,44 - 900) \times 100 = 10644 \text{ (hPa)} \quad (17)$$

Khi ESP32-S3 Gateway nhận được giá trị $P_{enc} = 10644$, áp suất được khôi phục lại như sau:

$$P_{hPa} = (10644 / 100) + 900 = 1006,44 \text{ (hPa)} \quad (18)$$

Nếu không sử dụng mốc quy ước, giá trị $1006,44 \times 100 = 100644$ sẽ vượt quá phạm vi của số nguyên không dấu 16 bit. Sau khi trừ mốc 900 hPa, giá trị cần lưu chỉ còn 10644, nằm trong phạm vi biểu diễn của 2 byte. Tuy nhiên, cách biểu diễn này chỉ phù hợp khi áp suất đo được lớn hơn hoặc bằng mốc P_0 . Nếu hệ thống được triển khai ở khu vực có độ cao lớn, nơi áp suất có thể thấp hơn 900 hPa, mốc P_0 cần được điều chỉnh thấp hơn hoặc cần thay đổi cách biểu diễn dữ liệu áp suất.

Ngoài nhiệt độ, độ ẩm và áp suất, payload còn chứa nhiệt độ tại khu vực cảm biến từ BMP388, điện áp pin đã được nén và trường trạng thái health_flag. Nhiệt độ từ BMP388 được biểu diễn tương tự nhiệt độ môi trường, tức là nhân với 100 và lưu bằng số nguyên có dấu 16 bit. Điện áp pin được nén về 1 byte dựa trên dải điện áp làm việc của pin Lithium 1S. Trong thiết kế này, điện áp pin được ánh xạ trong khoảng từ 3,0 V đến 4,2 V theo công thức:

$$V_{enc} = [(V_{bat} - 3,0) / (4,2 - 3,0)] \times 255 \quad (19)$$

Trong đó, V_{bat} là điện áp pin đo được và V_{enc} là giá trị điện áp sau khi nén về 1 byte. Khi Gateway cần khôi phục điện áp pin, công thức ngược được sử dụng như sau:

$$V_{bat} = (V_{enc} / 255) \times (4,2 - 3,0) + 3,0 \quad (20)$$

Các giá trị nhỏ hơn 3,0 V hoặc lớn hơn 4,2 V cần được giới hạn trong phạm vi 0 đến 255 để tránh tràn dữ liệu khi đóng gói. Cách nén này giúp giảm kích thước payload trong khi vẫn đủ để đánh giá trạng thái tương đối của pin trong quá trình vận hành.

Trường `health_flag` có độ dài 1 byte, được sử dụng để đánh dấu các trạng thái bất thường trong quá trình đọc cảm biến, giám sát nguồn hoặc vận hành node. Khi không phát hiện lỗi, trường này có thể được đặt bằng 0x00. Khi xuất hiện lỗi cảm biến, lỗi giao tiếp hoặc trạng thái nguồn bất thường, firmware sẽ cập nhật các bit tương ứng trong `health_flag`. Nhờ có trường này, Gateway không chỉ nhận được dữ liệu đo mà còn có thêm thông tin để đánh giá độ tin cậy của dữ liệu tại thời điểm truyền.

Hình 3.6 trình bày cấu trúc payload cảm biến 10 byte. Các trường trong payload được sắp xếp theo thứ tự cố định gồm nhiệt độ môi trường, độ ẩm, áp suất, nhiệt độ BMP388, điện áp pin và `health_flag`. Việc quy định cố định thứ tự trường dữ liệu giúp STM32 đóng gói và ESP32-S3 Gateway giải mã thống nhất.

Temp (env)	Humidity	Pressure	Temp (board)	Vbat	health flag
2 bytes	2 bytes	2 bytes	2 bytes	1 byte	1 byte

Hình 3.6. Cấu trúc payload cảm biến 10 byte

Để minh họa quá trình tạo payload, giả sử tại một chu kỳ đo, STM32 Sensor Node đọc được các giá trị gồm nhiệt độ môi trường 30,96 °C, độ ẩm 77,82%, áp suất 1006,44 hPa, nhiệt độ BMP388 31,15 °C, điện áp pin 3,85 V và không phát hiện lỗi cảm biến. Khi đó, nhiệt độ môi trường được chuyển thành 3096, độ ẩm được chuyển thành 7782, áp suất được chuyển thành 10644, nhiệt độ BMP388 được chuyển thành 3115, điện áp pin được nén về một giá trị trong khoảng 0 đến 255, còn `health_flag` được đặt bằng 0x00. Các giá trị này được đóng gói thành payload nhị phân 10 byte trước khi đưa vào bước mã hóa.

Payload 10 byte chỉ là phần dữ liệu cảm biến trước khi truyền. Để gửi dữ liệu qua LoRa, payload được đưa vào một frame truyền có cấu trúc đầy đủ hơn. Frame LoRa của hệ thống có độ dài 18 byte, gồm ba thành phần chính: `frame_counter`, `ciphertext` và `authentication tag`. Trong đó, `frame_counter` có độ dài 4 byte, dùng để đánh số thứ tự frame và hỗ trợ quá trình tạo nonce cho cơ chế mã hóa xác thực. Phần `ciphertext` có độ dài 10 byte, tương ứng với payload cảm biến sau khi được mã hóa. Phần `authentication tag` có độ dài 4 byte, dùng để kiểm tra tính toàn vẹn và tính xác thực của dữ liệu ở phía Gateway.

Hình 3.7 trình bày cấu trúc frame LoRa 18 byte. Frame được thiết kế theo thứ tự gồm 4 byte frame_counter, 10 byte ciphertext và 4 byte authentication tag. Khi Gateway nhận dữ liệu từ module LoRa, bước kiểm tra đầu tiên là xác định frame có đủ 18 byte hay không. Những frame thiếu byte, thừa byte hoặc sai cấu trúc sẽ bị loại bỏ trước khi đưa vào bước xác thực và giải mã. Cách kiểm tra này giúp giảm khả năng dữ liệu lỗi được xử lý tiếp ở các tầng phía sau.

frame_counter	ciphertext	authentication tag
4 bytes	10 bytes	4 bytes

Hình 3.7. Cấu trúc frame LoRa 18 byte

Frame_counter là thành phần quan trọng trong thiết kế frame. Mỗi lần STM32 Sensor Node tạo frame mới, giá trị frame_counter được tăng lên và gắn vào frame trước khi truyền. Giá trị này giúp Gateway nhận biết thứ tự dữ liệu, đồng thời hỗ trợ phát hiện các trường hợp bất thường như mất frame, nhận lại frame cũ hoặc dữ liệu bị lặp. Ngoài ra, frame_counter còn được sử dụng trong quá trình tạo nonce cho cơ chế bảo mật, giúp mỗi lần mã hóa có đầu vào khác nhau ngay cả khi dữ liệu cảm biến có thể gần giống nhau giữa các chu kỳ đo.

Ví dụ, nếu frame_counter tại thời điểm gửi là 282, giá trị này được đưa vào phần đầu của frame dưới dạng số 32 bit. Payload 10 byte sau khi được mã hóa bằng ASCON-128a tạo thành ciphertext có cùng độ dài 10 byte. Đồng thời, thuật toán tạo ra authentication tag 4 byte để Gateway kiểm tra tính hợp lệ của frame. Do đó, frame truyền qua LoRa không chứa trực tiếp dữ liệu cảm biến dạng rõ, mà chứa số thứ tự frame, dữ liệu đã mã hóa và tag xác thực.

Việc tách payload và frame giúp thiết kế truyền thông rõ ràng hơn. Payload tập trung vào biểu diễn dữ liệu cảm biến ở dạng ngắn gọn, còn frame bổ sung các trường phục vụ truyền nhận và kiểm tra tính hợp lệ. Với cấu trúc 18 byte, frame LoRa đủ nhỏ để phù hợp với truyền thông dung lượng thấp, đồng thời vẫn chứa các thông tin cần thiết cho việc đánh số frame, bảo vệ dữ liệu và giải mã ở Gateway. Thiết kế này tạo nền tảng cho cơ chế bảo mật hai tuyến được trình bày ở mục tiếp theo.

3.6. Thiết kế cơ chế bảo mật hai tuyến bằng ASCON-128a

Dữ liệu trong hệ thống quan trắc thời tiết cục bộ được truyền qua hai đoạn chính. Đoạn thứ nhất là từ STM32 Sensor Node đến ESP32-S3 Gateway thông qua LoRa. Đoạn thứ hai là từ ESP32-S3 Gateway đến Supabase Cloud thông qua Internet. Hai đoạn truyền này có đặc điểm khác nhau về tài nguyên xử lý, kích thước dữ liệu, môi trường truyền và yêu cầu kiểm tra dữ liệu. Vì vậy, cơ chế bảo mật được thiết kế theo hướng hai tuyến, trong đó mỗi tuyến sử dụng một khóa ASCON-128a riêng.

ASCON-128a được lựa chọn vì đây là thuật toán mã hóa có xác thực hạng nhẹ, phù hợp với môi trường IoT hạn chế tài nguyên. Thuật toán hoạt động theo mô hình AEAD, cho phép đồng thời bảo vệ tính bí mật, tính toàn vẹn và tính xác thực của dữ liệu. Theo nguyên lý tổng quát, quá trình mã hóa nhận vào khóa bí mật, nonce, dữ liệu phụ trợ nếu có và plaintext. Sau quá trình mã hóa, kết quả thu được gồm ciphertext và authentication tag. Có thể ký hiệu tổng quát quá trình này như sau:

$$(C, \text{Tag}) = \text{Enc}(K, N, AD, P) \quad (21)$$

Trong đó, K là khóa bí mật, N là nonce, AD là dữ liệu phụ trợ, P là plaintext, C là ciphertext và Tag là mã xác thực. Ở phía nhận, quá trình giải mã chỉ khôi phục plaintext khi tag xác thực hợp lệ. Nếu tag không hợp lệ, dữ liệu được xem là không đáng tin cậy và bị loại bỏ.

Tuyến thứ nhất (STM32 Sensor Node - ESP32-S3 Gateway): STM32 Sensor Node sử dụng khóa ASCON thứ nhất để bảo vệ payload cảm biến trước khi truyền qua LoRa. Payload 10 byte sau khi được tạo từ các giá trị đo không được gửi trực tiếp ở dạng rõ, mà được mã hóa bằng ASCON-128a để tạo ciphertext. Đồng thời, thuật toán tạo authentication tag để Gateway có thể kiểm tra tính hợp lệ của frame nhận được. Do tuyến LoRa có băng thông thấp và node hoạt động bằng pin, authentication tag trong frame được rút gọn còn 4 byte nhằm giảm kích thước dữ liệu truyền. Frame truyền qua LoRa vì vậy gồm frame_counter, ciphertext và tag xác thực.

Nonce của tuyến LoRa được tạo dựa trên frame_counter kết hợp với thông tin định danh cố định của node theo quy ước firmware. Cách tạo nonce này giúp mỗi lần mã hóa có đầu vào khác nhau, kể cả khi dữ liệu cảm biến giữa hai chu kỳ đo có thể gần giống nhau. frame_counter vì vậy không chỉ dùng để đánh số thứ tự frame, mà còn tham gia vào cơ chế bảo mật của tuyến truyền. Để bảo đảm nonce không bị lặp lại trong phạm vi cùng một khóa, frame_counter cần được tăng sau mỗi lần tạo frame và cần được lưu hoặc khôi phục phù hợp sau khi node khởi động lại.

Ở phía ESP32-S3 Gateway, quá trình xử lý bắt đầu sau khi Gateway nhận đủ frame LoRa 18 byte. Gateway tách frame thành frame_counter, ciphertext và authentication tag. Từ frame_counter, Gateway tạo lại nonce theo cùng quy ước với STM32 Sensor Node. Sau đó, Gateway sử dụng khóa ASCON thứ nhất để kiểm tra tag và giải mã ciphertext. Nếu tag không hợp lệ, frame bị loại bỏ và dữ liệu không được đưa vào bước xử lý tiếp theo. Nếu tag hợp lệ, payload 10 byte được khôi phục và chuyển sang bước giải mã các trường dữ liệu vật lý.

Tuyến thứ hai (ESP32-S3 Gateway - Supabase Cloud): ESP32-S3 Gateway sử dụng khóa ASCON thứ hai để bảo vệ dữ liệu trước khi gửi lên Supabase Cloud. Sau khi Gateway đã giải mã thành công frame LoRa, các giá trị cảm biến được khôi phục về dạng vật lý và có thể được bổ sung thêm thông tin xử lý cục bộ như kết quả dự báo, mã thiết bị, thời gian, trạng thái gateway hoặc trạng thái tuyến LoRa. Trước khi gửi lên Supabase Edge Function, dữ liệu được mã hóa lại bằng khóa thứ hai. Ở tuyến này, do dữ liệu được truyền qua Internet và không còn bị giới hạn nghiêm ngặt như frame LoRa, tag xác thực có thể sử dụng độ dài lớn hơn (16 byte) để tăng khả năng kiểm tra tính toàn vẹn dữ liệu ở phía Cloud.

Supabase Edge Function đóng vai trò điểm tiếp nhận dữ liệu trên Cloud. Function kiểm tra token thiết bị, kiểm tra cấu trúc request, xác thực và giải mã dữ liệu bằng khóa ASCON thứ hai. Chỉ những request có token hợp lệ và dữ liệu giải mã thành công mới được ghi vào PostgreSQL Database. Cách tổ chức này giúp dữ liệu không được ghi trực tiếp vào cơ sở dữ liệu nếu chưa qua bước kiểm tra ở tầng Cloud.

Việc chia thành hai khóa có ý nghĩa quan trọng trong quản lý rủi ro. Nếu chỉ sử dụng một khóa chung cho toàn bộ hệ thống, cùng một khóa sẽ xuất hiện ở STM32 Sensor Node, ESP32-S3 Gateway và phía Cloud. Khi đó, nếu một thành phần bị lộ khóa, toàn bộ chuỗi truyền có thể bị ảnh hưởng. Với thiết kế hai tuyến, STM32 Sensor Node chỉ cần lưu khóa của tuyến LoRa, còn khóa gửi Cloud được quản lý tại ESP32-S3 Gateway và Supabase Edge Function. Nhờ vậy, phạm vi ảnh hưởng của từng khóa được giới hạn theo từng đoạn truyền.

Bên cạnh cơ sở về mặt bảo mật, việc lựa chọn ASCON-128a còn được hỗ trợ bởi các kết quả thực nghiệm từ công trình liên quan. Khi triển khai ASCON-128a trong mô hình IoT gồm ESP32 ở phía node và Raspberry Pi 4 ở phía gateway. Kết quả thực nghiệm cho thấy ESP32 đạt thông lượng mã hóa khoảng 2,42 MB/s, trong khi Raspberry Pi 4 đạt khoảng 124 MB/s khi có áp dụng các kỹ thuật tối ưu. Kết quả này cho thấy ASCON-128a có khả năng thực thi phù hợp trên các nền tảng IoT và đáp ứng tốt các hệ thống truyền dữ liệu dung lượng nhỏ.[23]

Tuy nhiên, các số liệu hiệu năng từ công trình tham khảo không được xem là kết quả đo trực tiếp của hệ thống trong đề tài, vì nền tảng phần cứng và môi trường thực thi không hoàn toàn tương đồng. Hệ thống được triển khai với STM32F103C8T6 ở phía Sensor Node, ESP32-S3 ở phía Gateway và Supabase Edge Function ở phía Cloud, trong khi công trình tham khảo sử dụng mô hình ESP32 và Raspberry Pi 4. Do đó, các kết quả này chỉ được dùng để cử trình tham khảo sử dụng mô hình ESP32 và Raspberry Pi 4. Do đó, các kết quả này chỉ được dùng để củng cố tính khả thi của việc lựa chọn ASCON-128a cho hệ thống IoT, không thay thế cho quá trình đánh giá thực nghiệm trên phần cứng thực tế.

Đối với STM32 Sensor Node, việc đánh giá ASCON-128a cần đặt trong toàn bộ chu kỳ hoạt động của node thay vì chỉ xét riêng thời gian mã hóa. Payload của hệ thống có độ dài 10 byte, nên chi phí mã hóa là một phần trong chuỗi thao tác gồm đánh thức vi điều khiển, đọc cảm biến, đóng gói dữ liệu, mã hóa, bật module LoRa, phát frame, chờ tín hiệu AUX và quay lại chế độ Standby. Vì node vận hành bằng pin kết hợp năng lượng mặt trời, tiêu chí quan trọng không chỉ là tốc độ mã hóa mà còn là ảnh hưởng của quá trình mã hóa đến thời gian thức và năng lượng tiêu thụ trong mỗi chu kỳ truyền.

Đối với ESP32-S3 Gateway, ASCON-128a được sử dụng ở cả hai chiều xử lý: giải mã frame nhận từ tuyến LoRa và mã hóa lại dữ liệu trước khi gửi lên Cloud. Hiệu năng thực tế trên Gateway phụ thuộc vào cấu hình firmware, tần số CPU, mức tối ưu trình biên dịch, cách tổ chức task FreeRTOS và cách cấp phát bộ nhớ. Vì vậy, nếu cần đánh giá định lượng, các chỉ số phù hợp gồm thời gian xử lý một frame LoRa, thời gian giải mã payload, thời gian mã hóa dữ liệu gửi Cloud và độ ổn định khi Gateway vận hành liên tục.

Đối với Supabase Edge Function, hiệu năng không nên được đánh giá theo thông lượng mã hóa đơn thuần như trên vi điều khiển. Ở phía Cloud, thời gian xử lý một request còn bao gồm độ trễ HTTPS, kiểm tra token thiết bị, giải mã dữ liệu, phân tích nội dung request, ghi dữ liệu vào PostgreSQL và trả phản hồi cho Gateway. Do đó, các chỉ số phù hợp hơn là độ trễ request trung bình, tỉ lệ request thành công, tỉ lệ lỗi xác thực và thời gian ghi dữ liệu vào cơ sở dữ liệu.

Như vậy, kết quả thực nghiệm từ công trình tham khảo có vai trò làm cơ sở lựa chọn thuật toán, còn hiệu năng thực tế của hệ thống cần được đánh giá theo đặc thù của từng khối triển khai. Cách tiếp cận này giúp tránh việc suy diễn trực tiếp số liệu từ nền tảng khác, đồng thời vẫn cho thấy ASCON-128a là lựa chọn phù hợp cho mô hình bảo mật hai tuyến của hệ thống.

3.7. Thiết kế ESP32-S3 Gateway

ESP32-S3 Gateway là khối trung gian giữa tuyến truyền LoRa cục bộ và nền tảng Cloud. Khối này có nhiệm vụ tiếp nhận frame từ STM32 Sensor Node, kiểm tra tính hợp lệ của dữ liệu, giải mã payload, khôi phục các giá trị vật lý, thực hiện xử lý cục bộ và gửi dữ liệu lên Supabase thông qua kết nối Internet. Ngoài luồng dữ liệu cảm biến, Gateway còn đảm nhiệm các chức năng hỗ trợ vận hành như quản lý kết nối WiFi, gửi trạng thái heartbeat, kiểm tra cập nhật OTA và hỗ trợ cấu hình WiFi bằng BLE provisioning.

Về phần cứng, ESP32-S3 Gateway giao tiếp với module LoRa E32 thông qua UART. Module LoRa E32 đóng vai trò nhận dữ liệu không dây từ STM32 Sensor Node và chuyển dữ liệu frame về ESP32-S3. Bên cạnh đường UART, các chân điều khiển như M0, M1 và AUX được sử dụng để cấu hình chế độ hoạt động và theo dõi trạng thái truyền nhận của module. Khi Gateway hoạt động, module LoRa được đưa về chế độ nhận dữ liệu phù hợp để chờ frame từ node gửi về.

Ở mức thiết kế phần mềm, Gateway được tổ chức theo hướng tách các nhóm nhiệm vụ chính thay vì xử lý toàn bộ trong một vòng lặp duy nhất. Các nhóm chức năng chính gồm nhận dữ liệu LoRa, xử lý frame, gửi dữ liệu lên Cloud, gửi heartbeat, kiểm tra OTA và xử lý BLE provisioning. Cách tổ chức này giúp firmware dễ quản lý hơn, đồng thời hạn chế việc một tác vụ có thời gian xử lý dài làm ảnh hưởng đến các chức năng khác của Gateway.

Trong quá trình vận hành, Gateway bắt đầu bằng việc khởi tạo các thành phần cần thiết như UART giao tiếp LoRa, WiFi, NVS, bộ đệm nhận dữ liệu và các tác vụ phần mềm. Sau khi khởi tạo hoàn tất, Gateway chuyển sang trạng thái chờ dữ liệu từ module LoRa E32. Khi có dữ liệu đến, firmware đọc dữ liệu từ UART và kiểm tra xem frame nhận được có đúng độ dài thiết kế hay không. Với hệ thống này, frame LoRa hợp lệ có độ dài 18 byte. Các frame thiếu byte, thừa byte hoặc không đúng cấu trúc sẽ bị loại bỏ trước khi đưa vào bước giải mã.

Sau khi nhận được frame hợp lệ về mặt độ dài, Gateway tách frame thành các trường gồm frame_counter, ciphertext và authentication tag. Từ frame_counter, firmware tạo lại nonce theo cùng quy ước với STM32 Sensor Node. Sau đó, Gateway sử dụng khóa ASCON của tuyến LoRa để kiểm tra tag và giải mã ciphertext. Nếu quá trình xác thực thất bại, frame được xem là không đáng tin cậy và bị loại bỏ. Nếu xác thực thành công, payload cảm biến 10 byte được khôi phục để chuyển sang bước giải mã dữ liệu vật lý.

Bước khôi phục dữ liệu vật lý được thực hiện dựa trên định dạng payload đã thiết kế ở mục 3.5. Các trường số nguyên trong payload được chuyển đổi ngược về giá trị đo tương ứng

nhiệt độ, độ ẩm, áp suất khí quyển, nhiệt độ tại cảm biến BMP388, điện áp pin và health_flag. Việc xử lý này được thực hiện tại Gateway thay vì trên Cloud nhằm giảm tải cho tầng lưu trữ và bảo đảm dữ liệu gửi lên Cloud đã ở dạng có ý nghĩa vật lý.

Sau khi khôi phục dữ liệu cảm biến, Gateway thực hiện xử lý cục bộ trước khi gửi dữ liệu lên Cloud. Trong đề tài, xử lý cục bộ bao gồm việc bổ sung thông tin thiết bị, trạng thái gateway, thời gian nhận dữ liệu và kết quả dự báo nhiệt độ ngắn hạn bằng mô hình Multiple Linear Regression. Mô hình này được sử dụng để khảo sát xu hướng nhiệt độ cục bộ dựa trên các biến đầu vào thu được từ hệ thống quan trắc. Kết quả dự báo không thay thế dữ liệu đo thực tế, mà đóng vai trò thông tin tham khảo bổ sung trên Web Dashboard.

Dữ liệu sau khi xử lý được Gateway đóng gói thành payload gửi Cloud. Trước khi truyền lên Supabase, payload này được mã hóa lại bằng khóa ASCON thứ hai. Việc mã hóa lại tại Gateway giúp tách biệt tuyến bảo mật LoRa với tuyến gửi dữ liệu lên Cloud. Sau đó, Gateway gửi request HTTPS đến Supabase Edge Function kèm dữ liệu đã mã hóa và các thông tin cần thiết để Cloud kiểm tra, giải mã và lưu trữ.

Bên cạnh luồng dữ liệu cảm biến, Gateway còn gửi heartbeat định kỳ lên Cloud. Heartbeat được sử dụng để theo dõi trạng thái hoạt động của Gateway, bao gồm các thông tin như thời gian hoạt động, trạng thái WiFi, bộ nhớ còn lại, phiên bản firmware, trạng thái nhận LoRa gần nhất và các thông tin hỗ trợ chẩn đoán. Nhờ có heartbeat, Web Dashboard có thể hiển thị trạng thái của Gateway ngay cả khi trong một khoảng thời gian chưa có frame cảm biến mới từ STM32 Sensor Node.

Gateway cũng cần lưu một số thông tin cấu hình trong NVS. Các thông tin này có thể gồm cấu hình WiFi, trạng thái firmware, mã định danh thiết bị, thông tin phiên bản OTA đã áp dụng và các tham số phục vụ vận hành. Việc sử dụng NVS giúp Gateway giữ lại cấu hình sau khi khởi động lại, hạn chế việc phải cấu hình lại từ đầu trong mỗi lần cấp nguồn.

Đối với kết nối mạng, Gateway sử dụng WiFi để gửi dữ liệu lên Supabase. Trong điều kiện triển khai thực tế, thông tin WiFi có thể thay đổi theo vị trí lắp đặt. Vì vậy, BLE provisioning được tích hợp để hỗ trợ cấu hình WiFi mà không cần nạp lại firmware. Khi cần cấu hình mạng, người dùng có thể đưa Gateway vào chế độ provisioning, truyền thông tin WiFi qua BLE và lưu cấu hình vào NVS. Sau khi cấu hình thành công, Gateway sử dụng thông tin này để kết nối mạng và tiếp tục luồng gửi dữ liệu lên Cloud.

Chức năng OTA firmware update được tích hợp để hỗ trợ cập nhật phần mềm Gateway từ xa. Gateway định kỳ kiểm tra thông tin firmware trên Cloud, so sánh với trạng thái đã lưu trong NVS và chỉ thực hiện cập nhật khi phát hiện bản firmware mới hợp lệ. Việc kiểm tra

thông tin như số thứ tự OTA và mã SHA-256 giúp hạn chế tình trạng cập nhật lặp lại cùng một firmware hoặc cập nhật sai file. Chi tiết luồng OTA được trình bày ở mục thiết kế giao diện và chức năng hỗ trợ vận hành.

Trong thiết kế này, lưu đồ hoạt động chính của Gateway không chia theo từng lõi xử lý của ESP32-S3, mà được mô tả theo luồng chức năng. Nguyên nhân là mục tiêu của chương thiết kế là làm rõ dữ liệu đi qua các bước nhận frame, kiểm tra, giải mã, xử lý và gửi Cloud. Việc chia tác vụ theo core chỉ nên được trình bày khi firmware có cấu hình cố định bằng các tác vụ được pin vào từng core. Do đó, báo cáo mô tả Gateway theo hướng tổ chức task và nhóm chức năng, giúp phù hợp hơn với mức thiết kế hệ thống.

Với cách tổ chức trên, ESP32-S3 Gateway đóng vai trò cầu nối giữa thiết bị đo ngoài trời và nền tảng lưu trữ trực tuyến. Gateway không chỉ chuyển tiếp dữ liệu, mà còn thực hiện các bước kiểm tra, giải mã, khôi phục giá trị vật lý, xử lý cục bộ và quản lý trạng thái vận hành. Điều này giúp giảm độ phức tạp cho STM32 Sensor Node, đồng thời bảo đảm dữ liệu gửi lên Cloud đã được kiểm tra và xử lý ở tầng trung gian.

CHƯƠNG 4. TRIỂN KHAI HỆ THỐNG

4.1. Môi trường và phạm vi triển khai

Sau khi hoàn thành thiết kế hệ thống ở Chương 3, các khối chức năng được hiện thực thành phần cứng, firmware và nền tảng phần mềm có thể vận hành thực tế. Chương này trình bày quá trình triển khai hệ thống theo từng cụm chính, gồm trạm đo STM32 Sensor Node, bộ trung tâm ESP32-S3 Gateway, nền tảng Supabase Cloud, Web Dashboard và cơ chế cập nhật firmware từ xa.

Khác với chương thiết kế, nội dung Chương 4 tập trung vào cách các ý tưởng đã được hiện thực. Vì vậy, phần trình bày không lặp lại chi tiết nguyên lý payload, frame LoRa hay cơ chế ASCON-128a, mà chỉ nêu cách các thiết kế này được đưa vào phần cứng, firmware và nền tảng phần mềm. Đối với từng thiết bị, phần cứng và firmware được trình bày cùng nhóm để thể hiện rõ mối liên hệ giữa kết nối mạch và chương trình điều khiển.

Các công cụ chính được sử dụng trong quá trình triển khai gồm KiCad cho thiết kế schematic và PCB, STM32CubeIDE cho firmware STM32F103C8T6, ESP-IDF cho firmware ESP32-S3, Supabase cho Edge Function, PostgreSQL Database và Storage, cùng Web Dashboard cho hiển thị dữ liệu và thao tác OTA. Ngoài ra, ST-Link, UART log, máy hiện sóng và Wireshark được sử dụng trong quá trình nạp chương trình, kiểm tra tín hiệu và theo dõi kết nối mạng.

Nhóm triển khai	Công cụ / nền tảng	Vai trò sử dụng
Thiết kế phần cứng	KiCad	Thiết kế schematic, PCB, kiểm tra kết nối và mô hình 3D
Firmware trạm đo	STM32CubeIDE, HAL, ST-Link	Lập trình, nạp và kiểm tra firmware STM32F103C8T6
Firmware gateway	ESP-IDF, FreeRTOS	Lập trình ESP32-S3, WiFi, BLE, HTTPS, NVS và OTA

Bảo mật dữ liệu	ASCON-128a	Mã hóa và xác thực dữ liệu trên tuyến LoRa và tuyến Gateway–Cloud
Nền tảng Cloud	Supabase Edge Function, PostgreSQL, Storage	Tiếp nhận, xử lý, lưu dữ liệu và lưu firmware OTA
Giao diện giám sát	Web Dashboard, App	Hiển thị dữ liệu, trạng thái hệ thống và hỗ trợ OTA
Công cụ kiểm tra	UART log, máy hiện sóng, Wireshark	Theo dõi firmware, tín hiệu phần cứng và kết nối mạng

Bảng 4.1. Công cụ và nền tảng triển khai hệ thống

4.2. Triển khai trạm đo STM32 Sensor Node

4.2.1. Phần cứng trạm đo

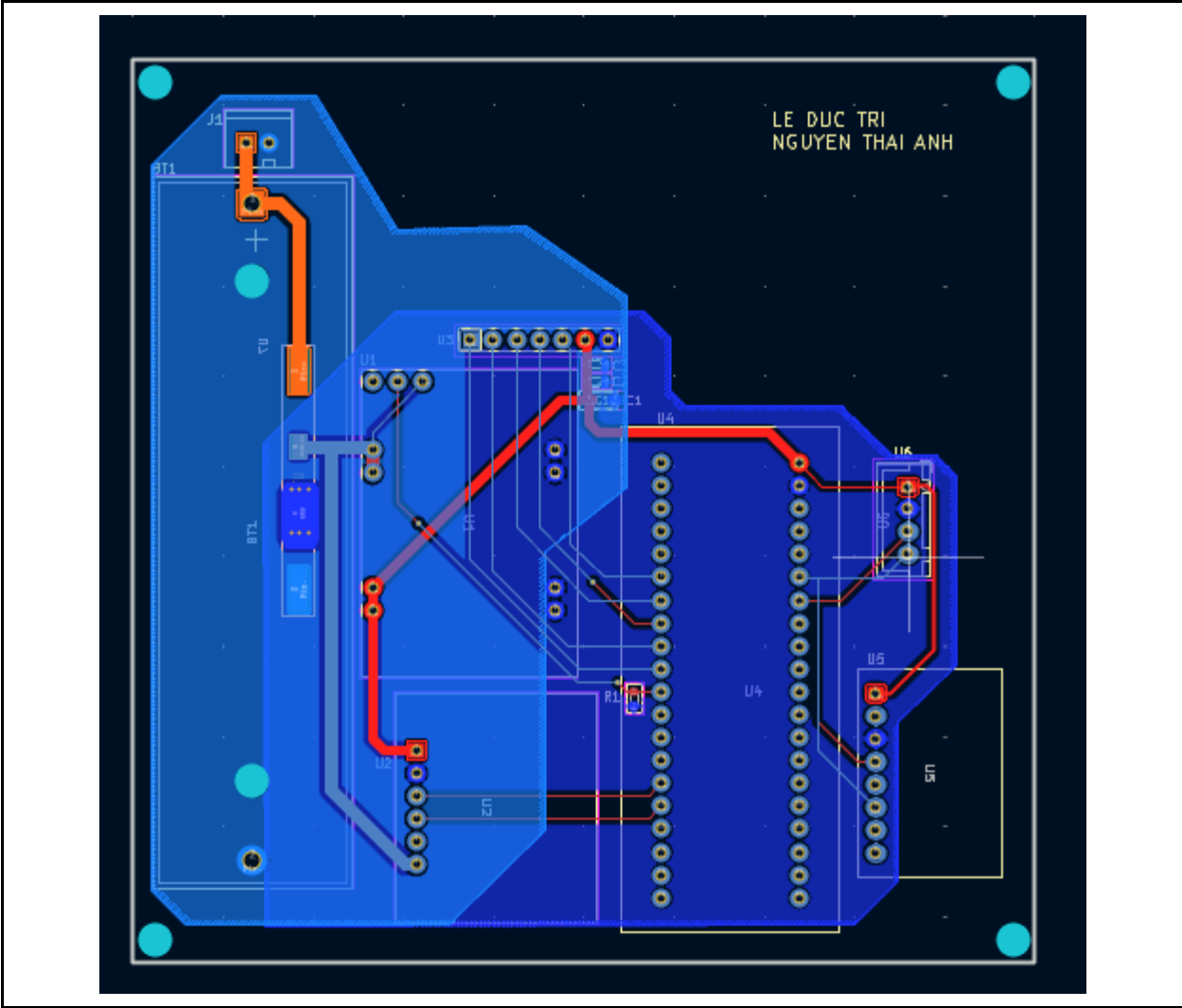
Trạm đo STM32 Sensor Node là thiết bị đặt tại vị trí quan trắc, có nhiệm vụ thu thập dữ liệu môi trường, giám sát trạng thái nguồn và truyền dữ liệu về gateway thông qua LoRa. Phần cứng của trạm đo được triển khai xoay quanh vi điều khiển STM32F103C8T6, cụm cảm biến SHT30, BMP388, INA219, module LoRa E32 và khối nguồn sử dụng pin kết hợp năng lượng mặt trời.

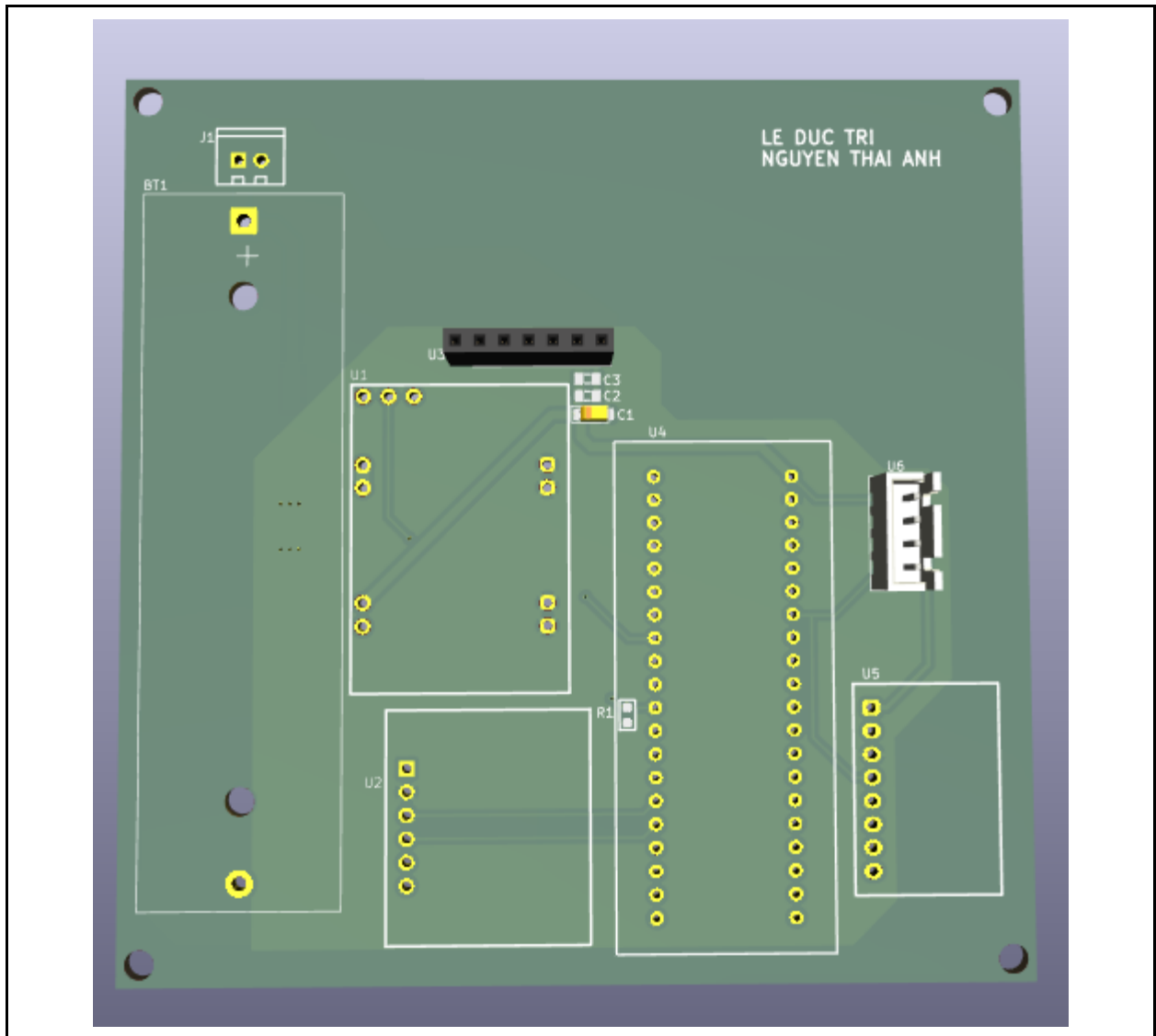
Khối nguồn của trạm đo gồm tấm pin năng lượng mặt trời, module sạc CN3791, pin Lithium EVE 1S, mạch bảo vệ pin 1S và bộ chuyển đổi TPS63020. Năng lượng từ tấm pin được đưa vào module CN3791 để sạc pin lưu trữ. Điện áp từ pin sau đó đi qua mạch bảo vệ và bộ chuyển đổi TPS63020 để tạo đường nguồn ổn định cấp cho vi điều khiển, cảm biến và module LoRa. Chân PS/SYNC của TPS63020 được kết nối về STM32 để firmware có thể điều chỉnh trạng thái nguồn trong thời điểm LoRa phát dữ liệu.

Cụm cảm biến của trạm đo gồm SHT30, BMP388 và INA219. SHT30 được sử dụng để đo nhiệt độ và độ ẩm môi trường. BMP388 được dùng để đo áp suất khí quyển và nhiệt độ tại

Khởi truyền thông sử dụng module LoRa E32. STM32 giao tiếp với module này thông qua UART, đồng thời sử dụng các chân M0, M1 và AUX để điều khiển chế độ hoạt động và theo dõi trạng thái sẵn sàng của module. Khi đến chu kỳ truyền dữ liệu, STM32 đưa LoRa về Normal Mode, gửi frame qua UART, chờ AUX báo hoàn tất rồi đưa module về trạng thái tiết kiệm năng lượng.

Sau khi hoàn thành schematic, PCB của trạm đo được thiết kế trên KiCad. Bố cục PCB được phân vùng theo nhóm chức năng gồm khối nguồn, khối xử lý, cụm cảm biến và khối truyền thông LoRa. Việc phân vùng này giúp quá trình đi dây rõ ràng hơn, đồng thời thuận lợi cho kiểm tra phần cứng sau khi lắp ráp. Mô hình 3D được dùng để quan sát vị trí linh kiện, đầu nối và tương quan cơ khí trước khi đặt mạch vào vỏ bảo vệ.





Hình 4.2. PCB layout và mô hình 3D của trạm đo STM32 Sensor Node

Sau khi lắp ráp, trạm đo được đưa vào vỏ bảo vệ để phù hợp với điều kiện quan trắc ngoài môi trường. Bo mạch chính được đặt trong hộp bảo vệ, trong khi khu vực cảm biến được bố trí với kết cấu che chắn dạng lồng Stevenson nhằm hạn chế ảnh hưởng trực tiếp của nắng và mưa nhưng vẫn cho phép không khí lưu thông. Cách bố trí này giúp dữ liệu đo ổn định hơn khi thiết bị được đặt ngoài trời.





Hình 4.3. Trạm đo STM32 Sensor Node sau khi lắp ráp trong vỏ bảo vệ

4.2.2. Firmware trạm đo

Firmware của trạm đo được triển khai trên STM32F103C8T6 với các ngoại vi chính gồm GPIO, I2C, UART, RTC và Independent Watchdog. Chương trình được tổ chức theo chu kỳ hoạt động gồm đánh thức, kiểm tra chu kỳ gửi, đọc cảm biến, đóng gói payload, mã hóa dữ liệu, truyền frame LoRa và quay lại chế độ Standby. Cách tổ chức này phù hợp với trạm đo sử dụng pin và năng lượng mặt trời.

Ở giai đoạn khởi động, firmware cấu hình clock, GPIO, I2C, UART, RTC và watchdog. Trước khi đọc cảm biến, chương trình thực hiện bước phục hồi bus I2C nhằm hạn chế trường hợp thiết bị ngoại vi giữ đường SDA ở mức thấp sau reset hoặc sau khi nguồn thay

đổi trạng thái. Sau đó, firmware khởi tạo SHT30, BMP388, INA219 và đưa LoRa E32 về trạng thái phù hợp.

Khi đến chu kỳ đo, firmware kích hoạt cảm biến, chờ quá trình đo hoàn tất và đọc dữ liệu. Dữ liệu hợp lệ được chuyển sang dạng số nguyên theo hệ số tỉ lệ để tạo payload 10 byte. Nếu cảm biến lỗi hoặc nguồn bất thường, firmware cập nhật bit tương ứng trong trường `health_flag`. Trước khi mã hóa, `frame_counter` được reserve trong backup register để hạn chế khả năng dùng lại cùng một counter nếu hệ thống reset bất thường.

```
void node_run_cycle(void)
{
    recover_i2c_bus();
    sensors_init();

    sensors_trigger_measurement();
    wait_sensor_ready();

    SensorData_t payload = sensors_collect_and_pack();

    uint32_t counter = frame_counter_reserve();
    LoRaFrame_t frame = ascon_key1_encrypt(payload, counter);

    lora_set_normal_mode();
    wait_lora_aux_ready();

    tps_ps_sync_high();
    lora_send_frame(frame);
    wait_lora_aux_ready();
    tps_ps_sync_low();

    lora_set_sleep_mode();

    refresh_watchdog();
    schedule_next_rtc_alarm();
    enter_standby();
}
```

Payload và frame truyền được định nghĩa với kích thước cố định để bảo đảm STM32 và Gateway xử lý thống nhất. Payload chứa dữ liệu cảm biến ở dạng đã nén, còn frame LoRa chứa `frame_counter`, ciphertext và authentication tag. Việc cố định kích thước giúp giảm sai lệch khi truyền dữ liệu qua LoRa và đơn giản hóa quá trình kiểm tra ở phía Gateway.

```

typedef struct __attribute__((packed)) {

    int16_t env_temp;    // SHT30 temperature * 100

    uint16_t env_hum;    // SHT30 humidity * 100

    uint16_t air_press;  // (pressure_hPa - P0) * 100

    int16_t board_temp;  // BMP388 temperature * 100

    uint8_t batt_volt;   // compressed battery voltage

    uint8_t health_flag; // error bitmask

} SensorData_t;

typedef struct __attribute__((packed)) {

    uint32_t frame_counter;

    uint8_t ciphertext[10];

    uint8_t mac_tag[4];

} LoRaFrame_t;

```

Trong quá trình truyền, firmware phối hợp điều khiển LoRa E32 và TPS63020. Trước khi gửi frame, LoRa được đưa về Normal Mode và PS/SYNC của TPS63020 được đặt ở trạng thái phù hợp để hỗ trợ tải tức thời khi module phát dữ liệu. Sau khi AUX báo quá trình truyền hoàn tất, firmware trả PS/SYNC về trạng thái tiết kiệm năng lượng và đưa LoRa về Sleep/Config Mode. Nhờ đó, chương trình không chỉ truyền dữ liệu mà còn phối hợp với khối nguồn để giảm tiêu thụ năng lượng sau mỗi chu kỳ truyền.

4.3. Triển khai ESP32-S3 Gateway

4.3.1. Phần cứng Gateway

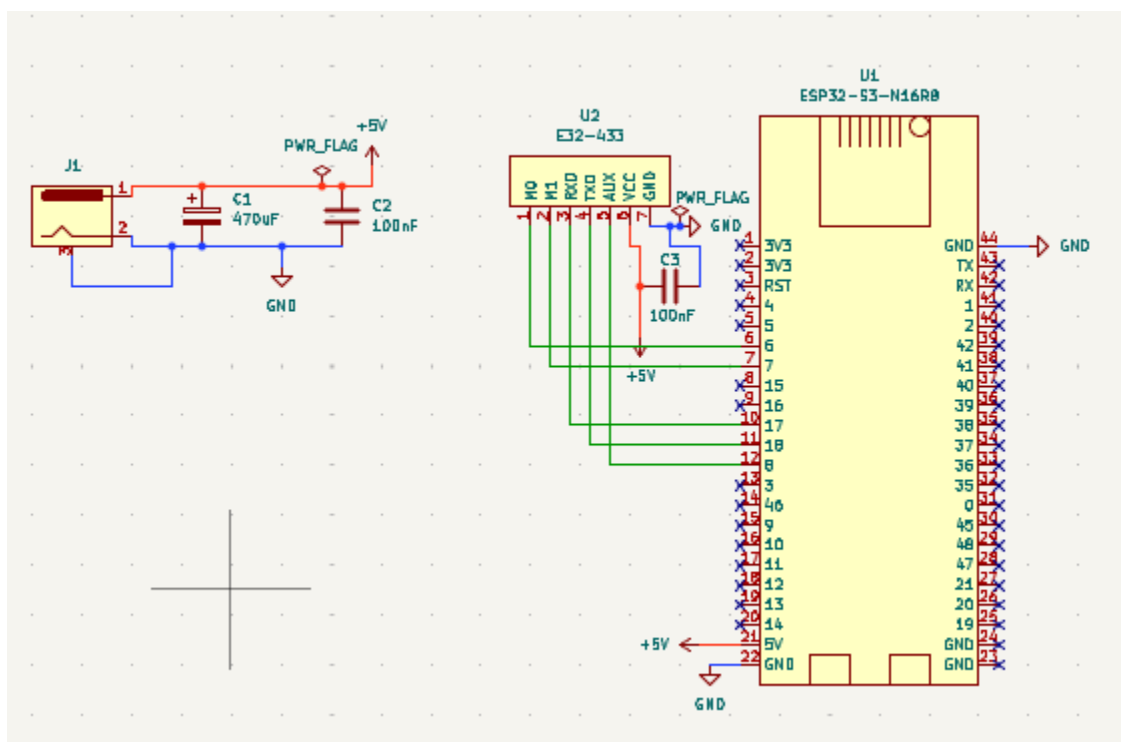
ESP32-S3 Gateway là bộ trung tâm của hệ thống, có nhiệm vụ nhận dữ liệu LoRa từ trạm đo, xử lý frame và gửi dữ liệu lên Supabase thông qua WiFi. Khác với trạm đo, Gateway

không hoạt động theo chu kỳ ngủ/thức mà ưu tiên khả năng nhận dữ liệu ổn định, xử lý liên tục và duy trì kết nối Internet.

Phần cứng Gateway gồm ESP32-S3, module LoRa E32 phía thu, khối cấp nguồn và các linh kiện hỗ trợ ổn định nguồn. ESP32-S3 giao tiếp với module LoRa E32 thông qua UART, đồng thời sử dụng các chân M0, M1 và AUX để cấu hình chế độ hoạt động và theo dõi trạng thái của module. Module LoRa được cấu hình ở chế độ nhận để chờ frame truyền từ STM32 Sensor Node.

Khối nguồn của Gateway được triển khai theo hướng cung cấp điện áp ổn định cho ESP32-S3 và module LoRa. Do Gateway phải duy trì WiFi, HTTPS, BLE provisioning và OTA, nguồn cấp cần ổn định trong các thời điểm truyền dữ liệu mạng hoặc tải firmware. Các tụ lọc được bố trí gần ESP32-S3 và module LoRa để giảm sụt áp tức thời trong quá trình vận hành.

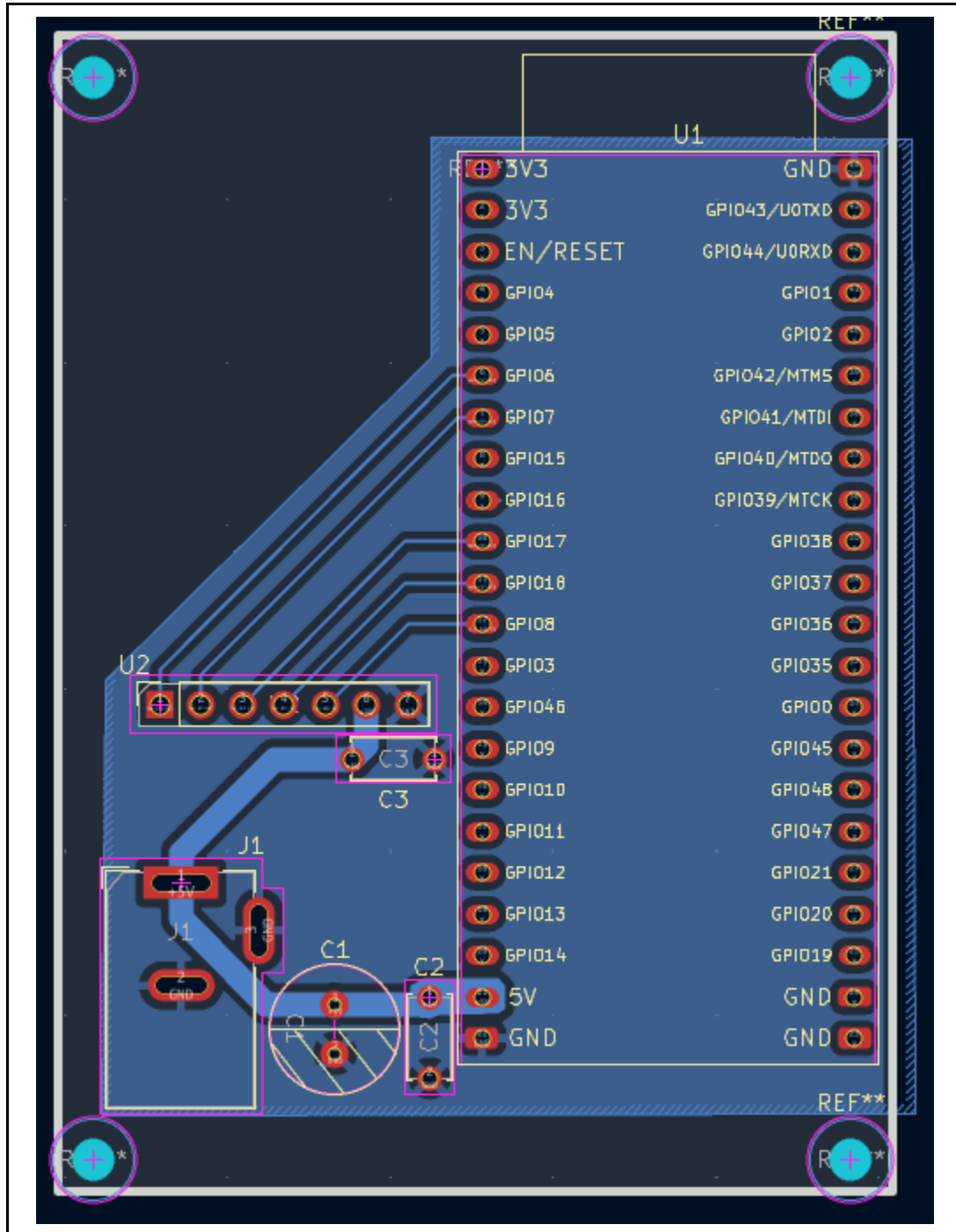
Schematic tổng thể của Gateway được dùng để thể hiện các kết nối chính giữa ESP32-S3, module LoRa E32, khối nguồn và các đường tín hiệu UART, M0, M1, AUX.

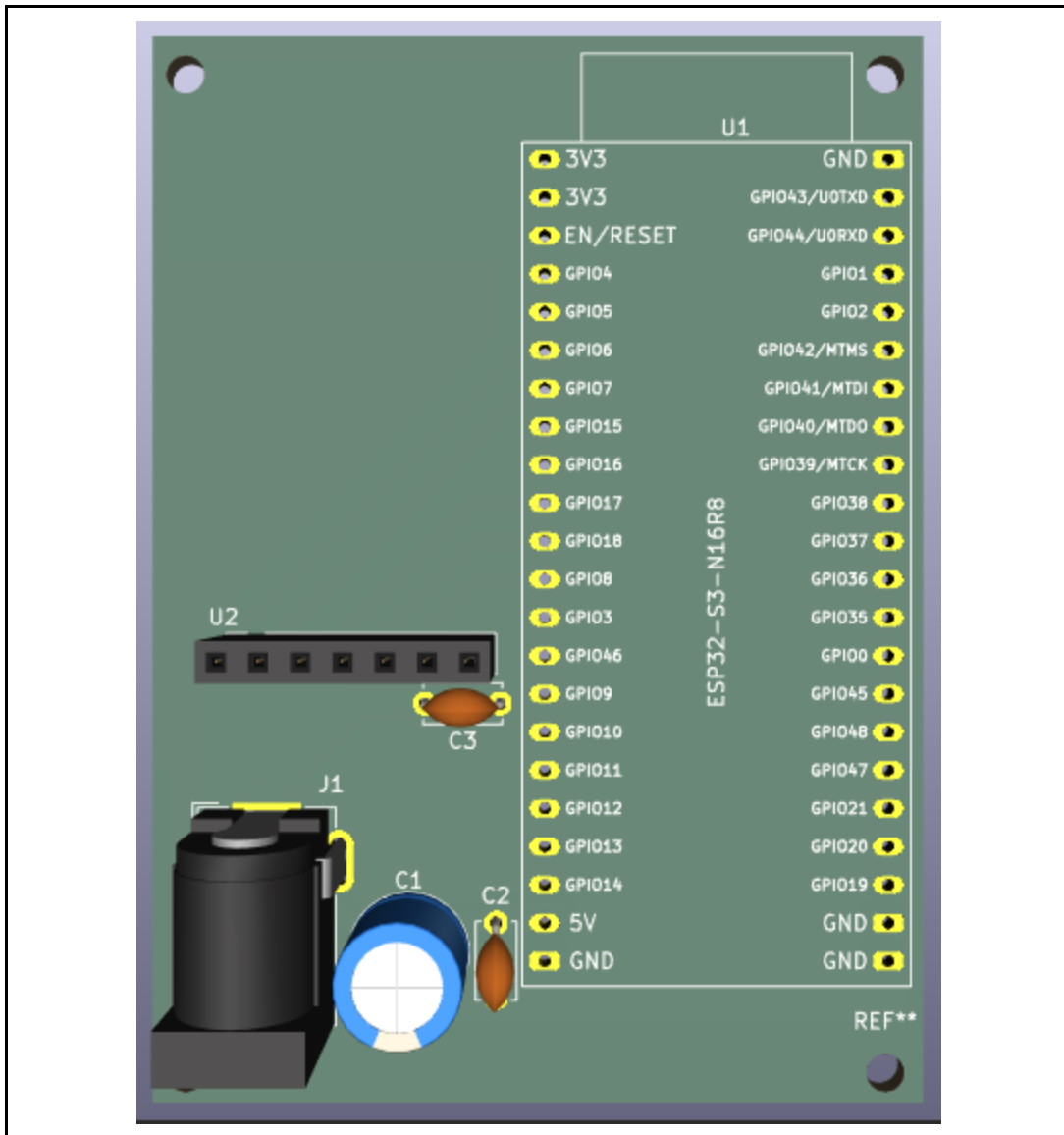


Hình 4.4. Schematic tổng thể ESP32-S3 Gateway

PCB của Gateway được bố trí theo hướng tách rõ khu vực ESP32-S3, module LoRa, đầu nối nguồn và các linh kiện phụ trợ. Việc bố trí các đường UART và đường điều khiển LoRa

ngắn gọn giúp giảm rủi ro lỗi truyền thông. Mô hình 3D được dùng để quan sát bố cục linh kiện trước khi lắp ráp.





Hình 4.5. PCB layout và mô hình 3D của ESP32-S3 Gateway

Sau khi lắp ráp, Gateway được kiểm tra các kết nối chính gồm nguồn cấp, UART LoRa, các chân M0/M1/AUX, khả năng khởi động ESP32-S3 và khả năng kết nối WiFi. Đây là bước chuẩn bị trước khi nạp firmware Gateway và tích hợp với trạm đo.



Hình 4.6. ESP32-S3 Gateway sau khi lắp ráp

4.3.2. Firmware Gateway

Firmware ESP32-S3 Gateway được phát triển bằng ESP-IDF và tổ chức theo các nhóm task FreeRTOS. Các chức năng chính gồm nhận frame LoRa qua UART, xử lý và giải mã frame, đưa dữ liệu vào queue, gửi dữ liệu lên Supabase, gửi heartbeat trạng thái, kiểm tra OTA và hỗ trợ BLE provisioning. Việc tách các chức năng thành nhiều task giúp tránh trường hợp thao tác HTTPS hoặc OTA làm chặn quá trình nhận frame LoRa.

```

void app_main(void)
{
    nvs_init();

    wifi_init_or_ble_provisioning();

    lora_uart_init();

    supabase_queue_init();

    xTaskCreatePinnedToCore(lora_rx_task, "LORA_RX", 8192, NULL, 5, NULL, 0);
    xTaskCreatePinnedToCore(edge_task, "EDGE", 8192, NULL, 4, NULL, 0);
    xTaskCreatePinnedToCore(status_task, "STATUS", 8192, NULL, 3, NULL, 1);
    xTaskCreatePinnedToCore(ota_task, "OTA", 8192, NULL, 4, NULL, 1);
}

```

Tác vụ nhận LoRa đọc dữ liệu từ UART và gom byte vào buffer. Vì frame từ STM32 có kích thước cố định 18 byte, firmware chỉ gọi hàm xử lý khi đã nhận đủ độ dài frame. Nếu khoảng cách giữa các byte vượt quá ngưỡng thời gian cho phép, buffer đang nhận dở được xóa để tránh xử lý frame lỗi.

Sau khi nhận đủ frame, Gateway tách frame_counter, ciphertext và authentication tag. Từ frame_counter, firmware tạo lại nonce theo cùng quy ước với STM32. Sau đó, ASCON key1 được sử dụng để xác thực và giải mã payload. Nếu tag không hợp lệ, frame bị loại bỏ. Nếu hợp lệ, payload 10 byte được khôi phục và chuyển sang bước giải mã các giá trị vật lý.

```

void process_lora_frame(uint8_t frame[18])

```

```

{
    uint32_t counter = parse_frame_counter(frame);
    if (is_replay_frame(counter)) {
        return;
    }
    uint8_t nonce[16];
    build_nonce_from_counter(counter, nonce);
    uint8_t plaintext[10];
    if (!ascon_key1_decrypt(frame, nonce, plaintext)) {
        log_authentication_error();
        return;
    }
    SensorDecodedData_t data = decode_sensor_payload(plaintext);
    data.predicted_temp_2h = mlr_predict(
        data.temperature,
        data.humidity,
        data.pressure
    );
    queue_send_to_cloud(data);
}

```

Payload sau giải mã vẫn ở dạng đã nén theo quy ước của STM32. Gateway chuyển đổi ngược để khôi phục các giá trị vật lý như nhiệt độ, độ ẩm, áp suất, điện áp pin và trạng thái

health_flag. Sau đó, Gateway chạy mô hình Multiple Linear Regression để tạo giá trị dự báo nhiệt độ ngắn hạn. Giá trị dự báo này được gửi lên Cloud như một thông tin tham khảo bổ sung, không thay thế dữ liệu đo thực tế.

Dữ liệu sau xử lý không được gửi HTTPS trực tiếp trong task nhận LoRa. Thay vào đó, dữ liệu được đưa vào queue để task gửi Cloud xử lý. Cách tách này giúp task nhận LoRa không bị chặn bởi thời gian kết nối mạng, TLS hoặc phản hồi từ Supabase.

Trước khi gửi dữ liệu lên Supabase, Gateway mã hóa lại payload bằng ASCON key2. Dữ liệu mã hóa được chuyển sang chuỗi hex và đưa vào JSON gửi đến Supabase Edge Function. Request HTTPS kèm theo các header cần thiết như Authorization, apikey và x-device-token để phía Cloud kiểm tra nguồn gửi.

```
void edge_task(void *arg)
{
    while (1) {
        SensorDecodedData_t data;
        if (xQueueReceive(supabase_queue, &data, portMAX_DELAY)) {
            char payload_key2_hex[HEX_LEN];
            char json_body[JSON_LEN];
            ascon_key2_encrypt_hex(data.plaintext,
                                   data.frame_counter,
                                   payload_key2_hex);
            build_sensor_json(data, payload_key2_hex, json_body);
            https_post_to_edge_function(json_body);
        }
    }
}
```

```
}
```

Bên cạnh dữ liệu cảm biến, Gateway gửi heartbeat định kỳ để cập nhật trạng thái vận hành. Heartbeat gồm các thông tin như phiên bản firmware, SHA-256 firmware đang chạy, RSSI WiFi, địa chỉ IP, uptime và free heap. Nhờ heartbeat, giao diện giám sát có thể phân biệt trường hợp Gateway còn hoạt động nhưng chưa nhận được frame LoRa mới với trường hợp Gateway mất kết nối hoặc dừng hoạt động.

4.3.3. Triển khai mô hình Multiple Linear Regression trên Gateway

Mô hình Multiple Linear Regression được triển khai nhằm khảo sát khả năng dự báo xu hướng nhiệt độ cục bộ trong khoảng thời gian ngắn. Dữ liệu ban đầu không phải là mô hình có sẵn, mà là bộ dữ liệu thời tiết lịch sử được lấy từ Open-Meteo Historical Weather API và đưa vào môi trường huấn luyện trên Kaggle. Từ bộ dữ liệu này, các trường nhiệt độ, độ ẩm và áp suất khí quyển được sử dụng để xây dựng mô hình dự báo nhiệt độ sau 2 giờ.

Dữ liệu đầu vào có dạng chuỗi thời gian theo từng giờ, gồm các trường thời gian, nhiệt độ, độ ẩm tương đối và áp suất bề mặt. Sau khi đọc file CSV, chương trình chuẩn hóa tên cột, chuyển đổi kiểu dữ liệu và loại bỏ các dòng không hợp lệ. Do bài toán cần dự báo nhiệt độ sau 2 giờ, nhãn đầu ra được tạo bằng cách dịch cột nhiệt độ về trước 2 bước thời gian. Với dữ liệu hourly, thao tác này tương ứng với mục tiêu dự báo nhiệt độ tại thời điểm $(t + 2h)$.

Để mô hình phản ánh chu kỳ ngày và chu kỳ mùa trong dữ liệu thời tiết, các đặc trưng thời gian được biến đổi sang dạng sin và cos. Cách biểu diễn này giúp mô hình hiểu được tính tuần hoàn của giờ trong ngày và ngày trong năm, tránh trường hợp giá trị giờ 23 và giờ 0 bị xem là cách xa nhau về mặt số học. Ngoài ra, áp suất khí quyển được chuẩn hóa bằng cách trừ đi mốc 1000 hPa để tạo đặc trưng pressure_centered, giúp hệ số hồi quy ổn định hơn khi huấn luyện.

Các đặc trưng đầu vào của mô hình gồm nhiệt độ hiện tại, độ ẩm, áp suất đã chuẩn hóa, hour_sin, hour_cos, day_sin và day_cos. Dữ liệu được chia theo thứ tự thời gian với 80% đầu dùng để huấn luyện và 20% cuối dùng để kiểm tra. Cách chia này phù hợp với dữ liệu chuỗi thời gian hơn so với chia ngẫu nhiên, vì tập kiểm tra nằm sau tập huấn luyện theo trục thời gian.

```
features = [
```

```
temperature,  
  
humidity,  
  
pressure_centered,  
  
hour_sin,  
  
hour_cos,  
  
day_sin,  
  
day_cos  
]  
  
target = temperature_shift_minus_2_hours
```

Sau khi huấn luyện, mô hình baseline v1.0 đạt MAE khoảng 0,94 °C, RMSE khoảng 1,19 °C và hệ số (R^2) khoảng 0,885 trên tập kiểm tra. Kết quả này cho thấy mô hình tuyến tính có thể mô tả tương đối tốt xu hướng nhiệt độ ngắn hạn trong bộ dữ liệu được sử dụng. Tuy nhiên, khi phân tích sai số theo từng giờ trong ngày, mô hình vẫn tồn tại sai lệch có tính chu kỳ. Vì vậy, bước hiệu chỉnh bias theo giờ được bổ sung để tạo phiên bản v1.1.

Ở bước hiệu chỉnh, sai số trung bình của từng giờ trong ngày được tính theo công thức:

```
[  
bias_h = mean( $T_{\text{actual}}$  -  $T_{\text{predicted}}$ )  
]
```

Trong đó, (bias_h) là sai số trung bình tại giờ (h). Khi dự báo tại Gateway, kết quả MLR ban đầu được cộng thêm giá trị bias tương ứng với giờ hiện tại. Sau hiệu chỉnh, MAE giảm từ khoảng 0,94 °C xuống 0,70 °C, RMSE giảm từ khoảng 1,19 °C xuống 0,95 °C. Tỷ lệ mẫu có sai số trong khoảng ± 1 °C tăng từ khoảng 61,18% lên 77,76%, còn tỷ lệ mẫu có sai số trong khoảng ± 2 °C tăng từ khoảng 90,22% lên 96,19%.

Sau khi hoàn tất huấn luyện và hiệu chỉnh, các hệ số của mô hình được xuất ra file cấu hình để nhúng vào firmware ESP32-S3 Gateway. Gateway không huấn luyện lại mô hình trong quá trình vận hành, mà chỉ thực hiện phép tính dự báo từ các hệ số đã huấn luyện sẵn. Cách

triển khai này phù hợp với tài nguyên của thiết bị nhúng, vì quá trình dự báo chỉ gồm các phép tính số học đơn giản.

```
float mlr_predict_temp_2h(float temperature,
                          float humidity,
                          float pressure,
                          int hour,
                          int day_of_year)
{
    float hour_sin = sinf(2.0f * PI * hour / 24.0f);
    float hour_cos = cosf(2.0f * PI * hour / 24.0f);

    float day_sin = sinf(2.0f * PI * day_of_year / 365.25f);
    float day_cos = cosf(2.0f * PI * day_of_year / 365.25f);

    float pressure_centered = pressure - 1000.0f;

    float y = MLR_B0
              + MLR_B_TEMPERATURE      * temperature
              + MLR_B_HUMIDITY          * humidity
              + MLR_B_PRESSURE_CENTERED * pressure_centered
              + MLR_B_HOUR_SIN          * hour_sin
```

```
+ MLR_B_HOUR_COS      * hour_cos  
  
+ MLR_B_DAY_SIN       * day_sin  
  
+ MLR_B_DAY_COS       * day_cos;  
  
return y + MLR_HOUR_BIAS[hour];  
}
```

Trong luồng vận hành thực tế, mô hình MLR được gọi sau khi Gateway đã xác thực và giải mã thành công frame LoRa. Dữ liệu cảm biến được khôi phục về giá trị vật lý, sau đó đưa vào hàm dự báo cùng với đặc trưng thời gian lấy từ đồng hồ hệ thống. Kết quả `predicted_temp_2h` được gửi lên Supabase cùng với dữ liệu đo thực tế để Web Dashboard hiển thị. Giá trị dự báo này chỉ đóng vai trò tham khảo xu hướng nhiệt độ ngắn hạn, không thay thế dữ liệu cảm biến đo trực tiếp.

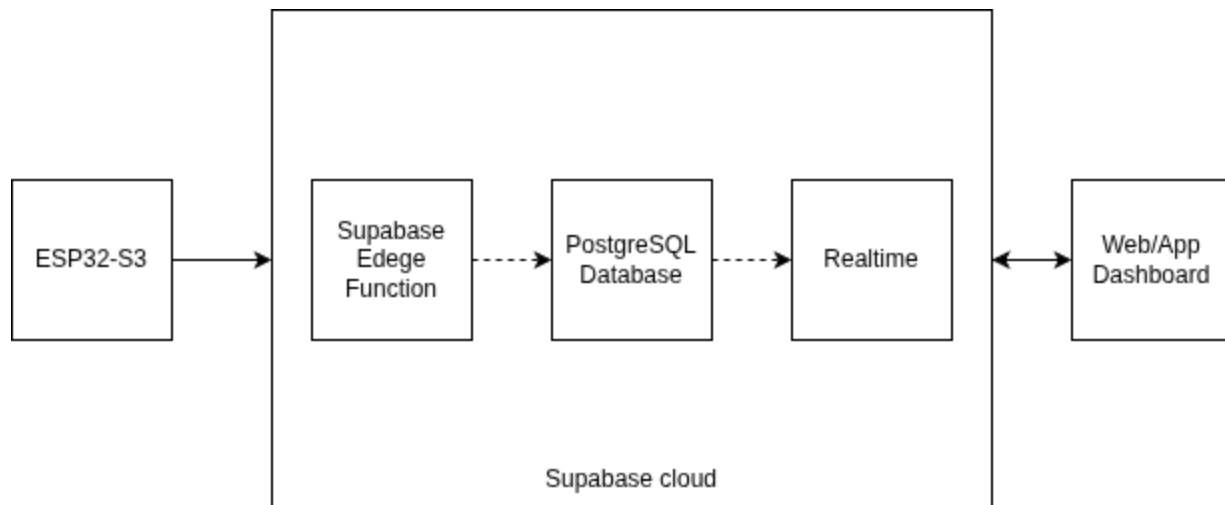
4.4. Triển khai Supabase, Web Dashboard và OTA

4.4.1. Supabase Cloud

Supabase được sử dụng làm nền tảng tiếp nhận, lưu trữ và quản lý dữ liệu. Edge Function đóng vai trò điểm vào của dữ liệu từ Gateway. Khi nhận request, Edge Function kiểm tra `x-device-token`, kiểm tra cấu trúc payload, giải mã dữ liệu bằng ASCON key2 và chỉ ghi dữ liệu hợp lệ vào PostgreSQL Database.

Cơ sở dữ liệu PostgreSQL lưu các nhóm dữ liệu chính gồm dữ liệu quan trắc, trạng thái Gateway và cấu hình thiết bị. Dữ liệu quan trắc chứa các trường như nhiệt độ, độ ẩm, áp suất, điện áp pin, `frame_counter` và `predicted_temp_2h`. Trạng thái Gateway chứa thông tin heartbeat như `firmware_version`, `firmware_sha256`, RSSI WiFi, uptime và free heap. Bảng cấu hình thiết bị chứa thông tin phục vụ OTA như `ota_url`, `ota_seq` và `ota_sha256`.

Supabase Storage được sử dụng để lưu file firmware .bin phục vụ OTA. Khi người dùng upload firmware từ Web Dashboard, file được lưu vào bucket firmware, sau đó URL công khai và thông tin SHA-256 được ghi vào bảng cấu hình thiết bị. Gateway đọc các trường này để quyết định có thực hiện cập nhật hay không.



Hình 4.7. Luồng triển khai Supabase Cloud

4.4.2. Web Dashboard và OTA firmware update

Web Dashboard được triển khai để hiển thị dữ liệu quan trắc, biểu đồ lịch sử, trạng thái Gateway, trạng thái tuyến LoRa và thông tin firmware. Giao diện đọc dữ liệu từ Supabase để hiển thị giá trị mới nhất và các bản ghi lịch sử. Ngoài chức năng giám sát, Dashboard còn hỗ trợ upload firmware cho Gateway.

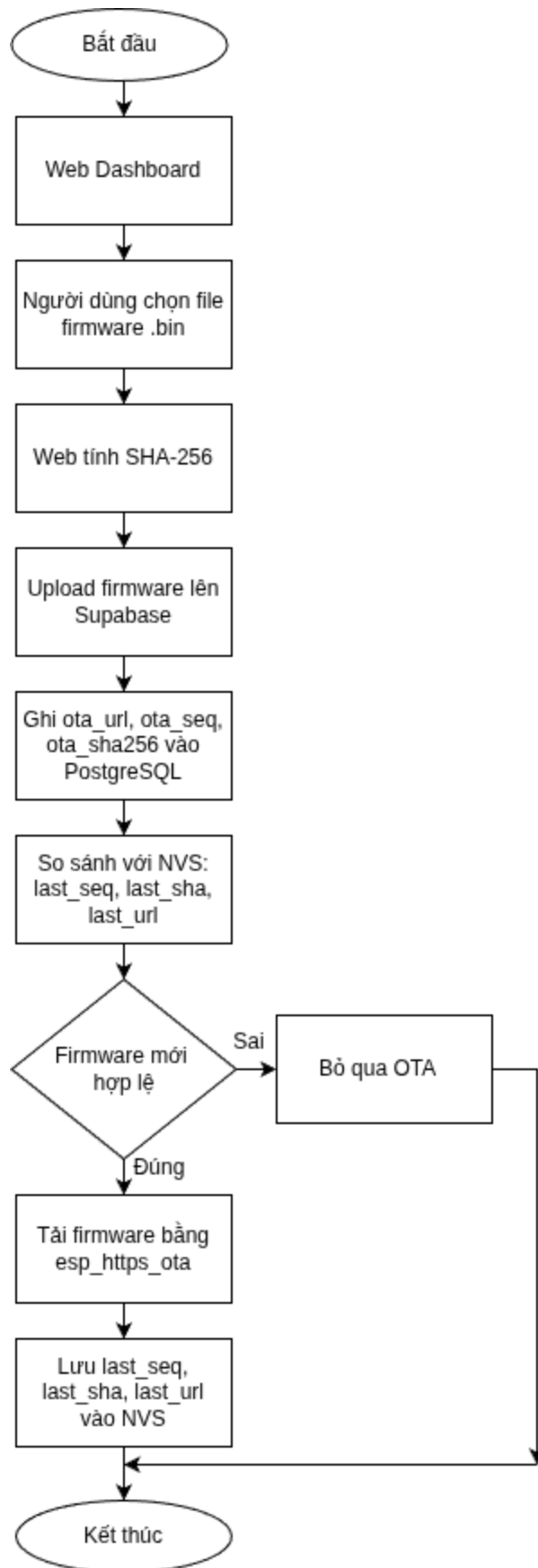
Khi người dùng chọn file firmware .bin, Web Dashboard tính SHA-256 của file trước khi upload. Sau đó, file được đưa lên Supabase Storage và bảng `device_configs` được cập nhật với `ota_url`, `ota_seq` và `ota_sha256`. `ota_seq` đóng vai trò số thứ tự bản cập nhật, còn `ota_sha256` giúp Gateway nhận biết firmware đã được áp dụng trước đó hay chưa.

Ở phía ESP32-S3, task OTA định kỳ đọc cấu hình OTA từ Supabase. Nếu `ota_seq` cũ hơn hoặc `ota_sha256` trùng với bản đã lưu trong NVS, Gateway bỏ qua cập nhật. Nếu phát hiện firmware mới hợp lệ, Gateway tạm dừng các request HTTP khác, tải firmware bằng `esp_https_ota`, ghi firmware vào phân vùng OTA, lưu trạng thái đã cập nhật vào NVS và khởi động lại

```
void ota_task(void *arg)
{
    while (1) {
```

```
if (!wifi_is_connected()) {  
    delay_next_check();  
    continue;  
}  
  
ota_config_t cfg = fetch_ota_config_from_supabase();  
if (is_new_firmware(cfg)) {  
    pause_http_posts();  
    if (esp_https_ota_update(cfg.ota_url) == ESP_OK) {  
        save_ota_state_to_nvs(cfg.ota_seq, cfg.ota_sha256);  
        esp_restart();  
    }  
    resume_http_posts();  
}  
    delay_next_check();  
}  
}
```

Việc lưu thông tin OTA trong NVS được thực hiện bằng cách ghi đè các khóa cố định như `last_seq`, `last_sha` và `last_url`. Firmware không tạo thêm nhiều bản ghi mới, mà chỉ lưu trạng thái firmware gần nhất đã áp dụng. Cách này phù hợp với mục tiêu tránh cập nhật lặp lại cùng một firmware và giảm dữ liệu ghi không cần thiết vào NVS.



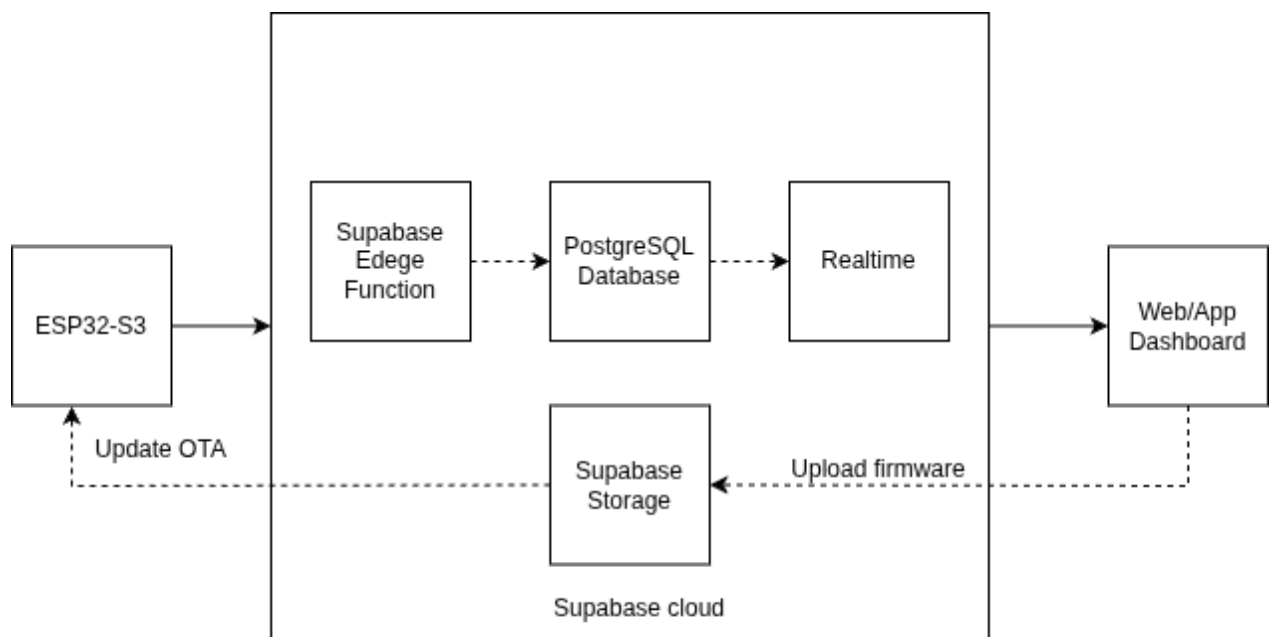
Hình 4.8. Luồng cập nhật firmware OTA cho ESP32-S3 Gateway

4.5. Tích hợp và vận hành hệ thống sau triển khai

Sau khi các thành phần phần cứng, firmware, Cloud và Web Dashboard được triển khai riêng lẻ, hệ thống được tích hợp thành một luồng vận hành hoàn chỉnh. Trạm đo STM32 Sensor Node được nạp firmware bằng ST-Link và cấp nguồn từ khối pin – năng lượng mặt trời. Gateway được nạp firmware bằng ESP-IDF, cấu hình WiFi bằng BLE provisioning nếu chưa có thông tin mạng, sau đó khởi động các task nhận LoRa, gửi Cloud, heartbeat và OTA.

Luồng dữ liệu chính bắt đầu khi STM32 Sensor Node đến chu kỳ truyền. Node đọc cảm biến, đóng gói payload, reserve frame_counter, mã hóa bằng ASCON key1 và gửi frame 18 byte qua LoRa. ESP32-S3 Gateway nhận frame từ module LoRa E32, gom đủ 18 byte, tạo nonce, xác thực và giải mã bằng ASCON key1. Nếu frame hợp lệ, Gateway khôi phục dữ liệu vật lý, chạy mô hình dự báo, mã hóa lại payload bằng ASCON key2 và gửi dữ liệu lên Supabase Edge Function bằng HTTPS.

Ở phía Cloud, Edge Function kiểm tra token thiết bị, giải mã payload key2 và ghi dữ liệu hợp lệ vào PostgreSQL Database. Web Dashboard đọc dữ liệu từ Supabase để cập nhật thông số môi trường, biểu đồ lịch sử, trạng thái Gateway và thông tin firmware. Song song với luồng dữ liệu cảm biến, Gateway gửi heartbeat định kỳ để duy trì trạng thái vận hành trên giao diện.



Hình 4.9. Luồng tích hợp vận hành hệ thống sau triển khai

Trong quá trình vận hành, log UART của STM32 và ESP32-S3 được sử dụng để theo dõi trạng thái firmware. Log STM32 cho biết quá trình khởi tạo cảm biến, đóng gói payload, mã hóa và truyền LoRa. Log ESP32-S3 cho biết trạng thái WiFi, frame LoRa nhận được, kết quả xác thực ASCON, trạng thái gửi Edge Function, heartbeat và OTA. Các log chi tiết, ảnh dashboard, dữ liệu ghi trên Supabase và kết quả kiểm tra bằng Wireshark được trình bày ở chương thực nghiệm và đánh giá.

CHƯƠNG 5. THỰC NGHIỆM, KẾT QUẢ VÀ ĐÁNH GIÁ

Sau khi hoàn thành thiết kế phần cứng, xây dựng firmware và triển khai nền tảng Cloud, hệ thống quan trắc thời tiết cục bộ được đưa vào vận hành thực nghiệm để đánh giá khả năng hoạt động theo chuỗi xử lý dữ liệu thực tế. Dữ liệu được tạo ra tại STM32 Sensor Node, truyền qua tuyến LoRa đến ESP32-S3 Gateway, sau đó được xử lý, gửi lên Supabase và hiển thị trên Web Dashboard cũng như App giám sát.

Nội dung chương này tập trung vào các kết quả ghi nhận trong quá trình vận hành thay vì mô tả lại cấu trúc thiết kế. Các bằng chứng thực nghiệm được lấy từ log thiết bị, dữ liệu lưu trên Supabase, giao diện hiển thị và công cụ quan sát mạng ở những phần cần đánh giá độ trễ. Cách trình bày này giúp đối chiếu được dữ liệu tại từng lớp của hệ thống, từ lúc node tạo payload đến khi dữ liệu xuất hiện trên giao diện người dùng.

Các phần tiếp theo được sắp xếp theo đúng luồng hoạt động của hệ thống: kết quả vận hành STM32 Sensor Node, kết quả truyền dữ liệu LoRa và xác thực ASCON key1, xử lý dữ liệu tại ESP32-S3 Gateway, lưu trữ và hiển thị trên Supabase/Web Dashboard/App, đánh giá độ trễ HTTPS, đánh giá mô hình Multiple Linear Regression và kết quả cập nhật firmware OTA. Phần cuối chương tổng hợp các kết quả đạt được và những hạn chế còn tồn tại trong quá trình thực nghiệm.

5.1. Kết quả vận hành STM32 Sensor Node

STM32 Sensor Node là điểm tạo dữ liệu ban đầu của hệ thống. Trong quá trình thực nghiệm, node được vận hành theo chu kỳ để phù hợp với yêu cầu tiết kiệm năng lượng của trạm đo ngoài trời. Khi đến thời điểm truyền dữ liệu, vi điều khiển đánh thức hệ thống, đọc dữ liệu từ các cảm biến, tạo payload, mã hóa bằng ASCON-128a và gửi frame qua module LoRa E32. Sau khi hoàn thành chu kỳ truyền, node đưa các khối liên quan về trạng thái tiết kiệm năng lượng và chờ lần đánh thức tiếp theo.

Log UART của STM32 cho thấy firmware phân biệt được giữa lần đánh thức duy trì hệ thống và lần đánh thức đến chu kỳ truyền dữ liệu. Ở các lần chưa đến thời điểm gửi, node chỉ thực hiện các thao tác cần thiết như làm mới watchdog, đặt lại RTC alarm và quay lại Standby. Khi đến chu kỳ truyền, node thực hiện đầy đủ quá trình đo và truyền. Kết quả này xác nhận cơ chế vận hành theo chu kỳ đã hoạt động đúng, đồng thời hạn chế thời gian vi điều khiển và module truyền thông ở trạng thái tiêu thụ cao.

```

[SYS] Skip E32 config check before TX
[SYS] ===== APP CYCLE START =====
[SHT30] I2C write fail: err=0x00000020
sht30: write command failed.
[SENSORS] SHT30 init FAILED
[BMP388] I2C read fail: err=0x00000020
bmp388: read chip id failed.
[SENSORS] BMP388 init FAILED
[SENSORS] INA219 init FAILED
[SENSORS] Init status = 0x07
[DATA] T=-1 H=65535 P=65535 BT=-1 VBAT=0 (raw) HF=0x07
[PAYLOAD] (10B): FF FF FF FF FF FF FF FF 00 07
[FRAME] (18B): 19 00 00 00 77 EA 80 DE 21 6F 7D AF 18 C6 CD 75 ED 2C
[LORA] AUX timeout before TX
[SYS] LoRa TX FAILED for frame32=25
[RTC] Next alarm in 5 sec at 00:08:27
[PWR] Enter STANDBY

```

Hình 5.1. Log STM32 Sensor Node trong một chu kỳ đo và truyền dữ liệu

Trong chu kỳ truyền hợp lệ, node đọc dữ liệu từ SHT30, BMP388 và INA219. Các giá trị đo được chuyển sang dạng số nguyên theo hệ số tỉ lệ để đóng gói vào payload 10 byte. Payload này chứa các trường cần thiết cho quá trình khôi phục dữ liệu ở Gateway, bao gồm nhiệt độ, độ ẩm, áp suất khí quyển, trạng thái nguồn và health_flag. Việc quan sát payload ở dạng hex giúp xác nhận kích thước và thứ tự byte trước khi dữ liệu được mã hóa.

```

[DATA] T=-1 H=65535 P=65535 BT=-1 VBAT=0 (raw) HF=0x07
[PAYLOAD] (10B): FF FF FF FF FF FF FF FF 00 07
[FRAME] (18B): 19 00 00 00 77 EA 80 DE 21 6F 7D AF 18 C6 CD 75 ED 2C

```

Hình 5.2. Payload 10 byte được tạo tại STM32 Sensor Node

Sau khi payload được tạo, STM32 sử dụng frame_counter để tạo nonce và mã hóa dữ liệu bằng ASCON-128a với khóa thứ nhất. Dữ liệu sau mã hóa gồm ciphertext và authentication tag. Các thành phần này được ghép với frame_counter để tạo frame LoRa 18 byte. Log frame cho thấy dữ liệu trước khi gửi có đúng kích thước thiết kế, gồm 4 byte frame_counter, 10 byte ciphertext và 4 byte tag xác thực.

```

[PAYLOAD] (10B): FF FF FF FF FF FF FF FF 00 07
[FRAME] (18B): 14 00 00 00 73 C5 5C 57 FD 09 D4 E8 DA D3 5C FE 51 E8
[LORA] AUX timeout before TX

```

Hình 5.3. Frame LoRa sau khi mã hóa tại STM32 Sensor Node

Trong quá trình truyền, STM32 điều khiển module LoRa E32 thông qua các chân M0, M1 và AUX. Module được đưa về chế độ truyền trước khi gửi frame, sau đó firmware chờ AUX phản ánh trạng thái hoàn tất để chuyển module về trạng thái tiết kiệm năng lượng. Đồng thời, chân PS/SYNC của TPS63020 được điều khiển trong thời điểm phát LoRa để đáp ứng trạng thái tải cao hơn của module truyền thông. Kết quả này cho thấy firmware đã

phối hợp giữa xử lý dữ liệu, truyền thông và quản lý nguồn trong cùng một chu kỳ vận hành.



Hình 5.4. Trạng thái chuyển đổi mode PS/SYNC

Từ các hình và log thực nghiệm, có thể thấy STM32 Sensor Node đã thực hiện được vai trò của trạm đo: đọc dữ liệu cảm biến, tạo payload đúng kích thước, mã hóa bằng ASCON-128a và truyền frame qua LoRa. Các yếu tố cần tiếp tục theo dõi trong vận hành dài hạn gồm độ ổn định của bus I2C, trạng thái nguồn pin, thời gian chờ cảm biến ổn định và chất lượng tín hiệu LoRa khi điều kiện môi trường thay đổi.

5.2. Kết quả truyền dữ liệu LoRa và xác thực ASCON

Sau khi STM32 Sensor Node tạo frame LoRa 18 byte, dữ liệu được gửi đến ESP32-S3 Gateway thông qua module LoRa E32. Ở phía Gateway, dữ liệu từ module LoRa được nhận qua UART và đưa vào bộ đệm xử lý. Firmware chỉ chuyển frame sang bước giải mã khi bộ đệm nhận đủ kích thước 18 byte theo định dạng đã thiết kế. Cách xử lý này giúp hạn

chế trường hợp dữ liệu thiếu byte hoặc dữ liệu bị ngắt quãng vẫn bị đưa vào bước xác thực ASCON.

Hình 5.5 thể hiện log ESP32-S3 Gateway khi nhận được frame LoRa hợp lệ từ STM32 Sensor Node. Trong log này, Gateway nhận đủ số byte của một frame hoàn chỉnh, sau đó chuyển frame sang hàm xử lý. Đây là bằng chứng đầu tiên cho thấy tuyến LoRa đã truyền được dữ liệu từ node đến Gateway ở mức frame.

```
(555085) WEATHER_GATEWAY: [LORA_RX] Đủ 18 byte, xử lý frame:
(555085) WEATHER_GATEWAY: 0x3fcee100 28 09 00 00 7c de a9 08 08 35 a9 31 09 c2 83 d2 |(...|....5.1....|
(555085) WEATHER_GATEWAY: 0x3fcee110 04 d6 |...|
(555095) WEATHER_GATEWAY: [LORA_RX] Gói tin Counter=2344 - Đang giải mã ASCON-128a...
(555105) WEATHER_GATEWAY: [LORA_RX] ✅ T=29.81°C H=80.53% P=1004.22hPa
(555105) MLR_ENGINE: MLR v3.8.3 | T=29.81 H=80.53 P=1004.22 | hour=23 doy=181 | raw=26.99 bias=0.65 -> pred=27.65
```

Hình 5.5. Log ESP32-S3 Gateway nhận đủ frame LoRa 18 byte

Sau khi nhận đủ frame, Gateway tách dữ liệu thành các trường gồm frame_counter, ciphertext và authentication tag. frame_counter được dùng để tạo lại nonce theo cùng quy ước với STM32 Sensor Node. Nếu nonce được tạo lại đúng và authentication tag hợp lệ, Gateway có thể giải mã ciphertext bằng ASCON-128a key1 để khôi phục payload 10 byte ban đầu. Ngược lại, nếu frame bị sai byte, sai tag hoặc không khớp nonce, quá trình xác thực sẽ thất bại và dữ liệu bị loại bỏ.

Hình 5.6 thể hiện log xác thực và giải mã frame LoRa tại Gateway. Kết quả này cho thấy frame nhận được không chỉ đúng kích thước, mà còn vượt qua bước kiểm tra ASCON. Điều này có ý nghĩa quan trọng vì trong hệ thống sử dụng mã hóa xác thực, một frame chỉ được xem là hợp lệ khi vừa nhận đủ dữ liệu, vừa xác thực thành công và khôi phục được payload.

```
(555085) WEATHER_GATEWAY: [LORA_RX] Đủ 18 byte, xử lý frame:
(555085) WEATHER_GATEWAY: 0x3fcee100 28 09 00 00 7c de a9 08 08 35 a9 31 09 c2 83 d2 |(...|....5.1....|
(555085) WEATHER_GATEWAY: 0x3fcee110 04 d6 |...|
(555095) WEATHER_GATEWAY: [LORA_RX] Gói tin Counter=2344 - Đang giải mã ASCON-128a...
(555105) WEATHER_GATEWAY: [LORA_RX] ✅ T=29.81°C H=80.53% P=1004.22hPa
(555105) MLR_ENGINE: MLR v3.8.3 | T=29.81 H=80.53 P=1004.22 | hour=23 doy=181 | raw=26.99 bias=0.65 -> pred=27.65
```

Hình 5.6. Log xác thực và giải mã frame LoRa bằng ASCON key1

Từ kết quả quan sát, tuyến LoRa giữa STM32 Sensor Node và ESP32-S3 Gateway đã đáp ứng được yêu cầu truyền dữ liệu cục bộ của hệ thống. Frame do STM32 tạo ra có thể được Gateway nhận đủ, tách đúng cấu trúc, xác thực bằng ASCON key1 và giải mã thành payload 10 byte. Như vậy, dữ liệu sau khi truyền qua LoRa vẫn giữ được tính hợp lệ để tiếp tục xử lý ở Gateway.

Trong quá trình thực nghiệm cũng xuất hiện một số trường hợp frame không được xử lý. Khi Gateway nhận chưa đủ 18 byte hoặc khoảng cách giữa các byte vượt quá ngưỡng cho phép, firmware xóa bộ đệm đang nhận dở và chờ frame mới. Nếu frame đủ kích thước nhưng dữ liệu bị sai lệch trong quá trình truyền, bước xác thực ASCON không thành công và frame không được gửi tiếp lên Cloud. Cơ chế này giúp hệ thống tránh sử dụng dữ liệu không hoàn chỉnh hoặc dữ liệu không hợp lệ.

Kết quả trên cho thấy tuyến LoRa đã thực hiện được vai trò truyền frame từ Sensor Node đến Gateway, đồng thời Gateway có khả năng lọc bỏ frame lỗi trước khi dữ liệu đi vào các bước phía sau. Tuy nhiên, phạm vi thực nghiệm hiện tại mới dừng ở mức xác nhận truyền nhận frame hợp lệ và xử lý frame lỗi. Các đánh giá định lượng như số frame gửi, số frame nhận thành công, số frame bị timeout, tỷ lệ mất gói theo khoảng cách và ảnh hưởng của vật cản cần được thực hiện thêm nếu muốn đánh giá đầy đủ độ ổn định của tuyến LoRa trong môi trường triển khai thực tế.

5.3. Kết quả xử lý dữ liệu tại ESP32-S3 Gateway

Sau khi frame LoRa vượt qua bước xác thực, ESP32-S3 Gateway khôi phục payload 10 byte về các giá trị vật lý. Nhiệt độ và độ ẩm được chuyển đổi lại theo hệ số tỉ lệ đã dùng ở phía STM32. Áp suất khí quyển được khôi phục về đơn vị hPa dựa trên quy ước trong quá trình đóng gói. Trạng thái nguồn và health_flag được giữ lại để phản ánh tình trạng phần cứng tại thời điểm đo.

Hình 5.7 thể hiện log khôi phục dữ liệu cảm biến tại Gateway sau khi giải mã thành công payload. Các giá trị hiển thị trong log cho thấy dữ liệu không dừng ở mức byte thô, mà đã được chuyển đổi về dạng có ý nghĩa vật lý để phục vụ lưu trữ, hiển thị và dự báo.

```
[ (555095) WEATHER_GATEWAY: [LORA_RX] Gói tin Counter=2344 - Đang giải mã ASCON-128a...  
[ (555105) WEATHER_GATEWAY: [LORA_RX] ✅ T=29.81°C H=80.53% P=1004.22hPa
```

Hình 5.7. Log khôi phục dữ liệu cảm biến tại ESP32-S3 Gateway

Sau khi khôi phục dữ liệu, Gateway tạo giá trị dự báo nhiệt độ cục bộ bằng mô hình Multiple Linear Regression. Mô hình sử dụng các đặc trưng gồm nhiệt độ, độ ẩm, áp suất khí quyển và thông tin thời gian. Khi thời gian hệ thống đã được đồng bộ, Gateway tạo trường predicted_temp_2h và đưa kết quả này vào dữ liệu gửi Cloud. Nếu thời gian chưa hợp lệ, Gateway vẫn có thể gửi dữ liệu đo nhưng không tạo giá trị dự báo để tránh sai lệch do thiếu bối cảnh thời gian.

```
[ (555105) WEATHER_GATEWAY: [LORA_RX] ✅ T=29.81°C H=80.53% P=1004.22hPa  
[ (555105) MLR_ENGINE: MLR v3.8.3 | T=29.81 H=80.53 P=1004.22 | hour=23 doy=181 | raw=26.99 bias=0.65 -> pred=27.65
```

Hình 5.8. Log tạo giá trị predicted_temp_2h tại ESP32-S3 Gateway

Trước khi gửi dữ liệu lên Supabase, Gateway mã hóa lại payload bằng ASCON-128a key2. Kết quả mã hóa được chuyển sang chuỗi hex dưới tên payload_key2 và đưa vào JSON gửi đến Supabase Edge Function. Việc sử dụng key2 giúp tách phạm vi bảo vệ giữa tuyến STM32 – Gateway và tuyến Gateway – Cloud. Khi đó, dữ liệu sau khi ra khỏi Gateway không còn là payload cảm biến dạng rõ mà là dữ liệu đã được bảo vệ thêm một lớp trước khi đi qua HTTPS.

```
Y: [EDGE_SENSOR] POST frame=2345 payload_key2(tag16)=9010214d45b3e46e0dcdca0e5e7c1375423f82b83f71161bc7f0
Y: [OTA_FIRMWARE] GET frame=2346 payload_key2(tag16)=9010214d45b3e46e0dcdca0e5e7c1375423f82b83f71161bc7f0
```

Hình 5.9. Log tạo payload_key2 trước khi gửi Supabase

Trong firmware Gateway, tác vụ nhận LoRa và tác vụ gửi Cloud được tách riêng bằng queue. Dữ liệu sau khi giải mã và xử lý được đưa vào queue để tác vụ HTTPS gửi lên Supabase. Cách tổ chức này giúp quá trình nhận LoRa không bị chặn bởi độ trễ mạng hoặc thời gian phản hồi của Edge Function. Nếu WiFi chưa sẵn sàng hoặc queue gửi Cloud bị đầy, Gateway có thể trì hoãn hoặc bỏ qua một số dữ liệu để duy trì khả năng tiếp nhận frame mới.

Từ kết quả trên, ESP32-S3 Gateway đã thực hiện đúng vai trò trung gian của hệ thống. Gateway không chỉ chuyển tiếp dữ liệu từ LoRa lên Cloud, mà còn kiểm tra frame, giải mã, khôi phục dữ liệu, tạo dự báo, mã hóa lại payload và điều phối luồng gửi HTTPS. Các yếu tố có thể ảnh hưởng đến quá trình này gồm trạng thái WiFi, thời gian hệ thống, phản hồi từ Edge Function, dung lượng queue và bộ nhớ còn lại của Gateway.

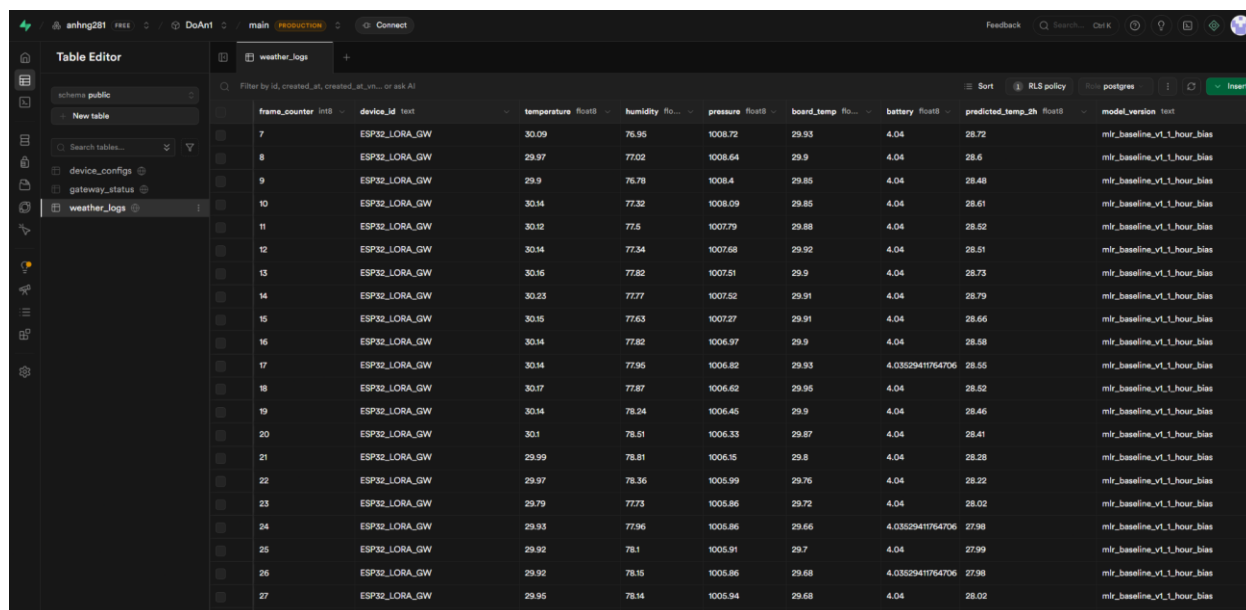
5.4. Kết quả lưu trữ và hiển thị trên Supabase/Web Dashboard/App

Sau khi ESP32-S3 Gateway xử lý xong dữ liệu từ tuyến LoRa, dữ liệu được gửi lên Supabase thông qua request HTTPS đến Edge Function. Ở tầng Cloud, Edge Function tiếp nhận request từ Gateway, kiểm tra token thiết bị, xác thực payload đã được bảo vệ bằng ASCON key2, giải mã dữ liệu và ghi bản ghi hợp lệ vào PostgreSQL Database. Kết quả thực nghiệm cho thấy luồng gửi dữ liệu từ Gateway lên Supabase đã hình thành được chuỗi lưu trữ hoàn chỉnh, trong đó dữ liệu cảm biến sau xử lý có thể được ghi nhận trên cơ sở dữ liệu và phục vụ hiển thị trên Web Dashboard cũng như App giám sát.

Trong quá trình gửi dữ liệu, ESP32-S3 Gateway đóng gói các thông tin cần thiết vào JSON, bao gồm mã thiết bị, phiên bản firmware, SHA-256 firmware đang chạy, frame_counter, payload_key2, predicted_temp_2h và các thông tin trạng thái như WiFi RSSI, địa chỉ IP,

uptime và free heap. Các thông tin này giúp bản ghi trên Cloud không chỉ chứa dữ liệu môi trường, mà còn phản ánh trạng thái vận hành của Gateway tại thời điểm gửi dữ liệu.

Ở phía Supabase, Edge Function kiểm tra request trước khi ghi dữ liệu vào cơ sở dữ liệu. Nếu token thiết bị không hợp lệ, request bị từ chối. Nếu payload_key2 không giải mã được hoặc authentication tag không hợp lệ, dữ liệu không được ghi vào bảng. Khi request hợp lệ, Edge Function giải mã payload, khôi phục các trường dữ liệu cảm biến và ghi vào PostgreSQL Database. Cách xử lý này giúp dữ liệu không đi trực tiếp từ thiết bị vào cơ sở dữ liệu mà phải thông qua lớp kiểm tra trung gian.



frame_counter	device_id	temperature	humidity	pressure	board_temp	battery	predicted_temp_2h	model_version
7	ESP32_LORA_GW	30.09	76.95	1008.72	29.93	4.04	28.72	mir_baseline_v1.1_hour_bias
8	ESP32_LORA_GW	29.97	77.02	1008.64	29.9	4.04	28.6	mir_baseline_v1.1_hour_bias
9	ESP32_LORA_GW	29.9	76.78	1008.4	29.85	4.04	28.48	mir_baseline_v1.1_hour_bias
10	ESP32_LORA_GW	30.34	77.32	1008.09	29.85	4.04	28.61	mir_baseline_v1.1_hour_bias
11	ESP32_LORA_GW	30.12	77.5	1007.79	29.88	4.04	28.52	mir_baseline_v1.1_hour_bias
12	ESP32_LORA_GW	30.14	77.34	1007.68	29.92	4.04	28.51	mir_baseline_v1.1_hour_bias
13	ESP32_LORA_GW	30.16	77.82	1007.51	29.9	4.04	28.73	mir_baseline_v1.1_hour_bias
14	ESP32_LORA_GW	30.23	77.77	1007.52	29.91	4.04	28.79	mir_baseline_v1.1_hour_bias
15	ESP32_LORA_GW	30.15	77.63	1007.27	29.91	4.04	28.66	mir_baseline_v1.1_hour_bias
16	ESP32_LORA_GW	30.14	77.82	1006.97	29.9	4.04	28.58	mir_baseline_v1.1_hour_bias
17	ESP32_LORA_GW	30.14	77.95	1006.82	29.93	4.0329411764706	28.55	mir_baseline_v1.1_hour_bias
18	ESP32_LORA_GW	30.17	77.87	1006.62	29.95	4.04	28.52	mir_baseline_v1.1_hour_bias
19	ESP32_LORA_GW	30.14	78.24	1006.45	29.9	4.04	28.46	mir_baseline_v1.1_hour_bias
20	ESP32_LORA_GW	30.1	78.51	1006.33	29.87	4.04	28.41	mir_baseline_v1.1_hour_bias
21	ESP32_LORA_GW	29.99	78.81	1005.15	29.8	4.04	28.28	mir_baseline_v1.1_hour_bias
22	ESP32_LORA_GW	29.97	78.36	1005.99	29.75	4.04	28.22	mir_baseline_v1.1_hour_bias
23	ESP32_LORA_GW	29.79	77.73	1005.86	29.72	4.04	28.02	mir_baseline_v1.1_hour_bias
24	ESP32_LORA_GW	29.93	77.96	1005.86	29.66	4.0329411764706	27.98	mir_baseline_v1.1_hour_bias
25	ESP32_LORA_GW	29.92	78.1	1005.91	29.7	4.04	27.99	mir_baseline_v1.1_hour_bias
26	ESP32_LORA_GW	29.92	78.15	1005.86	29.68	4.0329411764706	27.98	mir_baseline_v1.1_hour_bias
27	ESP32_LORA_GW	29.95	78.14	1005.94	29.68	4.04	28.02	mir_baseline_v1.1_hour_bias

Hình 5.10. Bản ghi dữ liệu cảm biến trên bảng weather_logs

Dữ liệu sau khi đi qua tuyến STM32 – LoRa – ESP32-S3 – Supabase vẫn giữ được đầy đủ các trường cần thiết cho quá trình giám sát. Các thông tin như nhiệt độ, độ ẩm, áp suất khí quyển, trạng thái nguồn, health_flag, frame_counter và predicted_temp_2h được lưu lại để phục vụ hiển thị và phân tích lịch sử. frame_counter hỗ trợ theo dõi thứ tự gói tin từ node cảm biến, trong khi health_flag giúp nhận biết trạng thái lỗi của cảm biến hoặc phần cứng tại thời điểm đo.

Bên cạnh dữ liệu cảm biến, Gateway cũng gửi trạng thái hoạt động định kỳ lên Supabase. Các thông tin trạng thái gồm firmware version, firmware SHA-256, WiFi RSSI, địa chỉ IP, uptime, free heap và thời điểm Gateway còn hoạt động. Dữ liệu trạng thái này được lưu tách biệt với dữ liệu cảm biến để phục vụ quá trình giám sát vận hành. Nhờ đó, hệ thống

có thể phân biệt giữa trường hợp Gateway vẫn hoạt động nhưng chưa nhận được frame LoRa mới và trường hợp Gateway không còn cập nhật trạng thái lên Cloud.

device_id	gateway_last_seen_at	gateway_last_seen_vn	last_lora_packet_at	last_lora_packet_vn	last_frame_counter
ESP32_LORA_GW	2026-06-28 07:34:24.002+00	2026-06-28 14:34:24.002	2026-06-28 07:25:57.867+00	2026-06-28 14:25:57.867	2017

mqtt_enabled	mqtt_connected	mqtt_last_change_at	mqtt_last_change_vn	mqtt_broker	wifi_rssi	firmware_version	ip_address
FALSE	FALSE	2026-06-28 07:34:24.002+00	2026-06-28 14:34:24.002	NULL	-54	3.8.3.17-production-wifi-ble	192.168.1.1

ip_address	uptime_sec	free_heap	status_note	updated_at	updated_at_vn	firmware_sha256	last_applied_ota_sh
192.168.100.124	69718	213752	gateway heartbeat	2026-06-28 07:35:29.442845+00	2026-06-28 14:35:29.442845	9e9ec782886177b678ba1b3fe5a672b89d	NULL

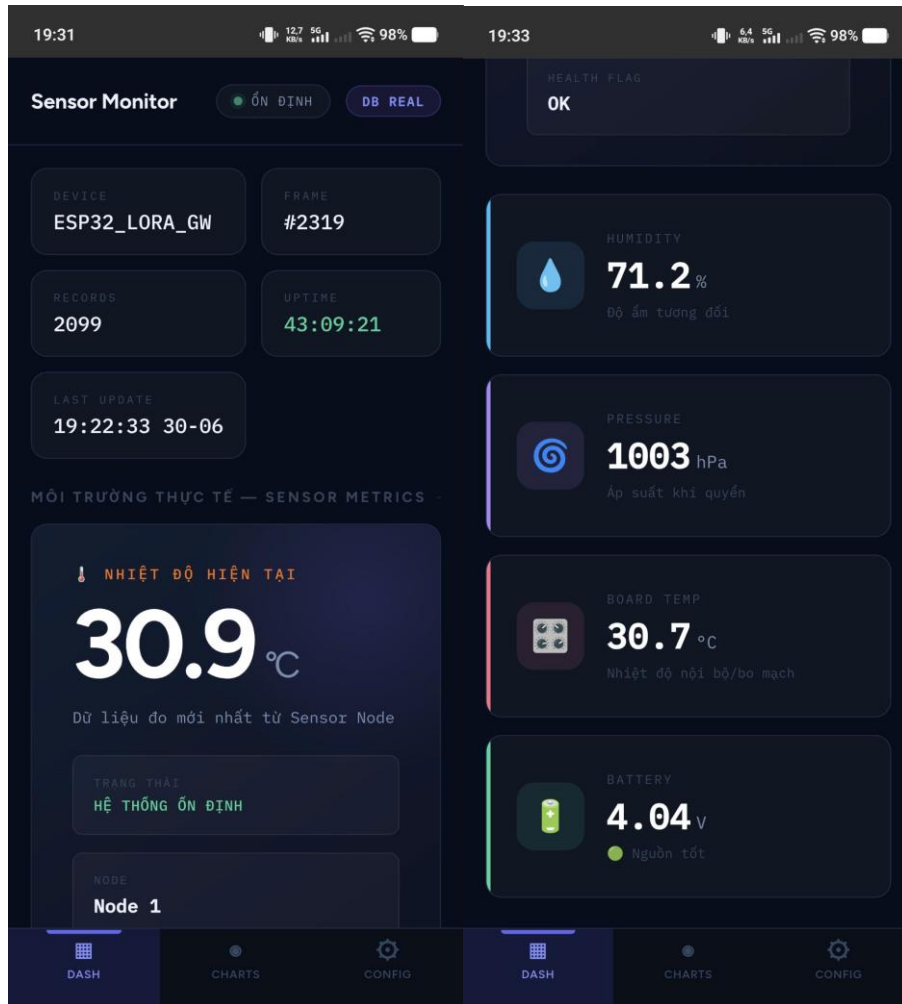
Hình 5.11. Bản ghi trạng thái ESP32-S3 Gateway trên Supabase

Web Dashboard sử dụng dữ liệu đã lưu trên Supabase để hiển thị thông tin cho người dùng. Giao diện hiển thị các thông số môi trường mới nhất, giá trị dự báo nhiệt độ cục bộ, trạng thái Gateway, trạng thái tuyến LoRa, biểu đồ lịch sử và thông tin phục vụ OTA. Việc tách dữ liệu cảm biến và dữ liệu trạng thái Gateway giúp giao diện phản ánh được cả hai khía cạnh: giá trị đo của môi trường và trạng thái vận hành của hệ thống.



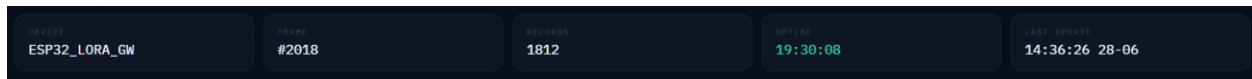
Hình 5.12. Web Dashboard hiển thị dữ liệu quan trắc mới nhất và biểu đồ lịch sử

Bên cạnh Web Dashboard, hệ thống còn được theo dõi thông qua App giám sát. App sử dụng dữ liệu đã lưu trên Supabase để hiển thị các thông số quan trắc chính và trạng thái vận hành. Trong quá trình thực nghiệm, App được dùng để đối chiếu khả năng truy xuất dữ liệu mới nhất từ Supabase với Web Dashboard. Các trường như nhiệt độ, độ ẩm, áp suất khí quyển, giá trị dự báo, thời điểm cập nhật và trạng thái Gateway được hiển thị để người dùng có thể theo dõi nhanh trên thiết bị di động hoặc môi trường ứng dụng.



Hình 5.13. App giám sát hiển thị dữ liệu quan trắc từ Supabase

Trong quá trình thử nghiệm, logic hiển thị trạng thái trên dashboard có vai trò quan trọng. Nếu chỉ dựa vào thời điểm bản ghi cảm biến mới nhất, giao diện có thể hiển thị cảnh báo sai trong trường hợp chu kỳ gửi LoRa dài hoặc Gateway vẫn hoạt động nhưng chưa đến thời điểm nhận frame mới. Do đó, trạng thái trên dashboard cần được xác định dựa trên cả dữ liệu cảm biến, thời điểm nhận LoRa cuối cùng và heartbeat của Gateway. Cách xử lý này giúp giao diện phản ánh đúng hơn trạng thái thực tế của hệ thống.



Hình 5.14. Trạng thái Gateway và tuyến LoRa trên Web Dashboard

Kết quả lưu trữ và hiển thị cho thấy Supabase đã thực hiện được vai trò trung tâm dữ liệu của hệ thống. Dữ liệu sau khi được Gateway xử lý có thể được ghi vào PostgreSQL và

được truy xuất bởi cả Web Dashboard lẫn App giám sát. Web Dashboard phù hợp cho quan sát tổng thể, biểu đồ lịch sử, trạng thái Gateway và thao tác OTA, trong khi App giúp theo dõi nhanh các thông số quan trắc chính. Tuy nhiên, khả năng cập nhật dữ liệu trên giao diện vẫn phụ thuộc vào trạng thái WiFi của Gateway, thời gian phản hồi của Edge Function, tốc độ ghi cơ sở dữ liệu và cơ chế làm mới dữ liệu của giao diện. Vì vậy, khi dữ liệu chưa hiển thị đúng thời điểm, cần đối chiếu đồng thời log Gateway, bản ghi Supabase và giao diện để xác định dữ liệu đang dừng ở lớp nào.

5.5. Đánh giá độ trễ truyền dữ liệu HTTPS

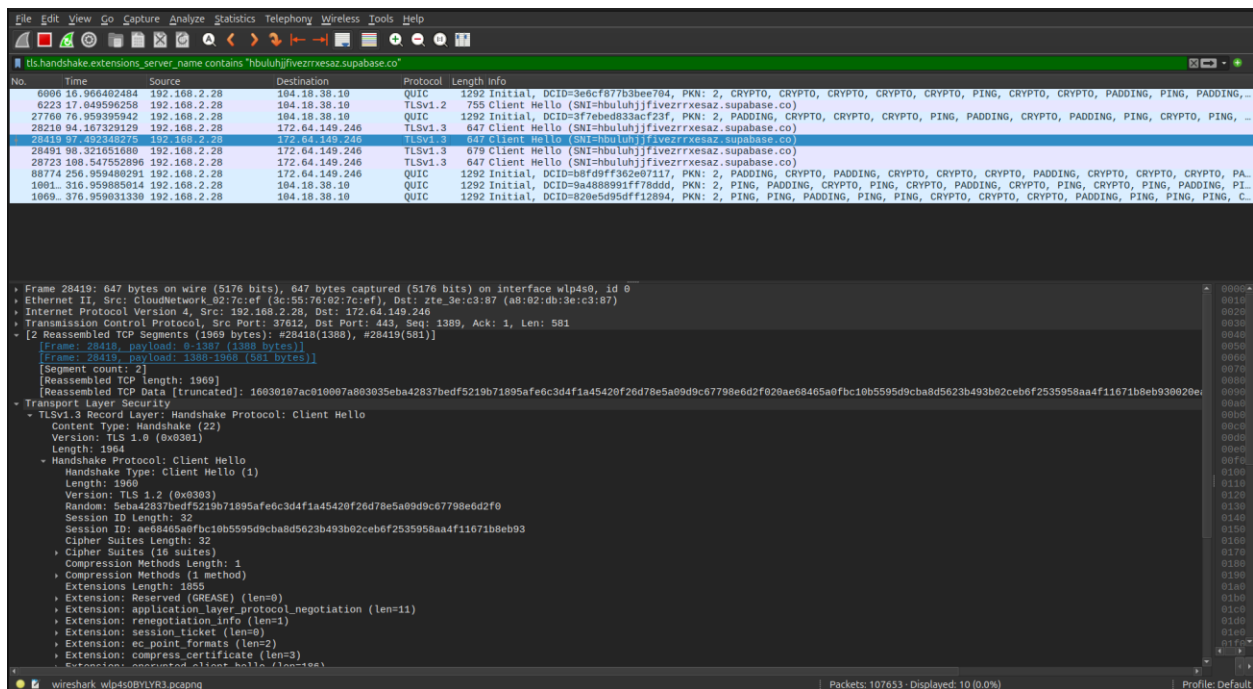
Độ trễ truyền dữ liệu được đánh giá để xác định thời gian phản hồi của tuyến Gateway – Cloud và tuyến giao diện – Cloud. Trong hệ thống này, ESP32-S3 Gateway gửi dữ liệu lên Supabase bằng HTTPS, trong khi Web Dashboard và App giám sát truy vấn dữ liệu đã lưu trên Supabase để hiển thị cho người dùng. Vì vậy, độ trễ được xem xét theo hai hướng: thời gian Gateway gửi dữ liệu lên Cloud và thời gian giao diện truy vấn dữ liệu từ Cloud.

Đối với tuyến Gateway – Supabase, độ trễ được đo trực tiếp trong firmware ESP32-S3 bằng cách ghi timestamp trước và sau hàm gửi HTTPS request. Giá trị này phản ánh thời gian từ lúc Gateway bắt đầu gửi request đến khi nhận phản hồi từ Supabase Edge Function. Đây là chỉ số phù hợp nhất để đánh giá tuyến ESP32-S3 đến Cloud vì phép đo được thực hiện ngay tại thiết bị gửi dữ liệu.

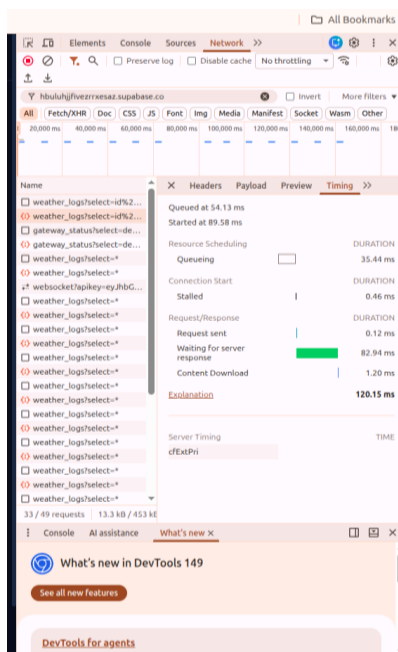
```
[HTTPS_LATENCY] GW STATUS POST start | url=https://hbuluhjjfivezrrxesaz.supabase.co/functions/v1/ingest-weather | payload=360B | rssi=-21 dBm | heap=20  
[HTTPS_LATENCY] GW STATUS POST done | err=ESP_OK | http=200 | latency=4675 ms | rx_len=1 | rssi=-20 dBm | heap=142580
```

Hình 5.15. Log đo độ trễ HTTPS từ ESP32-S3 Gateway đến Supabase

Đối với Web Dashboard, độ trễ truy vấn Supabase được quan sát bằng Chrome DevTools trong tab Network. Công cụ này cho phép xem thời gian DNS, thiết lập kết nối, TLS, thời gian chờ phản hồi và tổng thời gian request. Wireshark được sử dụng bổ sung để quan sát quá trình phân giải DNS, thiết lập TCP/TLS và trao đổi dữ liệu qua cổng 443. Do hệ thống sử dụng HTTPS, Wireshark không hiển thị nội dung JSON hoặc payload ASCON ở dạng rõ, mà chỉ cho thấy thông tin ở mức kết nối và thời điểm trao đổi gói.



Hình 5.16. Gói HTTPS đến Supabase được quan sát bằng Wireshark



Hình 5.17. Thời gian phản hồi request Supabase trên Chrome DevTools

Tuyến đo	Công cụ	Lần 1	Lần 2	Lần 3	Trung bình
ESP32-S3 Gateway → Supabase Edge Function	Log firmware ESP32-S3	5313 ms	3165 ms	5308 ms	4595.33 ms
Web Dashboard → Supabase	Chrome DevTools	120.15 ms	215.20 ms	145.17 ms	160.17 ms

Bảng 5.1. Kết quả đo độ trễ truyền dữ liệu HTTPS

Việc đo nhiều lần giúp kết quả đại diện hơn so với một lần đo đơn lẻ. Độ trễ có thể thay đổi do trạng thái WiFi, thời gian phản hồi của Edge Function, độ trễ Internet, tải hiện tại của Supabase và cơ chế làm mới dữ liệu của giao diện. Với tuyến HTTPS, Wireshark có vai trò xác nhận kết nối đến Supabase và thời điểm trao đổi gói, còn nội dung payload cần được đối chiếu bằng log Gateway và log Edge Function vì dữ liệu đã được bảo vệ bởi TLS.

5.6. Đánh giá mô hình dự báo Multiple Linear Regression

Kết quả kiểm chứng được thực hiện trên dữ liệu thực nghiệm thu thập từ Supabase, tính từ bản ghi đầu tiên đến trước 00:00 ngày 29/06/2026 theo giờ Việt Nam. Tập dữ liệu sau khi lọc có 1853 bản ghi, trong đó 1833 bản ghi có giá trị predicted_temp_2h. Sau khi loại các bản ghi không phù hợp và ghép giá trị dự báo với nhiệt độ thực tế đo được gần thời điểm sau 2 giờ, số mẫu dùng để đánh giá mô hình là 1755 mẫu.

Sau khi lọc dữ liệu từ bản ghi đầu tiên đến trước 00:00 ngày 29/06/2026 theo giờ Việt Nam, tập dữ liệu còn 1853 bản ghi. Trong đó, 1846 bản ghi có health_flag = 0, 1833 bản ghi có giá trị predicted_temp_2h và 1755 mẫu ghép được với nhiệt độ thực tế sau 2 giờ. Các mẫu này được sử dụng để kiểm chứng mô hình MLR đang triển khai trên ESP32-S3 Gateway.

Mô hình đánh giá	Số mẫu đánh giá	MAE (°C)	MSE (°C ²)	RMSE (°C)	R ²	Bias (°C)
MLR đang triển khai trên Gateway	1755	1,8403	7,7937	2,7917	0,7710	0,6160

Bảng 5.2. Kết quả đánh giá sai số của mô hình MLR đang triển khai trên ESP32-S3 Gateway

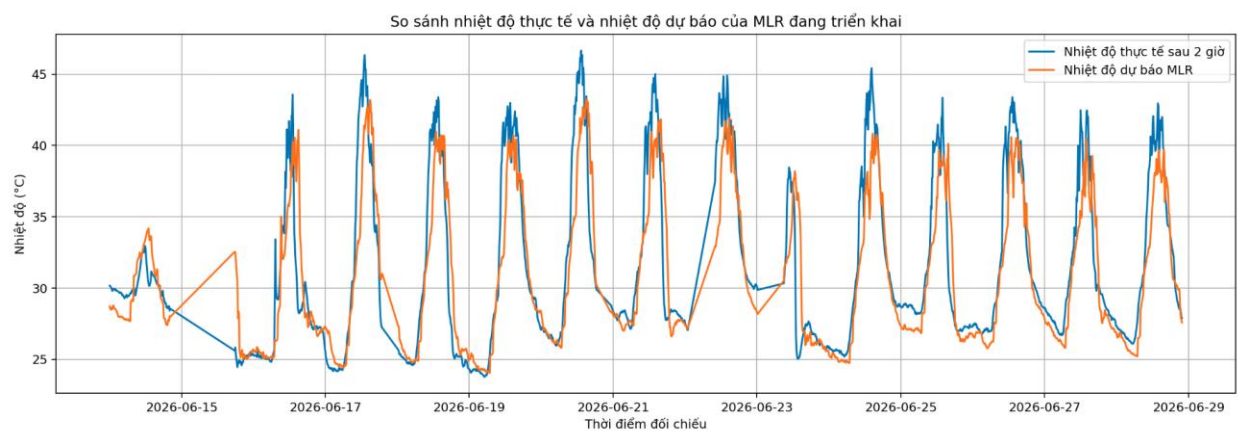
Thời điểm tạo dự báo	Nhiệt độ hiện tại (°C)	Thời điểm đối chiếu	Nhiệt độ dự báo sau 2 giờ (°C)	Nhiệt độ thực tế sau 2 giờ (°C)	Sai lệch thực tế - dự báo (°C)
13/06/2026 22:01:10	30,09	14/06/2026 00:01:10	28,72	30,17	1,45
16/06/2026 06:42:01	25,45	16/06/2026 08:42:01	28,60	30,25	1,65
17/06/2026 22:19:00	27,82	18/06/2026 00:19:00	27,32	25,67	-1,65
19/06/2026 08:32:06	28,31	19/06/2026 10:32:06	31,56	37,19	5,63
20/06/2026 23:15:04	28,80	21/06/2026 01:15:04	28,18	27,88	-0,30
22/06/2026 18:21:06	33,35	22/06/2026 20:21:06	30,55	30,95	0,40
24/06/2026 13:09:51	42,54	24/06/2026 15:09:51	39,46	42,10	2,64

25/06/2026 23:19:57	27,28	26/06/2026 01:19:57	26,88	27,38	0,50
27/06/2026 09:41:34	32,99	27/06/2026 11:41:34	34,52	39,17	4,65
28/06/2026 19:51:23	29,58	28/06/2026 21:51:23	27,58	27,90	0,32

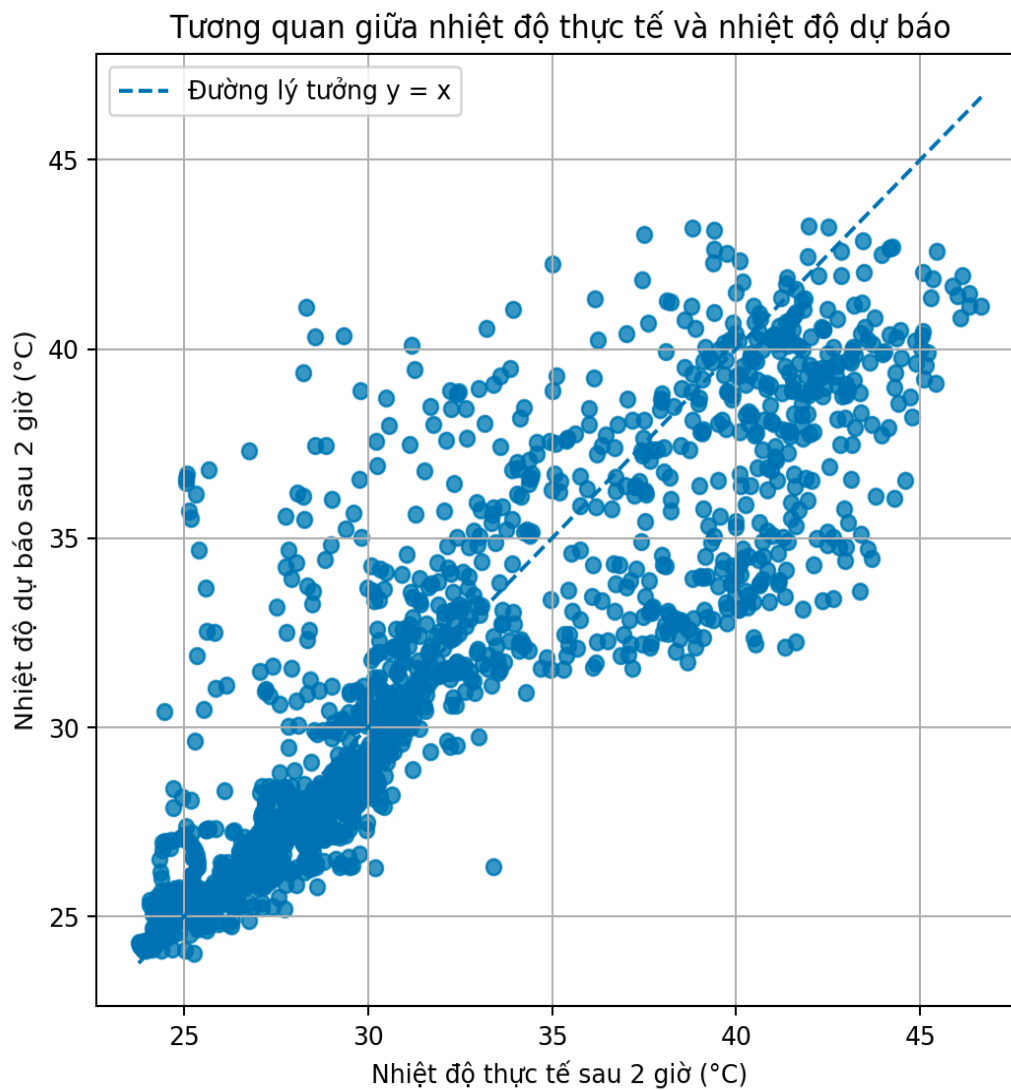
Bảng 5.3. Một số mẫu đại diện đối chiếu giữa nhiệt độ dự báo và nhiệt độ thực tế sau 2 giờ

Kết quả trong Bảng 5.2 cho thấy mô hình MLR đang triển khai trên ESP32-S3 Gateway đạt MAE 1,8403 °C, RMSE 2,7917 °C và R^2 đạt 0,7710 trên 1755 mẫu đánh giá. Giá trị R^2 cho thấy mô hình vẫn mô tả được xu hướng biến thiên nhiệt độ trong dữ liệu thực nghiệm. Tuy nhiên, MAE và RMSE còn tương đối lớn, đặc biệt tại các thời điểm nhiệt độ thực tế thay đổi nhanh.

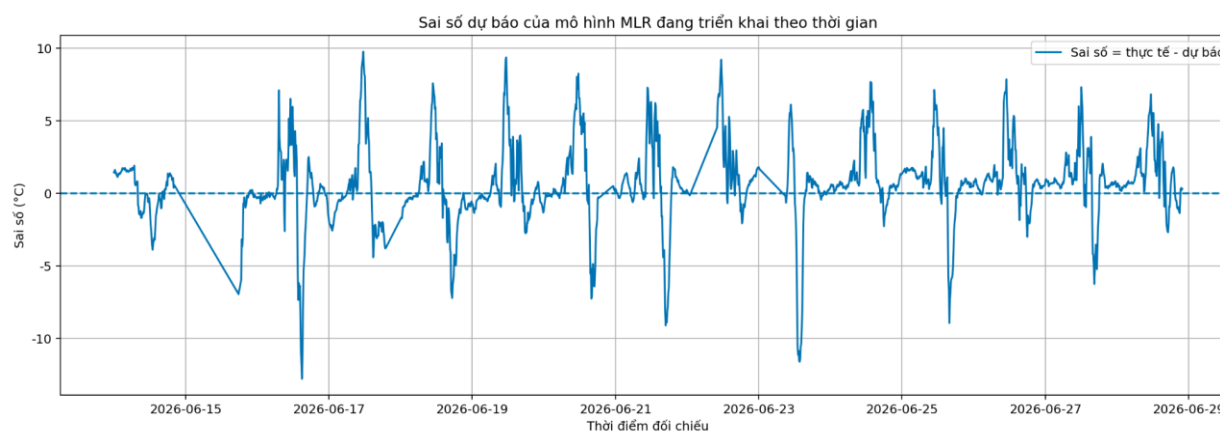
Bias đạt 0,6160 °C, trong đó bias được tính theo công thức nhiệt độ thực tế trừ nhiệt độ dự báo. Giá trị bias dương cho thấy nhiệt độ thực tế sau 2 giờ có xu hướng cao hơn giá trị dự báo, tức mô hình còn dự báo thấp hơn thực tế trong một số khoảng thời gian. Bảng 5.3 cho thấy một số mẫu đối chiếu cụ thể giữa nhiệt độ dự báo và nhiệt độ thực tế, qua đó có thể quan sát trực tiếp độ lệch của mô hình tại từng thời điểm.



Hình 5.18. So sánh nhiệt độ thực tế và nhiệt độ dự báo của mô hình MLR đang triển khai



Hình 5.19. Tương quan giữa nhiệt độ thực tế và nhiệt độ dự báo



Hình 5.20. Sai số dự báo của mô hình MLR theo thời gian

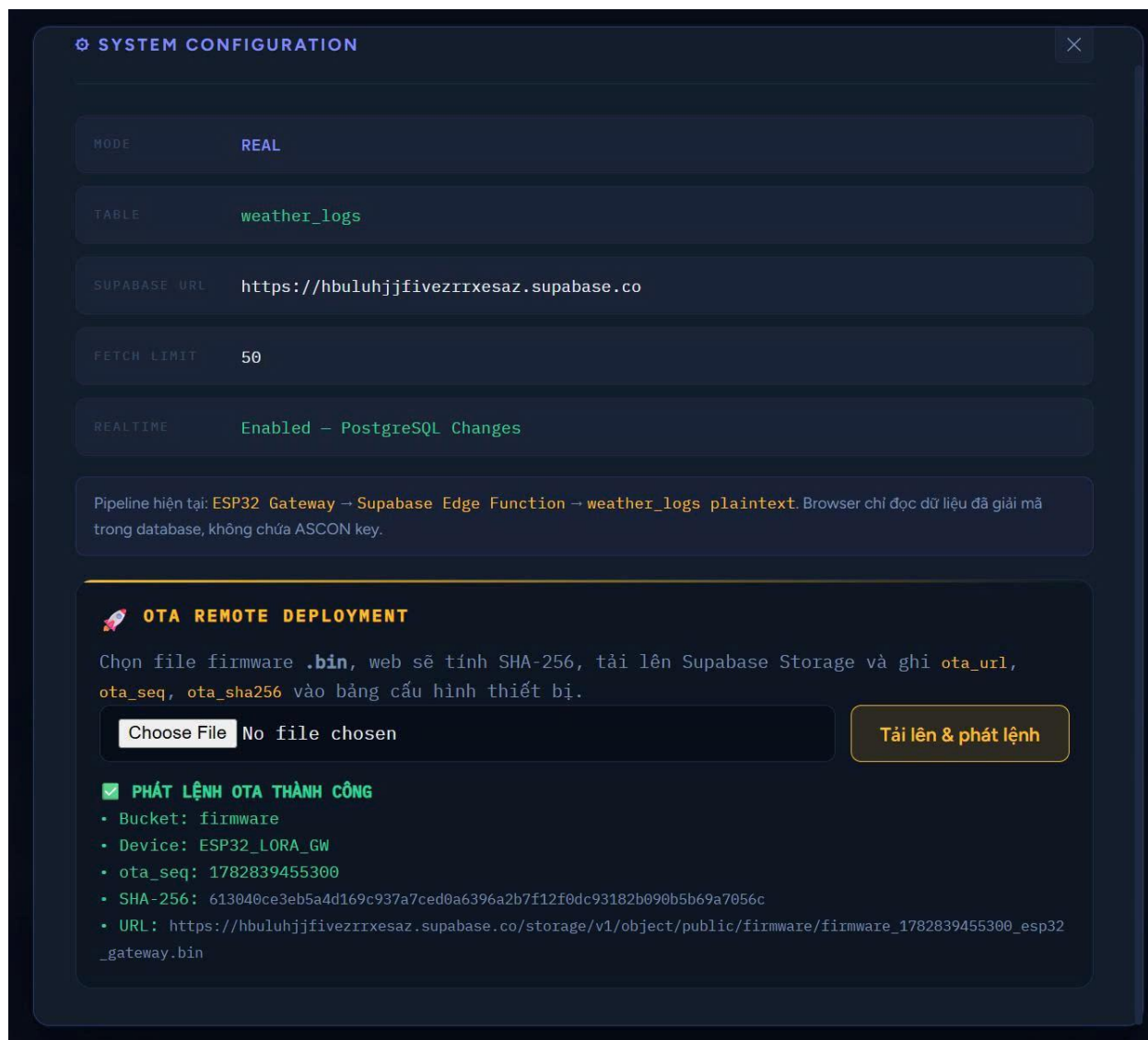
Ở các khoảng thời gian nhiệt độ biến thiên chậm, đặc biệt vào ban đêm và rạng sáng, đường dự báo có xu hướng gần với đường nhiệt độ thực tế hơn. Ngược lại, trong khoảng khoảng 10h đến 15h, sai số xuất hiện nhiều đỉnh lớn hơn. Điều này cho thấy các đặc trưng hiện tại của mô hình, gồm nhiệt độ, độ ẩm, áp suất và đặc trưng thời gian, vẫn chưa đủ để mô tả tốt các pha chuyển tiếp nhiệt độ nhanh trong ngày.

Sai lệch lớn nhất ghi nhận được tại dự báo tạo lúc 13:00:11 ngày 16/06/2026, với thời điểm đối chiếu sau 2 giờ là 15:00:11. Ở mẫu này, mô hình dự báo nhiệt độ sau 2 giờ là 41,10 °C, trong khi nhiệt độ thực tế gần thời điểm đối chiếu là 28,33 °C, tạo ra sai lệch - 12,77 °C. Sai lệch âm cho thấy mô hình dự báo cao hơn thực tế tại thời điểm này. Đây là trường hợp cho thấy mô hình dễ bị lệch khi nhiệt độ thực tế giảm nhanh sau giai đoạn đạt đỉnh.

Nhìn chung, mô hình MLR hiện tại có thể dùng để tham khảo xu hướng nhiệt độ ngắn hạn, nhưng độ chính xác còn phụ thuộc mạnh vào thời điểm trong ngày. Các giai đoạn nhiệt độ ổn định cho kết quả dự báo tốt hơn, trong khi các giai đoạn chuyển tiếp nhanh quanh buổi trưa và đầu giờ chiều tạo ra sai số lớn. Vì vậy, hướng cập nhật tiếp theo là bổ sung thêm dữ liệu thực nghiệm dài hạn và xem xét thêm các đặc trưng thể hiện tốc độ biến thiên nhiệt độ trước khi hiệu chỉnh lại mô hình.

5.7. Kết quả cập nhật firmware OTA

Cơ chế OTA được đánh giá bằng cách upload firmware mới từ Web Dashboard lên Supabase Storage, sau đó để ESP32-S3 Gateway tự kiểm tra và cập nhật. Khi người dùng chọn file firmware, dashboard tính SHA-256 của file, upload file .bin vào bucket firmware và cập nhật các trường ota_url, ota_seq, ota_sha256 trong bảng device_configs. Các trường này là cơ sở để Gateway xác định có firmware mới hay không.



Hình 5.21. Web Dashboard upload firmware OTA lên Supabase Storage

ESP32-S3 Gateway đọc cấu hình OTA theo chu kỳ. Nếu `ota_seq` mới hơn bản đã lưu và `ota_sha256` khác với firmware đã áp dụng, Gateway tải file firmware bằng HTTPS, ghi vào phân vùng OTA và khởi động lại sau khi cập nhật thành công. Thông tin firmware đã áp dụng được lưu trong NVS bằng các khóa `last_seq`, `last_sha` và `last_url`. Nhờ đó, nếu Cloud vẫn giữ cùng một bản firmware, Gateway sẽ bỏ qua và không cập nhật lặp lại.

xác thực và giải mã dữ liệu, khôi phục giá trị vật lý, tạo giá trị dự báo nhiệt độ, mã hóa lại payload bằng ASCON key2 và gửi dữ liệu lên Supabase. Edge Function tiếp nhận, kiểm tra và ghi dữ liệu vào PostgreSQL. Web Dashboard và App giám sát hiển thị dữ liệu quan trắc, biểu đồ, trạng thái Gateway và các thông tin phục vụ vận hành.

Về mặt chức năng, hệ thống đã đáp ứng các mục tiêu chính của đề tài: thu thập dữ liệu môi trường, truyền dữ liệu bằng LoRa, bảo vệ dữ liệu bằng ASCON-128a, xử lý tại Gateway, lưu trữ trên Supabase, hiển thị trên Web/App, dự báo nhiệt độ ngắn hạn và cập nhật firmware từ xa. Các kết quả này cho thấy kiến trúc STM32 Sensor Node – ESP32-S3 Gateway – Supabase – Web/App phù hợp với bài toán quan trắc thời tiết cục bộ có kết nối Cloud.

Bên cạnh các kết quả đạt được, hệ thống vẫn còn một số giới hạn. Tuyến LoRa mới được đánh giá ở mức truyền nhận frame hợp lệ, chưa đo đầy đủ tỷ lệ mất gói theo khoảng cách và vật cản. Mô hình MLR mới phục vụ khảo sát xu hướng nhiệt độ ngắn hạn, chưa phải mô hình dự báo tối ưu trong mọi điều kiện thời tiết. Độ trễ Gateway – Cloud và Web/App – Supabase cần được đo nhiều lần để có giá trị trung bình, nhỏ nhất và lớn nhất. Ngoài ra, hệ thống cần tiếp tục được vận hành dài hạn để đánh giá độ ổn định của nguồn, cảm biến, WiFi, Supabase và OTA.

Nhìn chung, kết quả thực nghiệm cho thấy hệ thống đáp ứng được định hướng thiết kế ban đầu. Việc tách hệ thống thành các lớp Sensor Node, Gateway, Cloud và giao diện giám sát giúp quá trình thu thập, truyền, xử lý, lưu trữ và hiển thị dữ liệu được tổ chức rõ ràng. Cách tổ chức này cũng hỗ trợ quá trình xác định lỗi trong vận hành, vì dữ liệu có thể được đối chiếu tại nhiều điểm từ trạm đo đến giao diện người dùng.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Kết luận

Đề tài đã xây dựng hệ thống quan trắc thời tiết cục bộ theo kiến trúc Sensor Node – Gateway – Cloud. Trong kiến trúc này, STM32 Sensor Node đảm nhiệm nhiệm vụ thu thập dữ liệu môi trường tại vị trí quan trắc, ESP32-S3 Gateway đảm nhiệm nhiệm vụ tiếp nhận và xử lý dữ liệu từ tuyến LoRa, còn Supabase và Web Dashboard đảm nhiệm nhiệm vụ lưu trữ, quản lý và hiển thị dữ liệu cho người dùng. Cách tổ chức này giúp hệ thống phân chia rõ các lớp chức năng gồm lớp thiết bị đo, lớp truyền thông cục bộ, lớp xử lý trung gian, lớp lưu trữ và lớp giao diện giám sát.

Về phần cứng, hệ thống đã triển khai được thiết bị đo ngoài trời sử dụng vi điều khiển STM32F103C8T6, cảm biến SHT30, BMP388, INA219 và module LoRa E32. Nguồn của thiết bị đo được thiết kế theo hướng hoạt động độc lập, sử dụng tấm pin năng lượng mặt trời, module sạc CN3791, pin Lithium EVE 1S, mạch bảo vệ pin và bộ chuyển đổi TPS63020. Ngoài ra, thiết bị còn được bố trí trong vỏ bảo vệ, kết hợp với lồng Stevenson và van thở nhằm hỗ trợ điều kiện lắp đặt ngoài môi trường.

Về phần Gateway, hệ thống đã triển khai ESP32-S3 Gateway làm bộ trung tâm tiếp nhận dữ liệu từ tuyến LoRa và kết nối lên nền tảng Cloud thông qua WiFi. Gateway có khả năng nhận frame từ module LoRa E32, kiểm tra kích thước frame, xác thực và giải mã dữ liệu bằng ASCON-128a, khôi phục các giá trị cảm biến, tạo giá trị dự báo nhiệt độ cục bộ và gửi dữ liệu lên Supabase thông qua HTTPS. Bên cạnh đó, gateway còn hỗ trợ heartbeat trạng thái, BLE provisioning và OTA firmware update.

Về firmware STM32 Sensor Node, hệ thống đã thực hiện được chu kỳ vận hành phù hợp với thiết bị đo dùng nguồn pin. Node có khả năng đánh thức theo chu kỳ, đọc dữ liệu cảm biến, kiểm tra trạng thái lỗi, đóng gói payload 10 byte, mã hóa dữ liệu bằng ASCON-128a và tạo frame LoRa 18 byte để truyền về gateway. Việc sử dụng RTC alarm, Standby mode và Independent Watchdog giúp node giảm thời gian hoạt động liên tục, đồng thời duy trì khả năng vận hành theo chu kỳ.

Về bảo mật dữ liệu, hệ thống đã áp dụng cơ chế mã hóa xác thực bằng ASCON-128a trên hai tuyến truyền. Tuyến STM32 Sensor Node đến ESP32-S3 Gateway sử dụng khóa thứ nhất để bảo vệ payload truyền qua LoRa. Sau khi giải mã và xử lý, gateway mã hóa lại dữ liệu bằng khóa thứ hai trước khi gửi lên Supabase. Cách tổ chức này giúp tách biệt phạm vi bảo vệ giữa tuyến truyền cục bộ và tuyến gửi dữ liệu lên Cloud.

Về xử lý dữ liệu, ESP32-S3 Gateway đã tích hợp mô hình Multiple Linear Regression để tạo giá trị dự báo nhiệt độ cục bộ sau 2 giờ. Mô hình này sử dụng các thông tin đầu vào như nhiệt độ, độ ẩm, áp suất khí quyển và thời gian để tạo giá trị `predicted_temp_2h`. Điều này cho thấy gateway không chỉ đóng vai trò chuyển tiếp dữ liệu, mà còn có khả năng thực hiện một phần xử lý cục bộ trước khi gửi dữ liệu lên nền tảng trực tuyến.

Về nền tảng Cloud và giao diện giám sát, Supabase đã được sử dụng để tiếp nhận, kiểm tra, giải mã và lưu trữ dữ liệu từ gateway. Web Dashboard có khả năng hiển thị dữ liệu môi trường, giá trị dự báo, trạng thái gateway, biểu đồ lịch sử và thông tin OTA. Nhờ đó, người dùng có thể theo dõi trạng thái hệ thống thông qua giao diện trực tuyến thay vì chỉ quan sát dữ liệu ở log thiết bị.

Về cơ chế OTA, ESP32-S3 Gateway đã có khả năng nhận thông tin firmware từ Supabase, tải file firmware từ Storage, cập nhật firmware và khởi động lại thiết bị. Việc sử dụng `ota_seq`, `ota_sha256` và lưu trạng thái OTA đã áp dụng vào NVS giúp hạn chế hiện tượng cập nhật lặp lại cùng một firmware. Cơ chế này có ý nghĩa trong vận hành thực tế vì gateway có thể được cập nhật từ xa mà không cần nạp firmware trực tiếp qua cổng USB.

Kết quả thực nghiệm cho thấy hệ thống đã đáp ứng được các yêu cầu cơ bản của một mô hình quan trắc thời tiết cục bộ có kết nối Cloud. Dữ liệu có thể được tạo ra tại thiết bị đo, truyền qua LoRa, xử lý tại gateway, lưu trữ trên Supabase và hiển thị trên Web Dashboard. Tuy nhiên, hệ thống vẫn còn một số giới hạn như thời gian thử nghiệm ngoài môi trường chưa đủ dài, tuyến LoRa chưa được đánh giá định lượng theo nhiều khoảng cách, mô hình dự báo chưa được so sánh sâu với dữ liệu chuẩn và logic cảnh báo trên giao diện vẫn cần tiếp tục hoàn thiện.

6.2. Hướng phát triển

Trong giai đoạn tiếp theo, hệ thống có thể được phát triển theo hướng mở rộng thời gian thực nghiệm ngoài môi trường. Việc vận hành trong thời gian dài sẽ giúp đánh giá rõ hơn độ ổn định của khối nguồn, cảm biến, module LoRa, vỏ bảo vệ, lồng Stevenson và van thở. Dữ liệu dài hạn cũng giúp phân tích ảnh hưởng của thay đổi thời tiết, chu kỳ ngày đêm, mức pin và điều kiện lắp đặt đến kết quả đo.

Tuyến truyền LoRa cần được đánh giá định lượng theo nhiều khoảng cách và điều kiện vật cản khác nhau. Các chỉ số như tỷ lệ mất gói, tỷ lệ frame hợp lệ, độ trễ truyền dữ liệu và độ ổn định theo vị trí anten cần được thu thập để xác định phạm vi hoạt động phù hợp của hệ thống. Việc bổ sung cơ chế ghi nhận số gói mất hoặc thống kê `frame_counter` tại gateway cũng có thể giúp quá trình đánh giá tuyến truyền rõ ràng hơn.

Phần nguồn của STM32 Sensor Node có thể tiếp tục được tối ưu để phù hợp hơn với yêu cầu vận hành độc lập. Các hướng cải tiến gồm đo dòng tiêu thụ ở từng trạng thái, tối ưu thời gian thức của vi điều khiển, giảm thời gian bật module LoRa, cải thiện thuật toán quản lý năng lượng và bổ sung cơ chế đánh giá trạng thái pin chính xác hơn. Khi có dữ liệu tiêu thụ thực tế, hệ thống có thể ước lượng thời gian hoạt động của node theo từng điều kiện sạc và xả.

Mô hình dự báo nhiệt độ cục bộ có thể được mở rộng bằng cách sử dụng tập dữ liệu lớn hơn và so sánh với các mô hình khác. Multiple Linear Regression phù hợp với giai đoạn triển khai ban đầu do có cấu trúc đơn giản và dễ nhúng vào gateway, nhưng các mô hình khác như Random Forest, Gradient Boosting hoặc mạng nơ-ron nhẹ có thể được xem xét nếu hệ thống có thêm dữ liệu huấn luyện. Việc so sánh sai số dự báo theo nhiều mô hình sẽ giúp đánh giá chính xác hơn hiệu quả của khối xử lý dữ liệu tại gateway.

Web Dashboard có thể được hoàn thiện thêm ở phần logic trạng thái và cảnh báo. Các trạng thái như gateway online/offline, LoRa có dữ liệu mới hay không, Sensor Node chưa đến chu kỳ truyền, Cloud ghi dữ liệu chậm hoặc OTA đang chờ xử lý cần được phân biệt rõ hơn. Việc tách các trạng thái này sẽ giúp người dùng xác định đúng vị trí phát sinh lỗi trong chuỗi Sensor Node – Gateway – Cloud.

Cơ chế OTA có thể được mở rộng theo hướng quản lý nhiều thiết bị và nhiều phiên bản firmware. Khi hệ thống có nhiều gateway hoặc nhiều node, cần có cơ chế phân nhóm thiết bị, quản lý firmware theo device_id, theo phiên bản phần cứng và theo trạng thái triển khai. Ngoài ra, hệ thống có thể bổ sung cơ chế rollback firmware nếu bản cập nhật mới không hoạt động ổn định sau khi OTA.

Về bảo mật, hệ thống có thể tiếp tục tăng cường cơ chế quản lý khóa, xác thực thiết bị và phân quyền truy cập dữ liệu. Các khóa ASCON nên được quản lý theo từng thiết bị thay vì dùng chung cho toàn hệ thống. Supabase Edge Function và Web Dashboard cũng cần được cấu hình phân quyền chặt chẽ hơn để hạn chế truy cập trái phép vào dữ liệu hoặc chức năng OTA.

Về khả năng mở rộng, hệ thống có thể phát triển từ một Sensor Node thành mạng nhiều node. Khi số lượng node tăng lên, gateway cần có cơ chế phân biệt node_id, quản lý frame_counter theo từng node, lưu dữ liệu theo từng vị trí quan trắc và hiển thị nhiều điểm đo trên dashboard. Đây là hướng phát triển phù hợp nếu hệ thống được mở rộng từ mô hình thử nghiệm cục bộ sang mạng quan trắc thời tiết phân tán.

TÀI LIỆU THAM KHẢO

- [1] M. G. dos Santos, D. Ameyed, F. Petrillo, F. Jaafar, and M. Cheriet, "Internet of Things Architectures: A Comparative Study," *arXiv preprint arXiv:2004.12936*, 2020.
- [2] STMicroelectronics, "STM32F103C8 Mainstream Performance Line MCU," STMicroelectronics Datasheet, 2020.
- [3] Espressif Systems, "ESP32-S3 Series SoC Datasheet," Espressif Systems, 2022.
- [4] Espressif Systems, "ESP-IDF FreeRTOS API Reference," Espressif Systems Documentation, 2023.
- [5] NXP Semiconductors, "UM10204: I2C-bus Specification and User Manual," NXP Semiconductors User Manual, Rev. 7.0, 2021.
- [6] Wikipedia contributors, "Universal Asynchronous Receiver-Transmitter," *Wikipedia, The Free Encyclopedia*, 2024.
- [7] Sensirion, "Datasheet SHT3x-DIS: Humidity and Temperature Sensor," Sensirion AG Datasheet, 2023.
- [8] Bosch Sensortec, "BMP388: Digital Pressure Sensor Datasheet," Bosch Sensortec GmbH Datasheet, 2020.
- [9] Texas Instruments, "INA219: Zero-Drift, Bidirectional Current/Power Monitor With I2C Interface," Texas Instruments Datasheet, 2015.
- [10] Chengdu Ebyte Electronic Technology Co., Ltd., "E32-433T30D User Manual: SX1278 433MHz 1W DIP Wireless Module," Chengdu Ebyte Electronic Technology Co., Ltd. User Manual, Rev. 1.9, 2019.
- [11] Conserve Microelectronics, "CN3791: 4A, 12V Solar Powered Lead-Acid Battery Charger IC with MPPT," Conserve Microelectronics Datasheet, 2016.
- [12] CNPNJI, "Nylon Air Vent Plug Protective Vent Plug Specification," Wenzhou CNPNJI Enterprise Co., Ltd. Datasheet, 2022.
- [13] Cults3D, "Mini Stevenson Weather Shelter for Temperature or Humidity Sensor 3D Model," Cults3D Design Repository, 2023.
- [14] National Institute of Standards and Technology (NIST), "Ascon-Based Lightweight Cryptography Standards for Constrained Devices: Authenticated Encryption, Hash, and Extendable Output Functions," *NIST Special Publication 800-232*, 2024.

- [15] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," RFC 8446, Internet Engineering Task Force (IETF), 2018.
- [16] Supabase, "Database Overview - Supabase Documentation," Supabase Inc., 2024.
- [17] Espressif Systems, "Wi-Fi Provisioning API Reference," Espressif Systems Documentation, 2022.
- [18] Espressif Systems, "Over The Air Updates (OTA) API Reference," Espressif Systems Documentation, 2023.
- [19] I. Gupta, H. Mittal, D. Rikhari, and A. K. Singh, "MLRM: A Multiple Linear Regression based Model for Average Temperature Prediction of A Day," *arXiv preprint arXiv:2203.05835*, 2022.
- [20] R. Prudden, S. Adams, D. Kangin, N. Robinson, S. Ravuri, S. Mohamed, and A. Arribas, "A Review of Radar-Based Nowcasting of Precipitation and Applicable Machine Learning Techniques," *arXiv preprint arXiv:2005.04988*, 2020.
- [21] Scikit-learn Developers, "Time-related Feature Engineering with Cyclical Features," Scikit-learn Example Documentation, 2024.
- [22] R. J. Hyndman and G. Athanasopoulos, "Dynamic Harmonic Regression," in *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021.
- [23] C. T. N. Quynh, N. T. Ngoc, L. T. Ninh, and P. T. T. An, "Throughput Optimization of the ASCON Lightweight Cryptographic Algorithm on IoT Devices," *Journal of Science and Technology on Information Security*, vol. 3, no. 26, pp. 92–100, Dec. 2025.